

EASY-BANK



שם בית הספר: תיכון אלון רמת השרון

שם העבודה: "EASY-BANK"

שם התלמידה: תמר איל

ת.ז. התלמידה: 326463718

שם המנחה: עודי רז, עידן פלדברג, רום רפסון וקובי שורצמן

שם החלופה: "הגנת סייבר"

תאריך ההגשה: 24.05.2025

תוכן עניינים

פרק 1- מבוא: 5.....

5..... ייזום:

- 5..... תיאור ראשוני של המערכת:
- 5..... הגדרה הלקוח:
- 5..... הגדרת מטרות:
- 5..... בעיות תועלת וחסכוניות:
- 6..... סקירת פתרונות קיימים:
- 6..... סקירת טכנולוגית הפרויקט:
- 6..... תיחום הפרויקט:

7..... פירוט תיאור המערכת:

- 7..... תיאור מפורט של המערכת:
- 7..... פירוט היכולות של המשתמשים:
- 8..... פירוט בדיקות:
- 10..... תכנון וניהול לוח זמנים:
- 11..... הסיכונים בפרויקט והדרכים להתמודד איתם:

פרק 2- תיאור תחום הידע: 12.....

- 12..... הרשמה למערכת:
- 12..... אימות כתובת המייל על ידי קוד חד פעמי שנשלח למייל לפני הרשמה:
- 12..... מסך התחברות:
- 12..... אימות דו שלבי- שליחת קוד חד פעמי למייל:
- 13..... העברת כספים למשתמש אחר:
- 13..... בקשת העברה ממשתמש אחר:
- 13..... אישור העברה למשתמש אחר:
- 13..... צפייה בהעברות קודמות:
- 14..... צפייה בגרפים של ההוצאות השנתיות בקטגוריה כלשהי:
- 14..... בקשת יעוץ פיננסי מבינה מלאכותית:

פרק 3- ארכיטקטורה של הפרויקט: 14.....

- 14..... תיאור החומרה:
- 15..... שרטוט המציג קשרים אלו:
- 15..... תיאור הטכנולוגיות הרלוונטיות:

17..... תיאור זרימת המידע:

- 17..... מסך הרשמה:
- 18..... מסך התחברות:
- 19..... מסך אימות דו שלבי (לאחר הרשמה):

- 20..... מסך אימות דו שלבי (לאחר התחברות ראשונית):
- 21..... מסך ראשי- העברת כספים:
- 22..... מסך ראשי- בקשת העברה:
- 23..... מסך ראשי- אישור העברה:
- 24..... מסך ראשי- צפייה בהעברות קודמות:
- 25..... מסך ראשי- צפייה בגרפים של ההוצאות השנתיות:

26..... תיאור האלגוריתמים המרכזיים בפרויקט:

- 26..... אלגוריתם התחברות למערכת:
- 26..... אלגוריתם הרשמה למערכת:
- 27..... אלגוריתם למניעת הצפת משתמשים:
- 27..... אלגוריתם ההצפנה:
- 28..... תיאור סביבת הפיתוח:
- 29..... תיאור פרוטוקול התקשורת:
- 29..... פירוט כלל ההודעות הזורמות במערכת:

34..... תיאור מסכי המערכת:

- 34..... מסך התחברות:
- 34..... מסך רישום:
- 35..... מסך אימות דו שלבי:
- 35..... מסך העברת כספים:
- 36..... מסך בקשת העברה:
- 36..... מסך אישור העברה:
- 37..... מסך צפייה בהעברות קודמות:
- 37..... מסך צפייה בגרפי הוצאות שנתיות לפי נושאים:
- 38..... מסך קבלת יעוץ מבינה מלאכותית:

39..... תרשים זרימה של המסכים בפרויקט:

40..... תיאור מבני הנתונים:

- 40..... קובץ ClientFileData.txt
- 40..... קובץ serverFileData.txt
- 41..... קובץ dataBaseTamarEyal.db
- 43..... סקירת חולשות ואיומים:

45..... פרק 4- מימוש הפרויקט:

45..... חלק א:

- 45..... סקירת כל המודולים ומחלקות המרכיבים את המערכת:

48..... מחלקות שאני מפתחת:

- 48..... מחלקה- Client

49.....	מחלקה- ClientGUIFirstPage
50.....	מחלקה- secondPageGUI
52.....	מחלקה- GmailEmailSender
53.....	מחלקה- Encryptions
53.....	מחלקה- Encryption1
54.....	מחלקה- Encryption2
54.....	מחלקה- Encryption3
54.....	מחלקה- Server
57.....	מחלקה- Database

59..... חלק ב:

59.....	אלגוריתם התחברות למערכת:
61.....	אלגוריתם הרשמה למערכת:
63.....	אלגוריתם למניעת הצפת משתמשים:
64.....	אלגוריתם ההצפנה:

65..... חלק ג:

65.....	מסמך בדיקות מלא:
68.....	בדיקות נוספות שביצעתי למערכת:

69..... פרק 5- מדריך למשתמש:

69.....	פירוט של כל קבצי המערכת:
---------	--------------------------

70..... התקנת המערכת:

70.....	סביבת העבודה:
70.....	פירוט הכלים הנדרשים:
70.....	רשת:
70.....	משתמשי המערכת:

71..... אופן הפעלת המערכת:

76..... הרחבות נוספות:

76.....	תרשים מחלקות המתאר קשרי ירושה:
---------	--------------------------------

76..... האלגוריתמיקה והמתמטיקה של ההצפנות:

76.....	:RSA
77.....	:AES
78.....	:DES

80..... פרק 6- רפלקציה:

81..... פרק 7- נספחים:

פרק 1- מבוא:

ייזום:

תיאור ראשוני של המערכת:

הפרויקט שלי הוא פלטפורמה של בנק שמאפשרת למשתמשים לבצע פעולות דיגיטליות ללא צורך להגיע פיזית לבנק על מנת לבצע פעולות אלו. הסיבה שבחרתי בפרויקט זה היא שכלל שעובדות השנים כך העולם מתחדש ומתקדם ולדעתי יש צורך בחדשנות דיגיטלית על מנת להבטיח נוחות מרבית למשתמשים.

השם שבחרתי לפרויקט הוא **Easy-Bank**. בחרתי בשם זה מאחר והוא מבטא את מטרת הפרויקט בצורה מדויקת- בנק ידידותי למשתמש, נגיש ומידי. הוא מציע טכנולוגיה מתקדמת לשירות המשתמשים בצורה הפשוטה ביותר.

הפרויקט יכול ממשק משתמש שבו המשתמש יכול לבצע פעולות בנקאיות כמו העברת כספים, בקשת העברה ממשתמש אחר, אישור ההעברה, צפייה בהעברות קודמות, צפייה בגרף הוצאות שנתיות של קטגוריה מסוימת וקבלת יעוץ מ-AI לגבי הוצאות שנתיות של קטגוריה מסוימת. בנוסף תהיה אבטחה מתקדמת הכוללת סיסמא אישית וקוד חד פעמי שישלח למייל בכל כניסה. לאחר שמשתמש ירשם לאתר (בכניסה הראשונית) או יכנס באמצעות הסיסמא האישית והסיסמא החד פעמית (בהתחברות לאתר), המשתמש יוכל לצפות ביתרה שנשארה לו ובהיסטורית ההעברות שלו. בנוסף הוא יוכל לבצע פעולות בממשק.

האתגרים המרכזיים בבניית הפרויקט הם אבטחת המידע והתמודדות עם מספר רב של משתמשים. מאחר ובבנק יש מידע רגיש ואישי על כל אחד מהלקוחות שלו יש חשיבות יתרה על שמירה על אתר מאובטח. בנוסף יש צורך לאפשר שימוש של מספר רב של משתמשים בו זמנית.

הגדרה הלקוח:

קהל היעד של הפרויקט הוא לקוחות פרטיים שרוצים שירות בנקאי דיגיטלי ומהיר, וצעירים שפותחים חשבון בנק בפעם הראשונה ויש להם ידע בסיסי בלבד בכל הנוגע לפעולות בבנק.

הגדרת מטרות:

מטרת הפרויקט היא לאפשר למשתמש לנהל את חשבונות הבנק, לבצע פעולות פיננסיות ולעקוב אחרי ההוצאות שלו בצורה נוחה וידידותית. בנוסף ליצור מערכת בנק מאובטחת ואמינה שניתן להתחבר אליה מכל מקום בעולם בכל שעה ביום. אבטחת המידע בבנק צריכה להיות ברמה גבוהה מאוד על מנת לשמור על הפרטים האישיים של כל אחד מהמשתמשים. בנוסף יש צורך להגן מפני מתקפות סייבר שעלולות לנסות לגנוב את המידע של המשתמשים. בנוסף יש לשמור על פרטיות המשתמשים ולמנוע שימוש לא ראוי בנתונים.

בעיות תועלת וחסכונות:

בעידן של היום, לקוחות מצפים לבצע פעולות בנקאיות בצורה פשוטה, מהירה ומאובטחת- אך ישנן מערכות רבות שסובלות מרמות אבטחה לא מספקות וממשקים שאינם ידידותיים למשתמש. לכן, הצורך לממשק בנק שמגן מפני תקיפות סייבר, זליגת מידע והתחזות משתמשים הולך וגדל- זוהי הבעיה שהפרויקט שלי פותר.

מטרת המערכת היא לספק למשתמשים פלטפורמה בנקאית נוחה ובטוחה, שתאפשר להם לבצע מגוון רחב של פעולות- כגון העברת כספים, הגשת בקשות, קבלת אישורים, צפייה בנתונים סטטיסטיים ודוחות שנתיים, בצורה קלה לשימוש ומאובטחת.

למערכת צפויה להיות תועלת רבה כמו: נוחות בשימוש בממשק הגרפי, אבטחת מידע ברמה גבוהה, שיפור ההתנהלות הכלכלית של המשתמשים, יכולת מעקב אחר הוצאות ועוד.

השירותים שהמערכת תיתן: רישום והתחברות עם אימות דו שלבי (קוד חד פעמי ממיייל וסיסמא אישית), ניהול חשבון אישי תוך צפייה בהיסטורית הפעולות, העברת כספים בין משתמשים, בקשות העברה ממשתמשים אחרים, אישור ודחייה של בקשות ממשתמשים אחרים, הצגת סטטיסטיקות גרפיות על פעילות כלכלית, מערכת למניעת התחזויות- לאחר מספר ניסיונות כושלים של התחברות המערכת חוסמת את הIP.

סקירת פתרונות קיימים:

כיום בשוק יש מערכות מקבילות עם אותן אפשרויות לפעולות. אך רובן לא מעוצבות לנוחות המשתמש ולא קיימת בהן מערכת AI מידית ליעוץ בהוצאות שנתיות. דבר זה מייחד את הפרויקט שלי משאר היישומים הקיימים בשוק.

סקירת טכנולוגית הפרויקט:

בפרויקט שלי אין טכנולוגיה חדשה ובלתי מוכרת אך יש שילוב של מספר רב של טכנולוגיות מוכרות, לדוגמא: הצפנת RSA, הצפנת DES, הצפנת AES וקריאה לשירותים חיצוניים כמו API של ChatGPT ושליחת מיילים אוטומטית.

בפרויקט קיימים סייגים הנובעים מהתלות בספריות חיצוניות ושירותים של צד שלישי. המערכת עושה שימוש בספריות חיצוניות המסופקות על ידי חברות חיצוניות (לדוגמא: הפניה ל- chat gpt ושליחת מיילים מתוך התוכנית). כדי שהפרויקט יעבוד בצורה תקינה, יש צורך בקיומם של שירותים אלה וזמינותם ברשת. כלומר, אם שירות חיצוני לא זמין, המערכת עלולה לא לפעול כראוי או להיפגע ביכולותיה.

תיחום הפרויקט:

הפרויקט מתמקד בפיתוח מערכת בנקאות דיגיטלית מבוססת לקוח-שרת, עם דגש על תקשורת רשת מאובטחת בין הלקוח לשרת. התקשורת מתבצעת בפרוטוקולי TCP/IP תוך שימוש בהצפנה דו-כיוונית (AES, DES, RSA), שנבחרים באופן רנדומלי) לשמירה על סודיות ושלמות המידע. בנוסף, הפרויקט עוסק בפיתוח ממשק משתמש גרפי (GUI) בפיתוח באמצעות ספריית customtkinter המאפשרת גישה נוחה וידידותית למשתמש הקצה.

המערכת עצמה לא עוסקת בתפעול הפנימי של המחשב- ניהול חומרה, התקנת דרייברים, ניהול זיכרון פנימי או תחזוקת מערכת ההפעלה עצמה. בנוסף המערכת לא מתמקדת בניהול רשתות מורכבות. המערכת היא מערכת דמה, והיא לא מחוברת למערכת בנקאית אמיתית ולכסף אמיתי. כל הנתונים שנמצאים הם סימולציה שנועדה לחקות את הפעולות של מערכת בנקאית אמיתית. המערכת לא מנהלת חשבונות אמיתיים ולא מבצעת עסקאות כספיות אמיתיות, היא לא מחוברת לשום חשבון בנק אמיתי.

פירוט תיאור המערכת:

תיאור מפורט של המערכת:

תחילה כאשר פותחים את התוכנה, כל לקוח קורא מקובץ txt ייחודי את המידע הדרוש להרצת התוכנית, בקובץ נמצאים גם המפתחות הדרושים לכל אחת משלוש ההצפנות. המסך הראשוני שנדלק כאשר מפעילים את המערכת הוא האפשרות להירשם או להתחבר. לאחר ההתחברות הדו שלבית המשתמש יגיע למסך הפעולות שבו ניתן לבצע את כל הפעולות בחשבון. למערכת יכולים להתחבר בו זמנית מספר קבוע של משתנים (ניתן לקבוע בתוך התוכנית), על מנת למנוע התקפת DDoS.

פירוט היכולות של המשתמשים:

לאחר הפעלת התוכנה יוצג על המסך מסך התחברות, בו יש גם אופציה להירשם לראשונה לבנק. לאחר שמזינים שם משתמש וסיסמא נכונים יש צורך להזין קוד חד פעמי בעל 6 ספרות שנשלח למייל של המשתמש, אותו המייל שהזין בהרשמה הראשונית. לאחר הזנת קוד מוצלח מוצג המסך הראשי. במסך זה מופיע בצד שמאל למעלה שם המשתמש של המשתמש, במרכז המסך מופיעה היתרה של המשתמש ובצד שמאל מופיעים כפתורים שכל אחד מהם יפתח פראם שיאפשר את הפעולה הדרושה. ישנם חמישה כפתורים שמאפשרים לבצע העברות, בקשת העברות, אישור או דחית העברות, צפייה בהעברות קודמות, הצגת דוחות שנתיים על הוצאות וקבלת יעוץ AI לגבי דוחות אלו. כאשר מעבירים כסף ומבקשים העברה, יש צורך להכניס את שם המשתמש שמבקשים להעביר אליו או לבקש ממנו, את הסכום ואת סיבת ההעברה. כאשר צופים בהעברות הקודמות ניתן לראות את כל הנתונים האלו ובנוסף במידה והמידע המוצג הוא בצבע ירוק המשתמש קיבל העברה, ובמידה והמידע המוצג הוא בצבע אדום המשתמש העביר כסף. למשתמש יש אפשרות לצפות בדוחות שנתיים לפי נושאים- הוא בוחר את השנה שהוא רוצה לבדוק בה את ההוצאות (רק שנים שבהן הוא ביצע העברה כלשהי) ולאחר מכן מוצג לו גרף של כל ההוצאות החודשיות של הקטגוריה בשנה. יש למשתמש אפשרות ליעוץ מ-AI לפי דוחות אלו.

פירוט בדיקות:

בפרויקט שלי קיים מגוון רחב של בדיקות שונות שברובן הבדיקה היא בדיקת הקלט המתקבל מהמשתמש.

מספר בדיקה	שם בדיקה	מטרת בדיקה	אופן ביצוע
1	תקינות ההתחברות.	ווידוא ששם המשתמש קיים במערכת והסיסמא של המשתמש היא נכונה.	המשתמש מזין שם משתמש וסיסמא, במידה והם נכונים הוא מקבל הודעה בהתאם, וצריך להזין קוד חד פעמי שנשלח למייל. אם הסיסמא שגויה מוצגת שגיאה בהתאם.
2	תקינות האימות הדו שלבי- קוד למייל.	אימות דו שלבי שמוודא שהמשתמש שמנסה להתחבר הוא האדם לו שייך החשבון.	לאחר ההתחברות הראשונית, המשתמש מזין את הקוד שנשלח לו במייל. אם הקוד נכון המשתמש מקבל הודעה בהתאם.
3	בדיקה שמשתמש (מאותו הIP) לא הזין 3 פעמים סיסמא אישית שגוייה.	לודא שאם מאותו IP משתמש הזין 3 פעמים סיסמא לא נכונה, הוא לא יוכל להיכנס לחשבון זה במשך יממה.	כאשר משתמש מזין את הסיסמא שלו, קיימת בדיקה האם הוא טעה כבר 3 פעמים (עם אותו הIP) ביממה האחרונה, ככל שכן מוצגת הודעה בהתאם ואין לו אפשרות להיכנס לחשבון באותה יממה.
4	בדיקה שלא 5 משתמשים (מאותו הIP) הזינו 3 פעמים סיסמא אישית שגוייה ביממה האחרונה.	בדיקה האם חמישה משתמשים כבר נחסמו במהלך היום מאותה כתובת IP.	כאשר משתמש מזין שם משתמש וסיסמא הסרבר בודק האם מאותה כתובת IP כבר נחסמו באותו היום 5 משתמשים, אם כן אין למשתמש אפשרות להתחבר גם אם הסיסמא שלו נכונה והוא מקבל הודעה בהתאם.
5	בדיקה שלא 10 אנשים (מאותו הIP) הזינו לפחות פעם אחת סיסמא שגוייה ביממה האחרונה.	בדיקה שלא 10 משתמשים מאותה כתובת IP טעו לפחות פעם אחת ביממה האחרונה.	כאשר משתמש מזין שם משתמש וסיסמא הסרבר בודק האם מאותה כתובת IP לא טעו בסיסמא 10 משתמשים שונים לפחות פעם אחת במהלך היממה האחרונה, אם כן הוא לא מאפשר התחברות

			גם אם הנתונים נכונים. מוצגת הודעה בהתאם.
6	רישום משתמש חדש: האם השם פנוי.	בדיקה האם כבר קיים שם משתמש זה במאגר.	בהרשמה כאשר המשתמש מנסה לעבור לאימות הדו שלבי, במידה ושם המשתמש שהוא מנסה להירשם אליו תפוס, מוצגת הודעה בהתאם.
7	רישום משתמש חדש: האם האורך של שם המשתמש הוא פחות מ-9 תווים. ושאין בו ";", " ,".	בדיקה ששם המשתמש שהוזן בהרשמה הוא פחות מ-9 תווים, ושאין בו את התווים הבאים: ";", " ,".	כאשר המשתמש מנסה ליצור חשבון חדש קיימת בדיקה בנוגע לפלט. הוא מקבל הודעה בהתאם.
8	בדיקת תקינות הקלט בהעברה: בדיקה ששם המשתמש קיים.	לבדוק שהמשתמש שמועבר אליו כסף קיים.	כאשר משתמש מנסה להעביר כסף ולוחץ על כפתור ההעברה, ישנה בדיקה ששם המשתמש שהוזן הוא שם משתמש שקיים במערכת. מוצגת הודעה בהתאם ואם הוא לא קיים לא מתבצעת העברה.
9	בדיקת תקינות הקלט בהעברה: בדיקה ששם המשתמש שמועבר אליו כסף הוא לא המשתמש עצמו.	לבדוק שהמשתמש לא מנסה להעביר כסף לעצמו.	כאשר המשתמש מנסה להעביר כסף, ישנה בדיקה ששם המשתמש שמועבר אליו הכסף הוא לא שם המשתמש שלו.
10	בדיקת תקינות הקלט בהעברה: בדיקה שהסכום המועבר הוא תקין.	כאשר משתמש מנסה להעביר כסף, קיימת בדיקה על הקלט של הסכום. בודקים שהסכום חיובי ושהוא מספר ולא תו אחר.	כשהמשתמש מנסה להעביר כסף, בודקים שהקלט הוא קלט תקין (מספר חיובי). מוצגת הודעה בהתאם.
11	בדיקת תקינות הקלט בהעברה: בדיקה שנשאר כסף ביתרה.	לבדוק שלאחר ההעברה תישאר יתרה חיובית.	כאשר המשתמש מנסה להעביר כסף, בודקים שהסכום שהוא רוצה להעביר נמוך מהיתרה שיש לו.
12	בדיקת תקינות הקלט בבקשת העברה: בדיקה ששם המשתמש קיים.	לבדוק שהמשתמש שמבוקש ממנו כסף קיים.	כאשר משתמש מנסה לבקש כסף ממשתמש אחר ולוחץ על כפתור בקשת העברה, ישנה בדיקה ששם המשתמש שהוזן הוא שם משתמש שקיים במערכת. מוצגת הודעה בהתאם ואם הוא לא קיים לא מתבצעת הבקשה.

13	בדיקת תקינות הקלט בבקשת העברה: בדיקה ששם המשתמש שמבוקש ממנו כסף הוא לא המשתמש עצמו.	לבדוק שהמשתמש לא מנסה לבקש כסף מעצמו.	כאשר המשתמש מנסה לבקש העברה, ישנה בדיקה ששם המשתמש שמבוקש ממנו הכסף הוא לא שם המשתמש שלו.
14	בדיקת תקינות הקלט בבקשת העברה: בדיקה שהסכום המועבר הוא תקין.	כאשר משתמש מנסה לבקש כסף, קיימת בדיקה על הקלט של הסכום. בודקים שהסכום חיובי ושהוא מספר ולא תו אחר.	כשהמשתמש מנסה לבקש העברה, בודקים שהקלט הוא קלט תקין (מספר חיובי). מוצגת הודעה בהתאם.
15	בדיקת תקינות הקלט באישור העברה: בדיקה שנשאר כסף ביתרה.	בדיקה שלאחר ההעברה תישאר יתרה חיובית.	כאשר המשתמש מנסה לאשר העברה למשתמש אחר, בודקים שהסכום שהוא רוצה להעביר הוא נמוך מהיתרה שיש לו.
16	חסימת משתמשים רבים בו זמנית.	לודא שלא יותר ממספר מסוים של משתמשים מחוברים בו זמנית כדי למנוע עומס משתמשים.	בכל יצירת תקשורת הסרבר בודק את מספר המשתמשים המחוברים אליו, הוא שולח הודעה בהתאם ותוכנת המשתמש נפתחת או נסגרת בהתאם.

תכנון וניהול לוח זמנים:

תאריך ההגשה של הפרויקט הוא 24.5.2025. לכן התחלתי לעבוד על הפרויקט בשלבים:

- יצירת דף הרשמה והתחברות באמצעות Tkinter
- יצירת שרת ויצירת תקשורת בין דף ההרשמה והתחברות אליו
- יצירת בסיס נתונים לשמירה על המידע של כל המשתמשים
- יצירת האימות הדו שלבי שלי- שליחת המייל
- פיתוח מנגנון ההצפנה שלי
- יצירת המסך הראשי של הפרויקט
- הוספת הפעולות הבסיסיות למסך הראשי- העברה ובקשת העברה
- הוספת הפעולות הנוספות לתוכנית- גרפים, אישור העברה, צפייה בהעברות קודמות
- הוספת חיבור לבינה מלאכותית- יעוץ למשתמש
- עבודה על ספר הפרויקט

הסיכונים בפרויקט והדרכים להתמודד איתם:

הסיכונים:

1. סיכון/ חשש שלא אספיק לסיים את הפרויקט בזמן ושלא אוסיף את כל הפיצ'רים שאני רוצה להוסיף.
2. הסיכון שהתקשורת שלי לא תהיה מאובטחת מספיק- הרי זו אפליקציית בנק, יש צורך באבטחה מרבית.

פתרונות:

1. תכננתי את זמני ואת סדר העבודה בצורה מפורטת ומדויקת.
2. למדתי את הנושא לעומק והשתמשתי בשלוש הצפנות שונות, גם סימטריות וגם אסימטריות על מנת לאבטח את הממשק בצורה מספקת.

פרק 2- תיאור תחום הידע:

הרשמה למערכת:

מהות: יצירת חשבון חדש עבור משתמש חדש הכולל שם משתמש, סיסמה ומייל.

אוסף יכולות הנדרשות למימוש היכולת: ממשק משתמש (מסך ההרשמה), קליטת נתונים מהמשתמש, בדיקת תקינותם, הצפנת הסיסמא, הצפנת ההודעה כולה, שליחה לשרת המרכזי, פענוח ההודעה, בדיקת תקינות אל מול בסיס הנתונים בשרת- לפי המידע מההודעה המפוענחת, הצפנת תשובה למשתמש, קבלת תשובה מהשרת, פענוח, הצגת התשובה למשתמש.

אובייקטים נחוצים: ממשק משתמש- מסך ראשוני, הצפנה/פענוח, תקשורת-server , client , בסיס נתונים.

אימות כתובת המייל על ידי קוד חד פעמי שנשלח למייל לפני הרשמה:

מהות: ווידוא כי כתובת המייל שהוזנה בהרשמה היא כתובת המייל האמיתית והתקינה של המשתמש. המשתמש מזין את הקוד החד פעמי שנשלח אליו למייל.

אוסף יכולות נדרשות למימוש היכולת: ממשק משתמש (מסך הזנת הקוד), שליחת מייל עם הקוד החד פעמי, קליטת נתונים מהמשתמש, בדיקת הנתונים מהמשתמש אל מול הקוד שנוצר עם שליחת המייל, הצגת התשובה למשתמש.

אובייקטים נחוצים: ממשק משתמש- מסך ראשוני, מערכת מייל, הצפנה/פענוח, תקשורת-server , client , בסיס נתונים.

מסך התחברות:

מהות: חיבור משתמש קיים למערכת.

אוסף יכולות נדרשות למימוש היכולת: ממשק משתמש (מסך התחברות), קליטת נתונים מהמשתמש, בדיקת תקינותם, הצפנת ההודעה, שליחה לשרת המרכזי, פענוח ההודעה, בדיקת תקינות אל מול בסיס הנתונים בשרת- לפי המידע מההודעה המפוענחת, הצפנת תשובה למשתמש, קבלת תשובה מהשרת, פענוח, הצגת תשובה למשתמש.

אובייקטים נחוצים: ממשק משתמש- מסך ראשוני, הצפנה/פענוח, תקשורת-server , client , בסיס נתונים.

אימות דו שלבי- שליחת קוד חד פעמי למייל:

מהות: ווידוא שהמשתמש שמנסה להתחבר לחשבון הוא המשתמש האמיתי. המשתמש מזין את הסיסמא שנשלחה לו במייל.

אוסף יכולות נדרשות למימוש היכולת: ממשק משתמש (מסך הזנת הקוד), שליחת מייל עם הקוד החד פעמי, קליטת נתונים מהמשתמש, בדיקת הנתונים מהמשתמש אל מול הקוד שנוצר עם שליחת המייל, הצגת התשובה למשתמש.

אובייקטים נחוצים: ממשק משתמש- מסך ראשוני, מערכת מייל, הצפנה/פענוח, תקשורת-server , client , בסיס נתונים.

העברת כספים למשתמש אחר:

מהות: מאפשר למשתמש להעביר כספים למשתמש אחר שנמצא במערכת.

אוסף יכולות נדרשות למימוש היכולת: ממשק משתמש (פריים העברת כספים), קליטת נתונים מהמשתמש, בדיקת תקינותם, הצפנת ההודעה, שליחה לשרת המרכזי, פענוח ההודעה, בדיקת תקינותם אל מול בסיס הנתונים בשרת- לפי המידע מהתשובה המפוענחת, עדכון בסיס הנתונים במידת הצורך, הצפנת תשובה למשתמש, קבלת תשובה מהשרת, פענוח, הצגת תשובה למשתמש.

אובייקטים נחוצים: ממשק משתמש- מסך שני, הצפנה/פענוח, תקשורת- server, client, בסיס נתונים.

בקשת העברה ממשתמש אחר:

מהות: מאפשר למשתמש לבקש ממשתמש אחר שנמצא במערכת להעביר לו כספים.

אוסף יכולות נדרשות למימוש היכולת: ממשק משתמש (פריים בקשת העברת כספים), קליטת נתונים מהמשתמש, בדיקת תקינותם, הצפנת ההודעה, שליחה לשרת המרכזי, פענוח ההודעה, בדיקת תקינותם אל מול בסיס הנתונים בשרת- לפי המידע מהתשובה המפוענחת, עדכון בסיס הנתונים במידת הצורך, הצפנת תשובה למשתמש, קבלת תשובה מהשרת, פענוח, הצגת תשובה למשתמש.

אובייקטים נחוצים: ממשק משתמש- מסך שני, הצפנה/פענוח, תקשורת- server, client, בסיס נתונים.

אישור העברה למשתמש אחר:

מהות: מאפשר למשתמש לאשר את ההעברה שמשתמש אחר ביקש ממנו.

אוסף יכולות נדרשות למימוש היכולת: ממשק משתמש (פריים אישור העברות), קבלת קלט מהמשתמש (על ידי לחיצה על כפתור), הצפנת ההודעה, שליחה לשרת המרכזי, פענוח ההודעה, עדכון בסיס הנתונים, הצפנת תשובה למשתמש, קבלת תשובה מהשרת, פענוח, הצגת תשובה למשתמש (במידה והוא מאשר- היתרה יורדת וניתן לראות את ההעברה בהעברות הקודמות).

אובייקטים נחוצים: ממשק משתמש- מסך שני, הצפנה/פענוח, תקשורת- server, client, בסיס נתונים.

צפייה בהעברות קודמות:

מהות: למשתמש יש יכולת לצפות בכל ההעברות הקודמות שלו- למי הוא העביר או מי העביר לו, כמה העבירו, באיזה תאריך ומאיזו סיבה. אם שורת ההעברה מופיעה בצבע ירוק המשתמש קיבל את הכסף, אם שורת ההעברה מופיעה בצבע אדום המשתמש העביר את הכסף.

אוסף יכולות נדרשות למימוש היכולת: ממשק משתמש (פריים צפייה בהעברות קודמות), הצפנת הודעה- עם ניסוח קבוע, שליחה לשרת המרכזי, פענוח ההודעה, קבלת מידע מבסיס הנתונים, הצפנת המידע, קבלת המידע מהשרת, פענוח, הצגת המידע למשתמש.

אובייקטים נחוצים: ממשק משתמש- מסך שני, הצפנה/פענוח, תקשורת- server, client, בסיס נתונים.

צפייה בגרפים של ההוצאות השנתיות בקטגוריה כלשהי:

מהות: למשתמש יש אפשרות לצפות בגרפים שנתיים- לפי קטגוריות של הוצאותיו.

אוסף יכולות נדרשות למימוש היכולת: ממשק משתמש (פריים גרפים), קבלת קלט מהמשתמש, בדיקת תקינותו, הצפנת ההודעה, שליחה לשרת המרכזי, פענוח ההודעה, קבלת מידע מבסיס הנתונים, הצפנת המידע, קבלת המידע מהשרת, פענוח, הצגת המידע למשתמש.

אובייקטים נחוצים: ממשק משתמש- מסך שני, הצפנה/פענוח, תקשורת- server, client, בסיס נתונים.

בקשת יעוץ פיננסי מבינה מלאכותית:

מהות: למשתמש יש אפשרות לבקש יעוץ מבינה מלאכותית בנוגע לגרפים השנתיים של הוצאותיו (בקטגוריה מסויימת).

אוסף יכולות נדרשות למימוש היכולת: ממשק משתמש (פריים גרפים), קבלת קלט מהמשתמש, שליחה לבינה מלאכותית הודעה עם ההוצאות, קבלת תשובה מהבינה המלאכותית, הצגת המידע למשתמש.

אובייקטים נחוצים: ממשק משתמש- מסך שני.

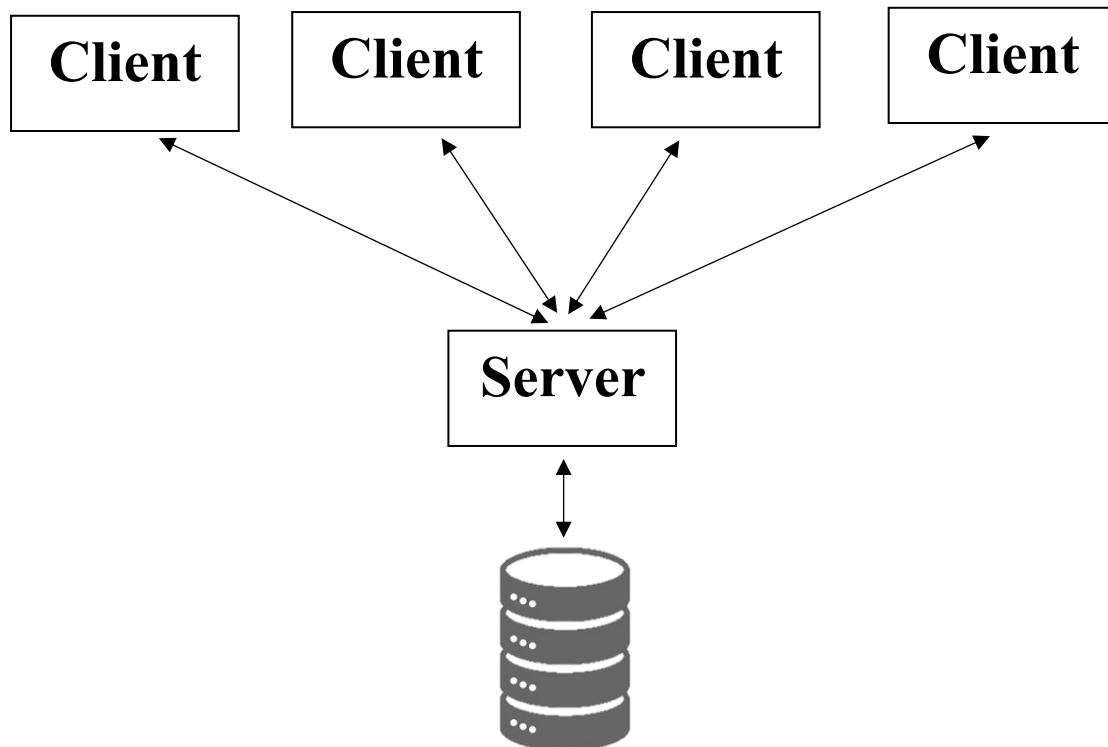
פרק 3- ארכיטקטורה של הפרויקט:

תיאור החומרה:

המערכת בנויה משרת מרכזי שאחראי לטפל בכל הפניות ובכל המידע שמגיע אליו מהלקוחות שלו. הוא אחראי לקבל את הבקשות מהמשתמש ולהחזיר מידע או להכניס מידע לבסיס הנתונים. כל הלקוחות מחוברים לשרת מרכזי וכל בקשה או העברה נכנסת לבסיס הנתונים. לפני שהמידע מועבר הוא עובר הצפנה- נבחר מספר רנדומלי ולפי מספר זה נבחרת ההצפנה הרצויה, או RSA או DES או AES. כאשר המידע מגיע לשרת הוא עובר פענוח- מספר ההצפנה משורשר להודעה עצמה.

ממשק המשתמש כתוב בפיתון עם ספריית Tkinter. והשרת כתוב בשפת פייתון ומשתמש ב socket לתקשורת. בסיס הנתונים הוא SQLite.

הפרויקט כתוב בתוכנת פייתון והחומרה הנדרשת עבור הרצת הפרויקט היא: מחשב עם חיבור אינטרנט, יכולת הרצה של פייתון על המחשב.

שרטוט המציג קשרים אלו:

dataBaseTamarEyal.db

תיאור הטכנולוגיות הרלוונטיות:**שפת התכנות- פיתון:**

פיתון היא שפת תכנות מאוד נפוצה וקלה להבנה. היא מתאימה למגוון שימושים כמו בניית ממשקים, ניהול מסדי נתונים, תקשורת בין מחשבים וגם הצפנה. בנוסף, התחביר שלה פשוט ונוח, מה שמקל על העבודה והתחזוקה של הקוד.

בסיס נתונים- SQLite:

SQLite הוא מסד נתונים קל ופשוט שמיועד לשימוש בתוך אפליקציה, בלי צורך בהתקנה של שרת חיצוני. כל הנתונים נשמרים בקובץ אחד במחשב, מה שהופך אותו לנוח מאוד לפרויקטים אישיים, אפליקציות שולחניות ואפילו אפליקציות מובייל. היתרונות שלו הם קלות השימוש, מהירות הביצוע והעובדה שלא צריך להתקין שום דבר נוסף.

מערכת הפעלה- windows 11:

העבודה על הפרויקט התבצעה על מערכת ההפעלה windows 11. זוהי הגרסה הכי עדכנית של מיקרוסופט. היא יציבה, נוחה, תומכת בכל הכלים שהייתי צריכה, ומתאימה לפיתוח בפיתון ולעבודה עם ממשקי משתמש, קבצים, רשתות ועוד.

פרוטוקול TCP:

TCP הוא פרוטוקול תקשורת המשמש לשליחת נתונים בין מחשבים בצורה אמינה ובטוחה. הוא דואג שכל המידע באמת יגיע ליעד, בלי שגיאות ובלי חלקים חסרים. הוא מתאים במיוחד כשחשוב שהמידע לא יאבד, ישנם פרוטוקולים יותר מהירים אז משתמשים בו כשאין צורך בתקשורת מהירה מאוד.

הצפנת RSA:

RSA היא שיטת הצפנה אסימטרית. משתמשים בשני מפתחות: מפתח ציבורי להצפנה ומפתח פרטי לפענוח. השיטה הזו בטוחה מאוד ונפוצה כשצריך להעביר מידע רגיש בצורה מאובטחת, כמו סיסמאות או קודים אישיים. היא נחשבת מאובטחת במיוחד.

הצפנת AES:

AES היא שיטת הצפנה סימטרית. גם להצפנה וגם לפענוח משתמשים באותו מפתח. היא מהירה, חזקה ומאוד נפוצה, במיוחד כשצריך להצפין כמויות גדולות של מידע, למשל קבצים או הודעות בזמן אמת.

הצפנת DES:

DES היא גם שיטת הצפנה סימטרית, שהייתה נפוצה בעבר. היום היא פחות בשימוש כי היא פחות בטוחה לעומת שיטות חדשות יותר.

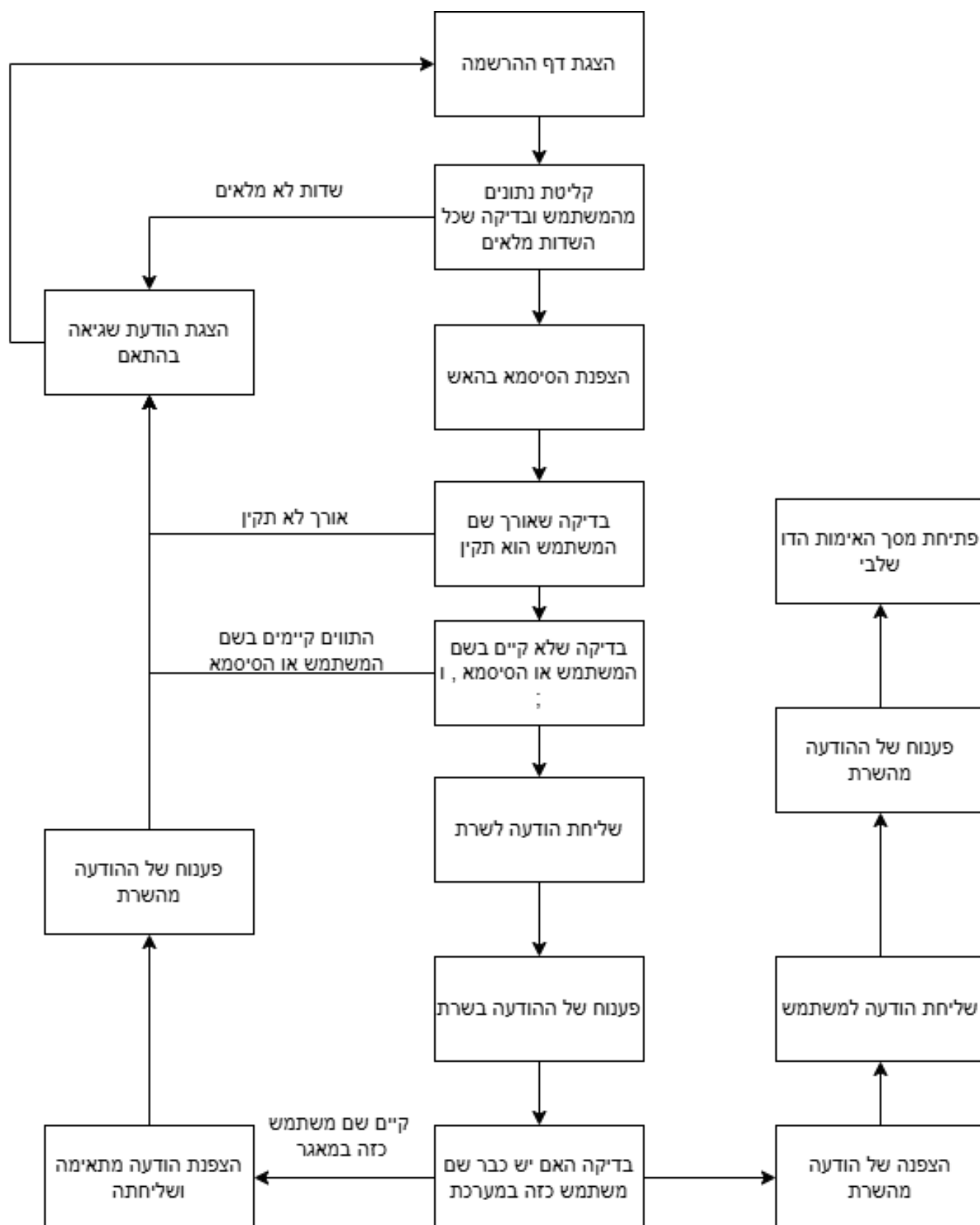
פרוטוקול SMTP:

זהו פרוטוקול תקשורת המשמש לשליחת הודעות דוא"ל בין שרתים. רשאי התיבות של הפרוטוקול הם: "simple mail transfer protocol". הפרוטוקול אינו משמש לקריאת הודעות אלא רק להעברתן.

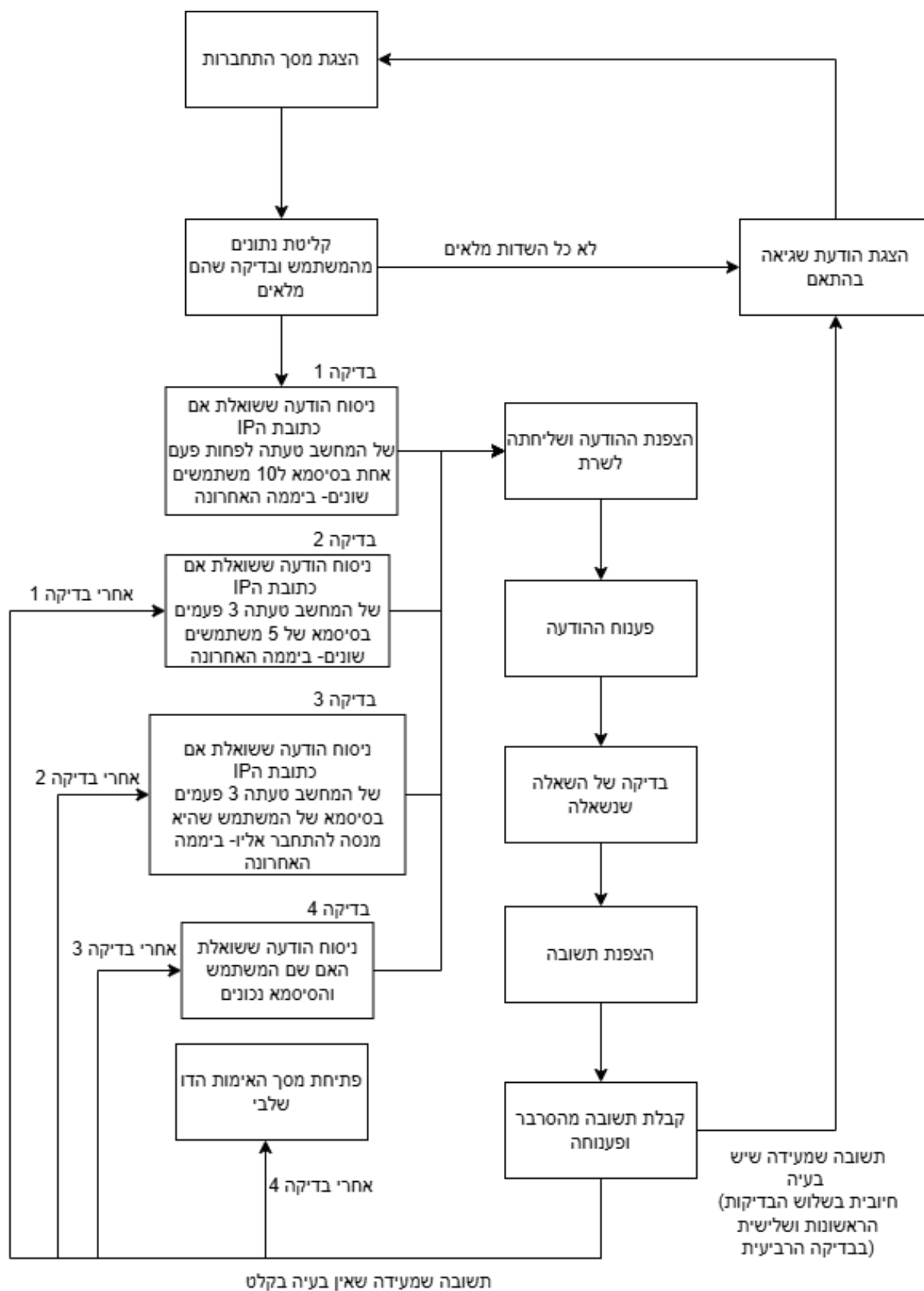
- לקוח (כמו Gmail, אפליקציה כמו Pycharm ועוד..) שולחות את ההודעה דרך SMTP.
- ההודעה מועברת לשרת SMTP של ספק האינטרנט או שירות הדואר.
- שרת זה "מתווך" את ההודעה ומעביר אותה הלאה. (לפעמים גם דרך עוד שרתים)
- ההודעה מגיעה לתיבת הדואר של הנמען.

תיאור זרימת המידע:

מסך הרשמה:



מסך התחברות:



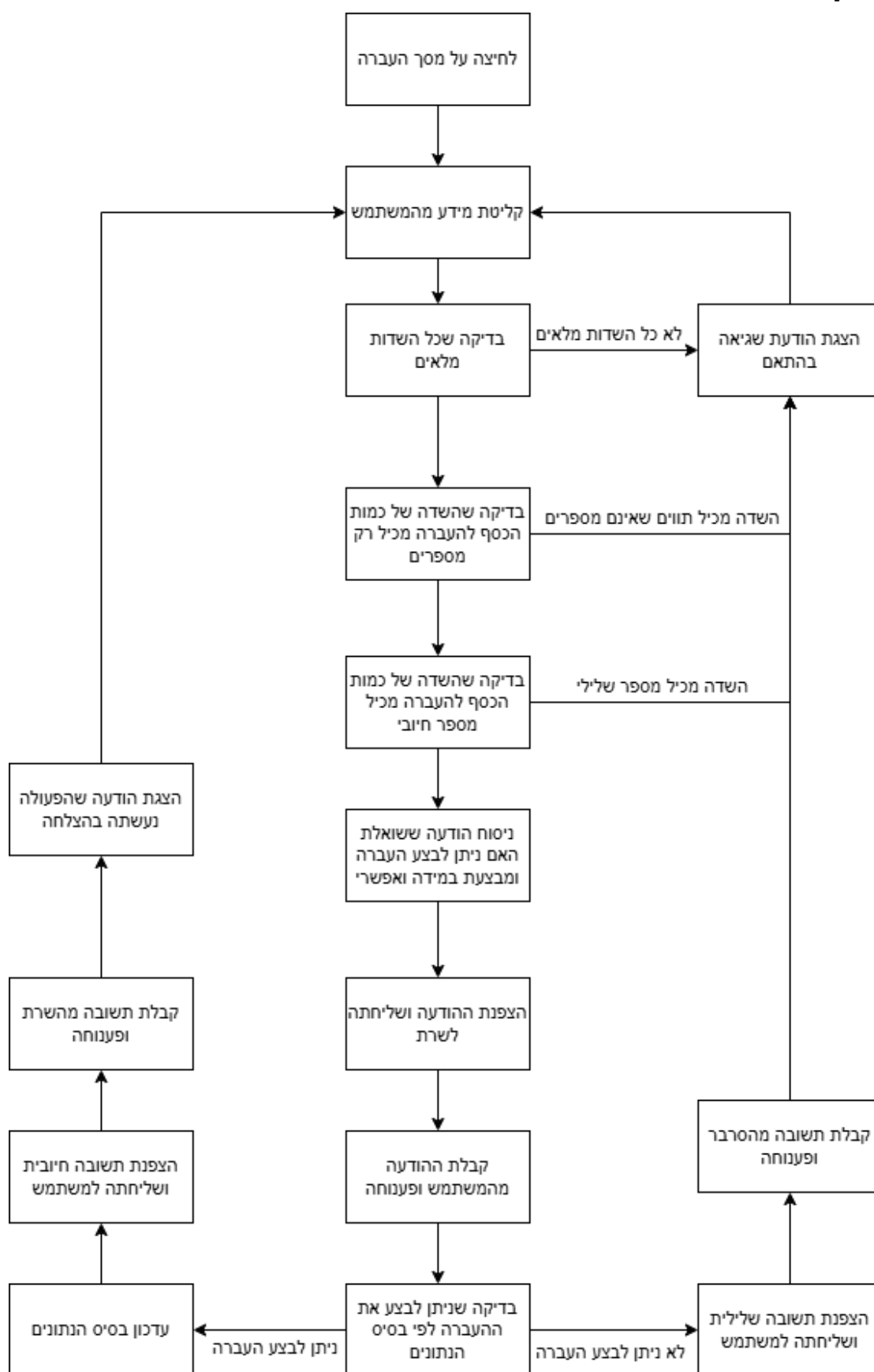
מסך אימות דו שלבי (לאחר הרשמה):



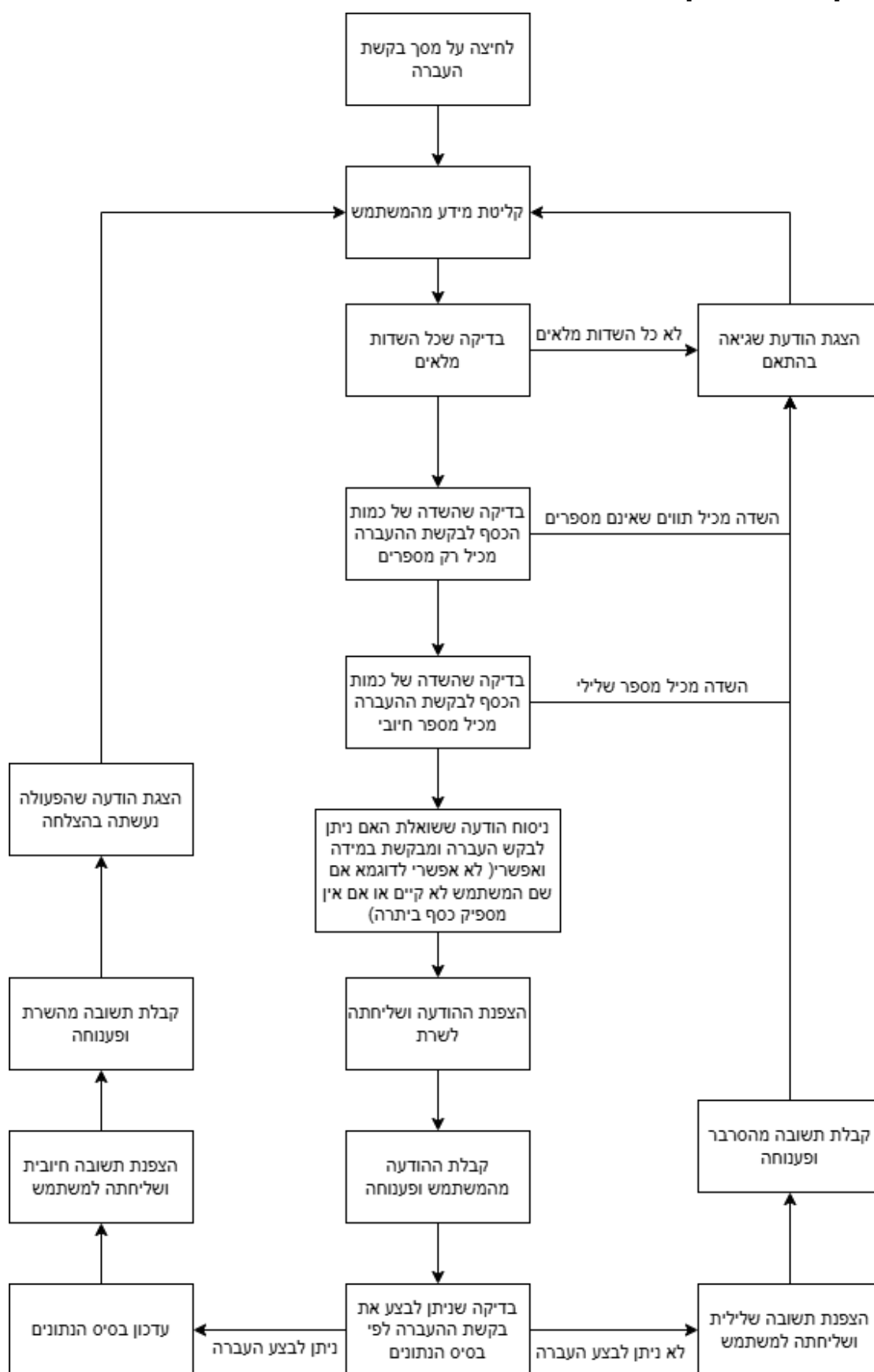
מסך אימות דו שלבי (לאחר התחברות ראשונית):



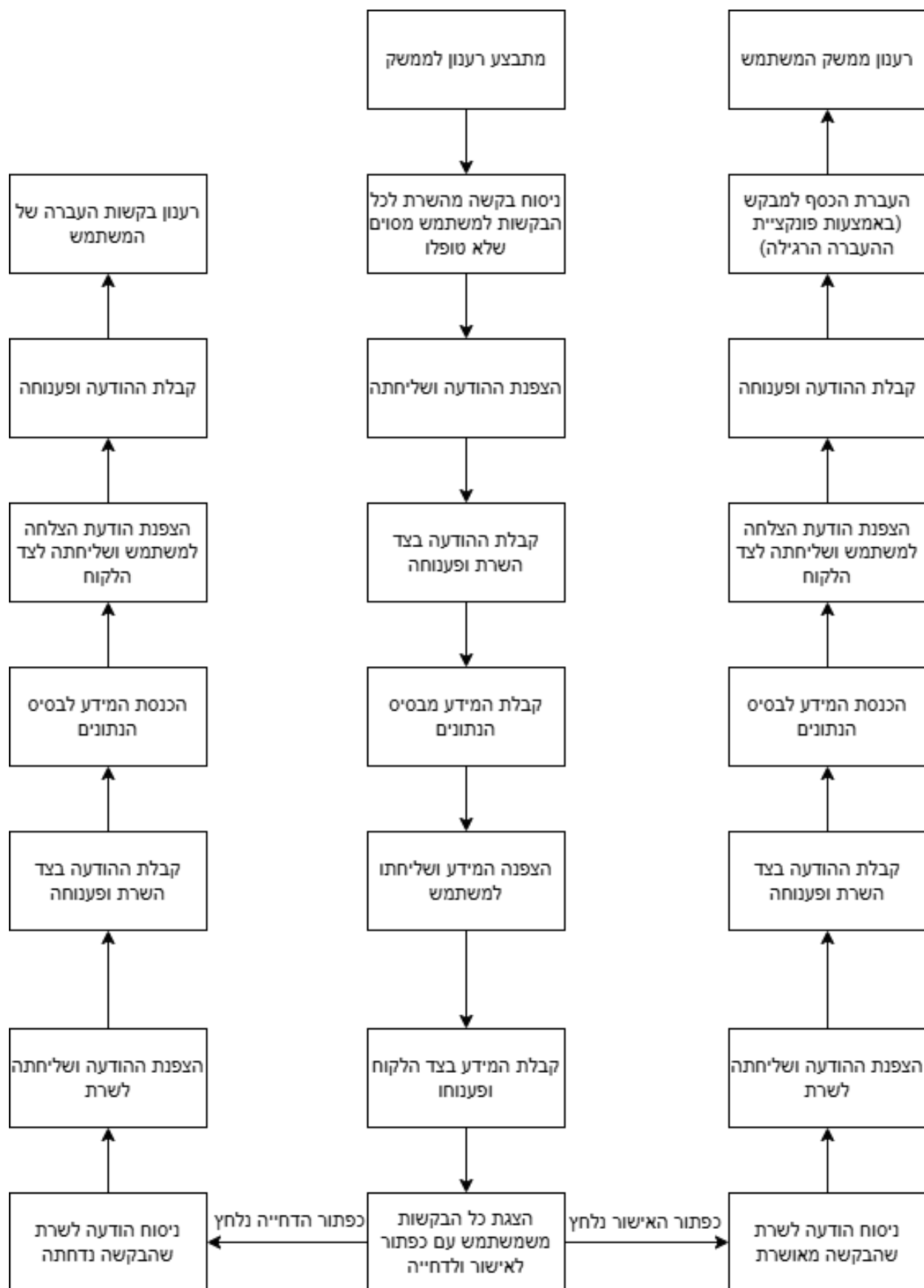
מסך ראשי- העברת כספים:

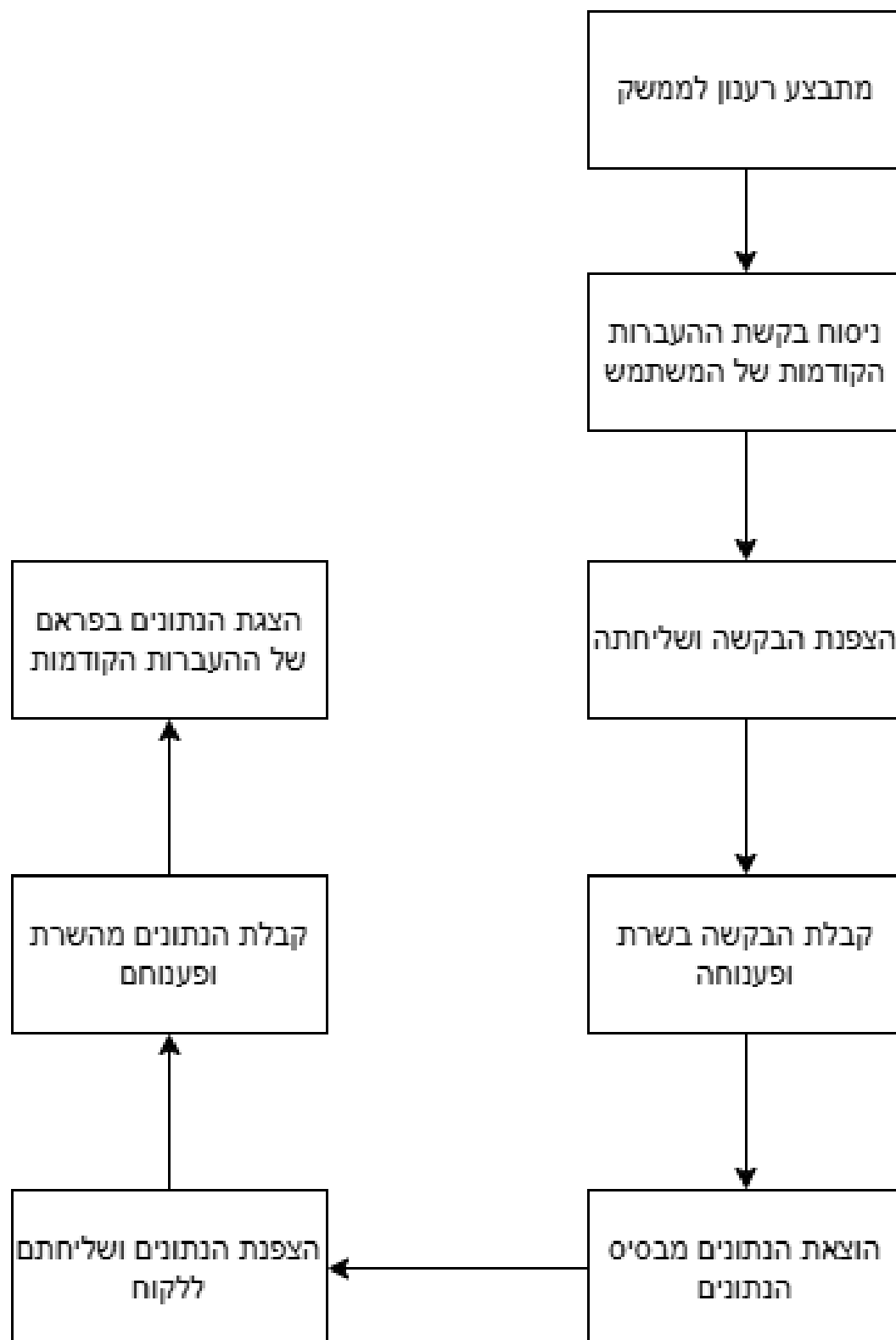


מסך ראשי- בקשת העברה:

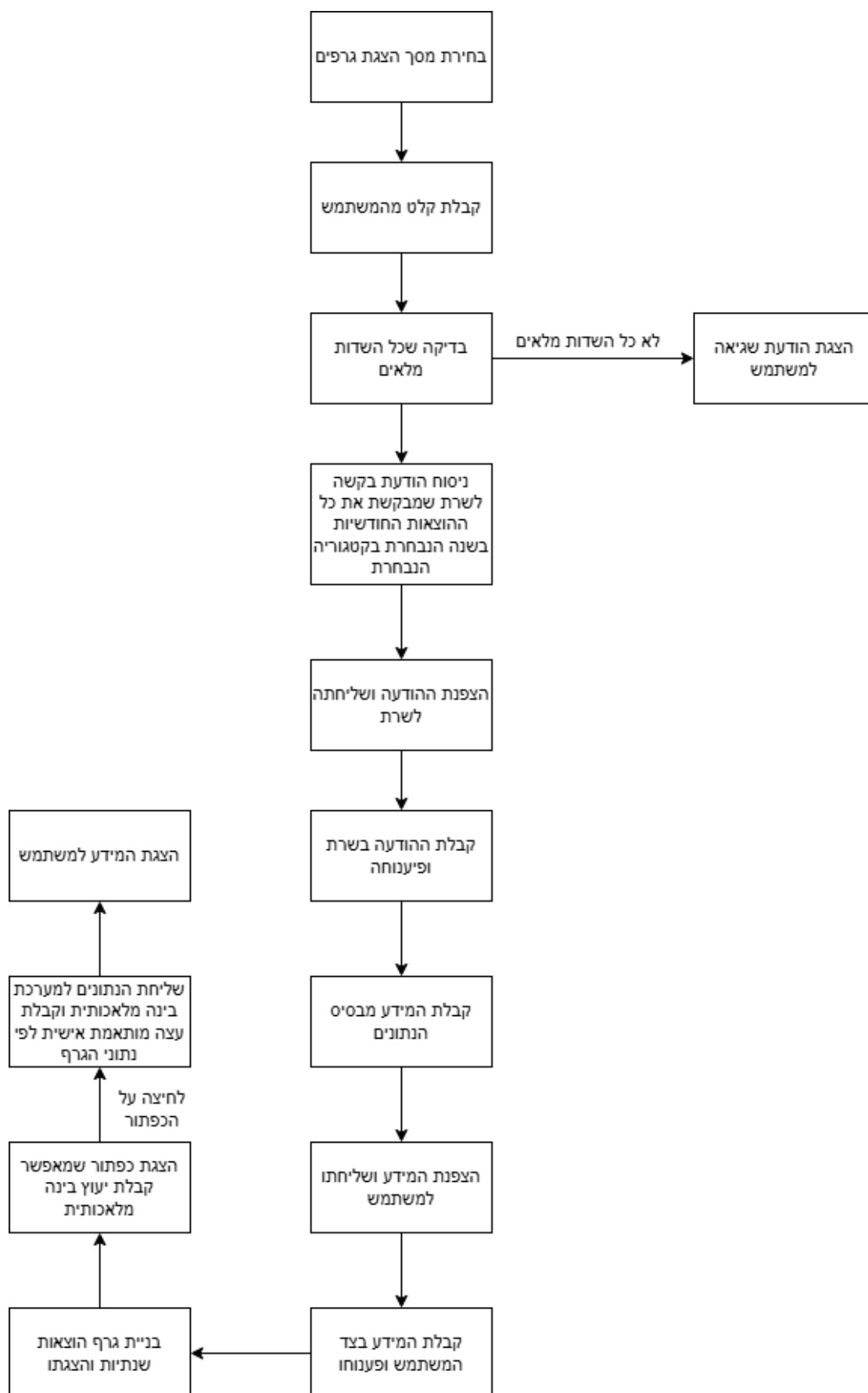


מסך ראשי- אישור העברה:



מסך ראשי- צפייה בהעברות קודמות:

מסך ראשי- צפייה בגרפים של ההוצאות השנתיות:



תיאור האלגוריתמים המרכזיים בפרויקט:

אלגוריתם התחברות למערכת:

כאשר משתמש מנסה להיכנס למערכת קיימות מספר בדיקות על מנת לאמת שהוא המשתמש האמיתי. בבסיס הנתונים קיימת טבלה שבה מפורטים כל הפעמים שמשתמש טעה בסיסמא שלו וכתובת ה-IP ממנה בוצעה הטעות. כאשר משתמש מנסה להיכנס לחשבון שלו המערכת בודקת האם מכתובת ה-IP של המשתמש 10 משתמשים שונים טעו בסיסמתם לפחות פעם אחת במהלך היום (לא בהכרח המשתמש הספציפי הזה). אם הם טעו, המשתמש לא יכול להיכנס לחשבון שלו. לאחר מכן קיימת בדיקה האם מכתובת ה-IP של המשתמש 5 משתמשים שונים טעו בסיסמתם שלוש פעמים במהלך היום (לא בהכרח המשתמש הספציפי הזה). אם הם טעו המשתמש לא יכול להיכנס לחשבון שלו. לאחר מכן קיימת בדיקה אם מכתובת ה-IP של המשתמש, המשתמש הספציפי הזה טעה 3 פעמים בהזנת הסיסמא במהלך היום. אם הוא טעה הוא לא יכול להיכנס לחשבון.

במידה והמשתמש עבר את כל הבדיקות נשלח קוד אישי חד פעמי למייל המשתמש. במידה והמשתמש טעה 3 פעמים בהזנת הקוד האישי הוא לא יכול להיכנס לחשבון שלו שוב במהלך המשך היום.

הבעיה האלגוריתמית: איך לוודא שהמשתמש שמנסה להיכנס הוא המשתמש האמיתי.

תיאור אלגוריתמים קיימים: בין האלגוריתמים הקיימים לבדיקה שהמשתמש שמתחבר הוא המשתמש האמיתי ישנם: התחברות באמצעות שם משתמש אישי וסיסמא אישית, מנגנון נעילת IP לאחר מספר ניסיונות כושלים, שימוש בקוד חד פעמי, הגבלת מספר הבקשות בזמן קצר, הגבלת מספר ניסיונות כניסה לפי IP ועוד..

סקירת הפתרון הקיים: בפרויקט יישמתי את האלגוריתמים הבאים: התחברות באמצעות שם משתמש אישי וסיסמא אישית, מנגנון נעילת IP לאחר מספר ניסיונות כושלים: שימוש בקוד חד פעמי. בחרתי בפתרונות אלו מאחר שהם מאכסנים בכל יום את המשתמשים החסומים ואת כל המשתמשים שטעו בסיסמתם בבסיס הנתונים ונותנים אפשרות למעקב אחר נתונים אלו, משלבים אימות דו שלבי- גם קוד חד פעמי למייל וגם קוד אישי והפתרונות חוסמים כתובות חשודות עם מספר רב של כניסות שגויות.

אלגוריתם הרשמה למערכת:

כאשר משתמש מנסה להירשם לממשק, ראשית הוא צריך לספק שם, סיסמא וכתובת מייל. המשתמש צריך לבחור שם משתמש שלא נמצא בשימוש על ידי משתמש אחר. לאחר שהוא מזין פרטים אלו נשלחת לו הודעה למייל עם קוד אישור. במידה והוא לא יזין את הקוד הוא לא יוכל להירשם לממשק.

הבעיה האלגוריתמית: ווידוא שהמשתמש שנרשם הוא משתמש אמיתי ולא בוט ושכל הפרטים שסופקו הם הפרטים הנכונים- במיוחד כתובת המייל.

תיאור אלגוריתמים קיימים: אלגוריתמים שקיימים לבעיה הם: אימות עם קוד חד פעמי, הרשמה דרך צד שלישי (לדוגמא: google), אימות מייל מאוחר יותר ועוד...

סקירת הפתרון הקיים: הפתרון שלי משתמש באימות עם קוד חד פעמי לאחר הזנת הפרטים של המשתמש. המשתמש לא יכול להירשם לממשק בלי קוד זה. זהו פתרון יעיל ופשוט והוא

אינו דורש שירותים חיצוניים. הוא מאפשר לוודא שכתובת המייל תקינה ונגישה. לא נבחרו פתרונות אחרים מאחר שחלקם דורשים צד שלישי וחלקם לא מספיק מאובטחים.

אלגוריתם למניעת הצפת משתמשים:

בפרויקט שלי על מנת להגן מפני מתקפת DDoS, קיימת בבסיס הנתונים טבלה של כל תקשורת שהייתה אי פעם למשתמש עם הסרבר- גם אם הוא לא התחבר לחשבון. כאשר מגיעה פניה לחיבור, השרת בודק האם מספר המשתמשים המחוברים קטן מהמספר שהוחלט שהוא מקסימום המשתמשים שיכולים להיות מחוברים בזמנית. אם המספר קטן המשתמש יכול להתחבר, ואם לא - מוצגת הודעת שגיאה.

הבעיה האלגוריתמית: איך ניתן לזהות עומס חריג של חיבורים ולמנוע חיבור של מספר רב של משתמשים.

תיאור אלגוריתמים קיימים: הגבלת מספר החיבורים בזמנית, חומת אש לפי מדיניות כלשהי ועוד...

סקירת הפתרון הקיים: בשרת שלי קיים מנגנון שמונע הצפת משתמשים בעזרת בדיקה בזמן אמת של מספר המשתמשים הפעילים. הקוד מוודא שהשרת לא יחובר ליותר מ-4 חיבורים בזמנית (המספר המקסימלי ניתן לשינוי בקוד). פתרון זה פשוט ליישום ובעל יעילות גבוהה. הוא מאפשר לצפייה בכל החיבורים לשרת אי פעם ונותן מעקב על מידע זה. חומת האש לא נבחרה כי היא חסימה כללית ולא נותנת שליטה ספציפית בפרויקט.

אלגוריתם ההצפנה:

בפרויקט שלי בחרתי להצפין את המידע באמצעות 3 הצפנות שונות. בכל שליחת מידע בין משתמש לשרת ובין השרת למשתמש נבחר מספר רנדומלי בין 0-2 ולפי מספר זה ההודעה מוצפנת. RSA-0, AES-1, DES-2. מספר ההצפנה משורשר לתחילת ההודעה וההודעה מפוענחת בהתאם. המפתחות לכל אחת מההצפנות האלו נשמרים בקובץ נפרד. בתחילת התוכנית קיימת בדיקה שהקובץ נמצא וקיים- בלעדיו אין אפשרות לקיים תקשורת בטוחה.

הבעיה האלגוריתמית: איך לאבטח את המידע כך שיהיה מוצפן אך גם שההצפנה תהיה מהירה ועמידה מפני סוגים שונים של מתקפות.

תיאור אלגוריתמים קיימים: קיים מספר רב של אלגוריתמים להצפנת מידע, ביניהם RSA, DES, AES ועוד. RSA זוהי הצפנה אסימטרית שנחשבת בטוחה אך איטית לעומת הצפנה סימטרית. AES זוהי הצפנה סימטרית, מהירה מאוד. DES זוהי הצפנה סימטרית ישנה יותר.

סקירת הפתרון הקיים: לפתרון הבעיה החלטתי שבמקום לבחור באלגוריתם בודד אשלב את כל שלושת האלגוריתמים ביחד ובכל שליחת הודעה תיבחר באופן רנדומלי הצפנה אחרת. בחרתי בפתרון זה מאחר שדבר זה יגרום לצופה חיצוני לא לדעת באיזו הצפנה נעשה שימוש בכל הודעה. בנוסף, פתרון זה מאפשר פיתוח עתידי- הוספת עוד הצפנות במערך ההצפנות. יתר על כן, בחרתי בשיטה זו כיוון ששימוש ב-RSA בלבד יכול להיות איטי עבור הודעות מרובות ושימוש ב-AES בלבד יכול להיות צפוי ולא מספיק מאובטח.

תיאור סביבת הפיתוח:

בשביל לפתח את הפרויקט השתמשתי ב PyCharm Community Edition. בשביל לערוך את בסיס הנתונים שלי השתמשתי ב SQLite. על מנת לבדוק את הפרויקט יש צורך ב- PyCharm Community Edition

בנוסף נדרשות הספריות הבאות:

socket

threading

json

random

os

sys

time

sqlite3

datetime

tkinter.messagebox as tkmb

tkinter as tk

customtkinter as ctk

matplotlib.figure

matplotlib.backends.backend_tkagg

openai

hashlib

smtplib

email.mime.text

email.mime.multipart

base64

cryptography.hazmat.primitives.ciphers

cryptography.hazmat.backends

cryptography.hazmat.primitives

binascii

בחלק מהמחשבים יש צורך לייבא:

Cryptodome.Cipher

Cryptodome.Util.Padding

ובחלק מהמחשבים יש צורך לייבא:

Crypto.Cipher

Crypto.Util.Padding

תיאור פרוטוקול התקשורת:

בפרוייקט זה השתמשתי בספריית socket של פייתון על מנת לשלוח הודעות. פרוטוקול התקשורת שממומש בתוכנית שלי הוא פרוטוקול TCP. זהו פרוטוקול תקשורת בעולם התקשורת והאינטרנט. זהו פרוטוקול אמין שמבטיח שכל המידע מגיע בשלמותו, בסדר הנכון וללא שגיאות.

על מנת לשלוח מידע בפרוטוקול, נוצר חיבור בין שני מחשבים בתהליך לחיצת יד משולשת. המחשב הראשון שולח הודעת SYN, המשתמש השני עונה לו עם SYN-ACK, והראשון מאשר עם ACK. לאחר תהליך זה ניתן לשלוח מידע.

פירוט כלל ההודעות הזורמות במערכת:

1. שם ההודעה: בקשת התחברות למערכת
 נשלחת מ: משתמש ל: שרת
 מבנה ההודעה: "LOGIN;{username};{passwordHashes}"
2. שם ההודעה: בקשה לבדיקת האפשרות להרשמה במערכת
 נשלחת מ: משתמש ל: שרת
 מבנה ההודעה: "CHECK_REGISTRATION;{username}"
3. שם ההודעה: הרשמת משתמש חדש לממשק
 נשלחת מ: משתמש ל: שרת
 מבנה ההודעה: "REGISTER;{username};{passwordHashed};{email}"
4. שם ההודעה: העברת כסף ממשתמש אחר (במידה וחוקי)
 נשלחת מ: משתמש ל: שרת
 מבנה השדות בהודעה:
 "TRANSFER;{username};{userToTransfer};{amountToTransfer};{reasonToTransfer}"

5. **שם ההודעה:** קבלת היתרה של המשתמש
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "GET_BALANCE;{username}"
6. **שם ההודעה:** קבלת רשימה של כל ההעברות הקודמות של המשתמש
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "PREVIOUS;{username}"
7. **שם ההודעה:** שליחת בקשה למשתמש אחר (במידה וחוקי)
נשלחת מ: משתמש ל: שרת
מבנה ההודעה:
"REQUEST;{username};{requestedUser};{amount_money_requested};{reasonToTransfer}"
8. **שם ההודעה:** קבלת רשימת ההעברות שהתבקשו מהמשתמש
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "GET_REQUESTS;{username}"
9. **שם ההודעה:** שינוי הסטטוס של הבקשה- מלא טופל למטופל
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "CHANGE_STATUS;{ID_num}"
10. **שם ההודעה:** קבלת מספר הבקשות של משתמש מסוים
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "AMOUNT_REQUESTED;{username}"
11. **שם ההודעה:** קבלת מספר ההעברות של משתמש מסוים
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "AMOUNT_TRANSFERS;{username}"
12. **שם ההודעה:** קבלת השנים שבמהלכן משתמש מסוים העביר כסף
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "GET_YEARS;{username}"
13. **שם ההודעה:** קבלת רשימת העברות חודשיות של משתמש בקטגוריה מסוימת ושנה מסוימת
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "GET_YEAR_SPENDING;{username};{year};{category}"
14. **שם ההודעה:** בקשת כמות הפעמים שמשתמש טעה בסיסמא שלו ביממה האחרונה
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "CHECK_ATTEMPTS;{username};{ip}"
15. **שם ההודעה:** עדכון כמות הפעמים שהמשתמש טעה בסיסמא ועדכון התאריך
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "UPDATE_ATTEMPTS;{username};{ip}"

16. שם ההודעה: קבלת כמות המשתמשים מאותה כתובת IP שטעו 3 פעמים
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "AMOUNT_OF_FAIL_ATTEMPTS;{ip}"
17. שם ההודעה: קבלת כמות המשתמשים מאותה כתובת IP שטעו לפחות פעם 1
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "ALL_FAIL_ATTEMPTS;{ip}"
18. שם ההודעה: קבלת כתובת האימייל של המשתמש
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "GET_EMAIL;{username}"
19. שם ההודעה: עדכון בסיס הנתונים לאחר שהוא טעה 3 פעמים בסיסמא החד פעמית
נשלחת מ: משתמש ל: שרת
מבנה ההודעה: "FAILED_SECOND_LOGIN;{username};{ip}"
20. שם ההודעה: אישור ששם המשתמש והסיסמא נכונים
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "LOGIN_SUCCESS"
21. שם ההודעה: הודעה ששם המשתמש והסיסמא לא נכונים
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "LOGIN_FAILED"
22. שם ההודעה: כמות הכסף להעברה היא לא INT
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "INVALID_AMOUNT"
23. שם ההודעה: אין מספיק כסף להעברה
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "NO_MONEY"
24. שם ההודעה: אין חשבון להעברה
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "NO_ACCOUNT"
25. שם ההודעה: ההעברה התבצעה בהצלחה
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "TRANSFER_SUCCESS"
26. שם ההודעה: החזרת היתרה של המשתמש
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "{מספר של היתרה}"

- 27. שם ההודעה:** החזרת כל ההעברות הקודמות
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: רשימות של רשימות כל ההעברות, לדוגמא:
["tamar", "einat", 117, "2025-05-14", 69, "Shopping"]
- 28. שם ההודעה:** החזרה שאין העברות קודמות
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "NO_TRANSFERS"
- 29. שם ההודעה:** החזרה שהמספר לבקשת העברה הוא לא INT
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "INVALID_AMOUNT"
- 30. שם ההודעה:** החזרה שאין משתמש קיים לבקשת ההעברה
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "NO_ACCOUNT"
- 31. שם ההודעה:** החזרה שהבקשה אושרה בהצלחה
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "REQUEST_SUCCESS"
- 32. שם ההודעה:** החזרה שאין בקשות מהמשתמש
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "NO_REQUESTS"
- 33. שם ההודעה:** החזרת כל הבקשות מהמשתמש
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: רשימה של רשימות של כל הבקשות מהמשתמש, לדוגמא:
["einat", "tamar", 4, "False", 36, "Entertainment"]
- 34. שם ההודעה:** החזרת כמות הבקשות מהמשתמש
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "{מספר הבקשות}"
- 35. שם ההודעה:** החזרת כמות ההעברות של המשתמש
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: "{מספר ההעברות}"
- 36. שם ההודעה:** החזרת כל השנים שבהן המשתמש העביר כסף
נשלחת מ: שרת ל: משתמש
מבנה ההודעה: רשימה של כל השנים, לדוגמא: ["2025", "2024", "2023"]

37. שם ההודעה: החזרת הרשימה של כל ההוצאות החודשיות בשנה מסוימת

בקטגוריה מסוימת

נשלחת מ: שרת ל: משתמש

מבנה ההודעה: רשימה של ההוצאות בשנה הנבחרת: לדוגמא:

["0", "3450", "0", "0", "780", "0", "0", "0", "0", "80", "0", "100"]

38. שם ההודעה: החזרת מספר הפעמים שמשתמש מסוים טעה בהזנת הסיסמא שלו

נשלחת מ: שרת ל: משתמש

מבנה ההודעה: "{מספר הפעמים}"

39. שם ההודעה: החזרת כמות האנשים שטעו 3 פעמים בסיסמא מאותו הIP

נשלחת מ: שרת ל: משתמש

מבנה ההודעה: "{כמות האנשים}"

40. שם ההודעה: החזרת האימייל של שם המשתמש

נשלחת מ: שרת ל: משתמש

מבנה ההודעה: "{כתובת האימייל שלו}"

*ישנן הודעות שנשלחות על ידי השרת אך אין להן משמעות- יש צורך בהחזרת תשובה בכל מצב אך לא תמיד יש משמעות להחזרת התשובה. בפונקציות הבאות אין משמעות לתשובה שנשלחת:

1. update_attempts

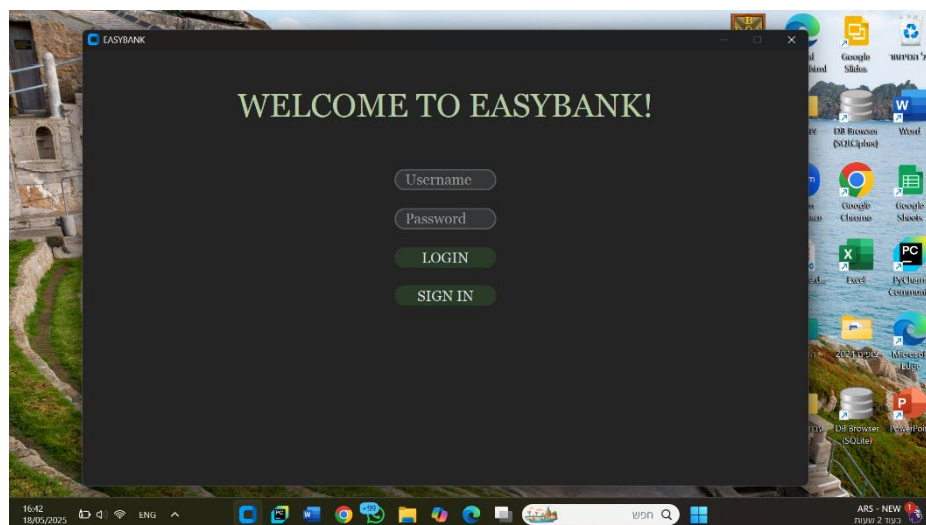
2.failedSecondLogIn

3. change_status

תיאור מסכי המערכת:

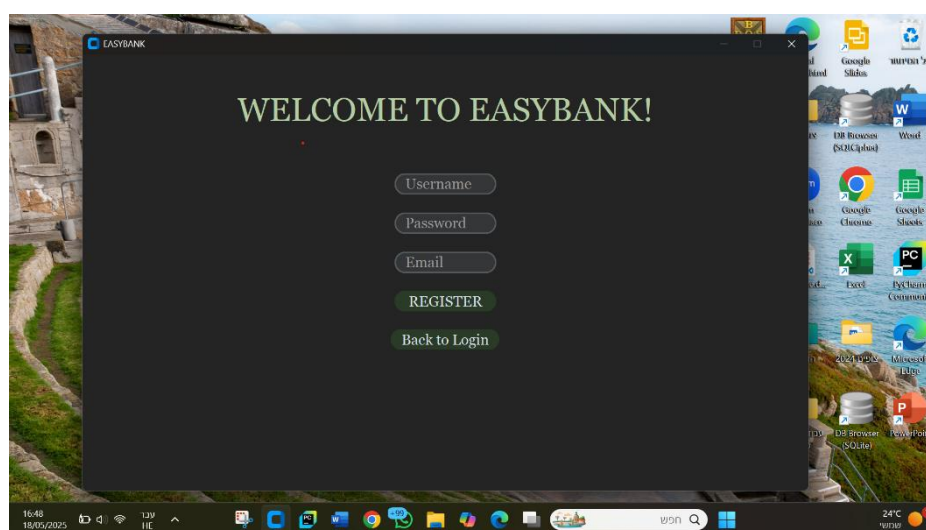
מסך התחברות:

במסך ההתחברות, המשתמש מקליד את שם המשתמש והסיסמא שלו. המערכת בודקת שכתובת הIP לא טעתה בסיסמא של 10 משתמשים שונים לפחות פעם אחד ביממה האחרונה, שכתובת הIP לא טעתה 3 פעמים בסיסמא של 5 משתמשים שונים ושכתובת הIP לא טעתה 3 פעמים בסיסמא של המשתמש שמנסה להתחבר. במידה וזה לא קרה המערכת מוודאת ששם המשתמש והסיסמא נכונים ואם הם נכונים היא מעבירה את המשתמש למסך הבא (אימות דו שלבי). אם הנתונים שגויים היא מציגה הודעת שגיאה בהתאם. בנוסף, המשתמש יכול לעבור ממסך זה למסך ההרשמה.



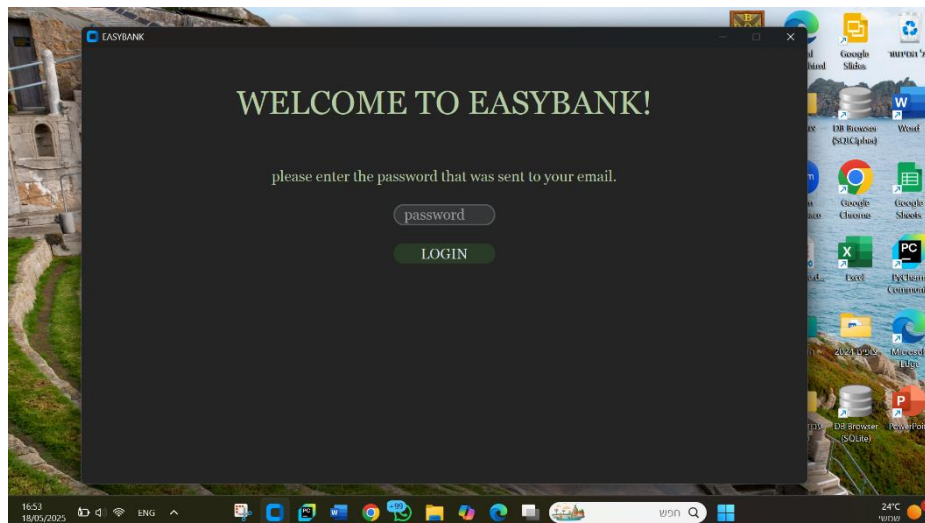
מסך רישום:

במסך ההרשמה המשתמש יכול ליצור משתמש חדש. הוא צריך להזין שם משתמש, סיסמא וכתובת אימייל. קיימת בדיקה שלא קיים משתמש אחר עם אותו שם משתמש. בנוסף, קיימת בדיקה שאין בשם המשתמש המוצע את התווים הבאים: ; , , . במידה ושם המשתמש המוצע הוא חוקי, המשתמש מועבר למסך הבא (אימות דו שלבי). אם הנתונים לא מתקבלים מוצגת הודעת שגיאה. המשתמש ממסך זה יכול לחזור למסך התחברות.



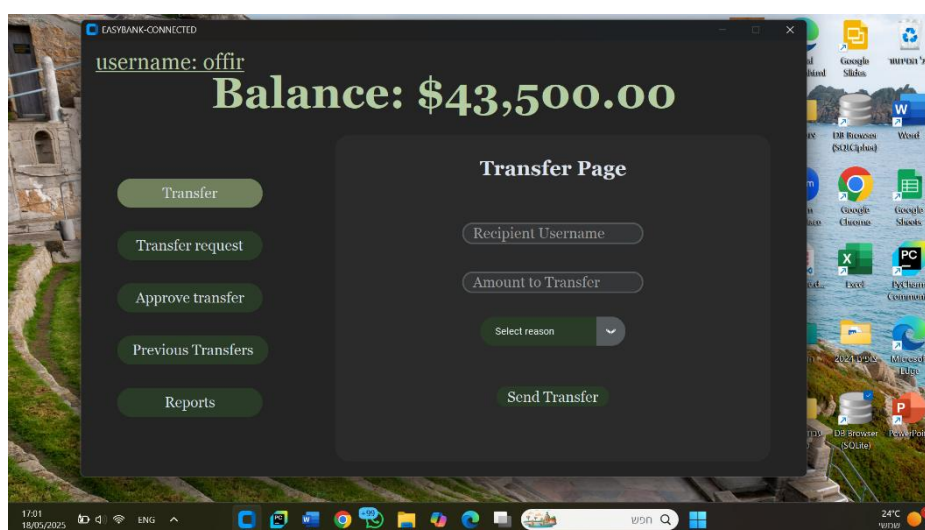
מסך אימות דו שלבי:

במסך האימות הדו שלבי על המשתמש להזין את הקוד החד פעמי שנשלח לו למייל. במידה והוא עדיין לא רשום, עד שהוא לא יזין קוד נכון הוא לא ירשם לממשק. במידה והוא רשום לממשק אם הוא טועה 3 פעמים בהזנת הקוד מהמייל, הוא יחסם מהממשק ויוכל לחזור לפעילות רק ביום למחרת. אם הוא מזין קוד נכון הוא נכנס לחשבון הבנק שלו.



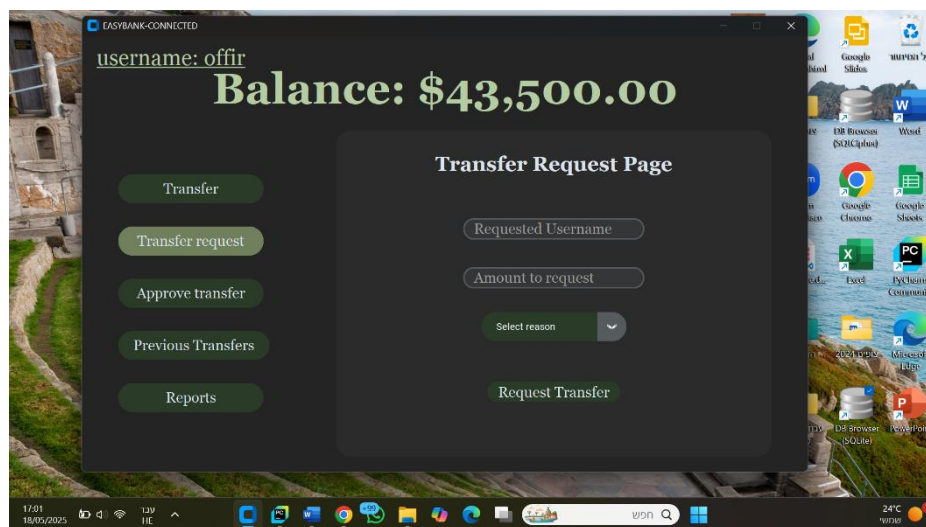
מסך העברת כספים:

במסך העברת הכספים על המשתמש להזין את המשתמש שאליו הוא רוצה להעביר כסף, כמות הכסף וסיבת ההעברה. קיימות מספר בדיקות שמוודאות שהקלט שמתקבל הוא חוקי-בדיקה כי כמות הכסף היא מספר שלם חיובי, בדיקה כי המשתמש הוא משתמש קיים ובדיקה שהשדות מלאים. במידה והכל חוקי מתבצעת העברה ומתעדכן בסיס הנתונים. אם הקלט לא חוקי מוצגת הודעת שגיאה בהתאם.



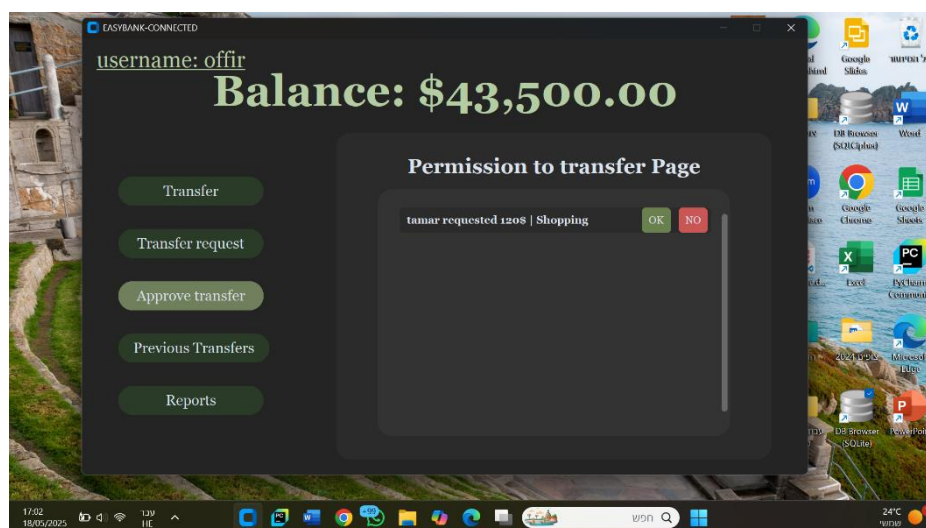
מסך בקשת העברה:

במסך בקשת ההעברה המשתמש נדרש להזין את שם המשתמש ממנו הוא מבקש את הכסף, את סכום הבקשה ואת סיבת הבקשה. קיימות מספר בדיקות שמוודאות שהקלט שמתקבל הוא חוקי- בדיקה שכמות הכסף היא מספר שלם חיובי, בדיקה כי המשתמש הוא משתמש קיים ובדיקה שהשדות מלאים. רק אם כל הבדיקות עוברות בהצלחה, הבקשה נשלחת למשתמש שאליו היא מיועדת, ומתווספת לרשימת הבקשות הממתינות לאישור או דחייה. אם הקלט לא חוקי מוצגת הודעת שגיאה בהתאם.



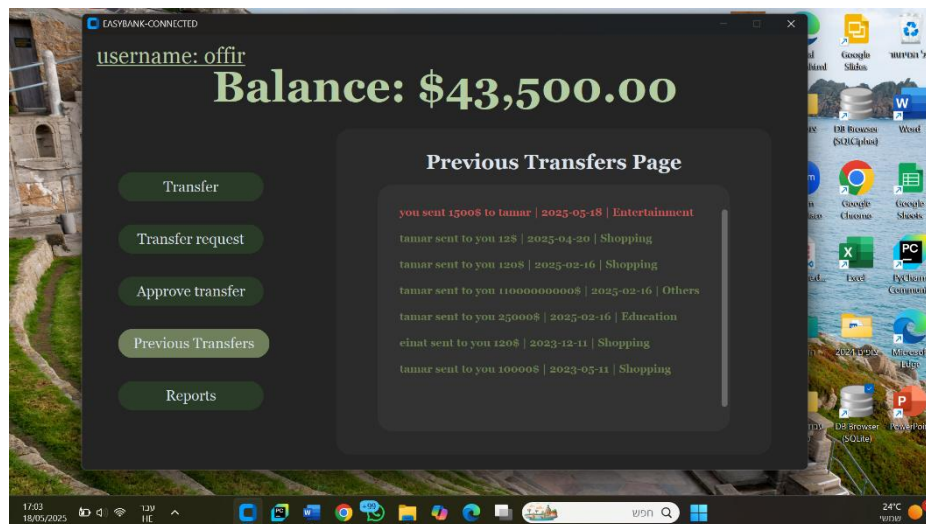
מסך אישור העברה:

במסך אישור ההעברה מוצגות למשתמש כל הבקשות שהתקבלו ממשתמשים אחרים, כולל פרטי הבקשה: שם השולח, הסכום שהתבקש וסיבת הבקשה. המשתמש יכול לבחור האם לאשר את הבקשה או לדחות אותה. כאשר בקשה מאושרת, הסכום המבוקש מועבר מחשבון המשתמש אל חשבון השולח, ובסיס הנתונים מתעדכן בהתאם. במידה והיתרה בחשבון אינה מספיקה לביצוע ההעברה, מוצגת הודעת שגיאה מתאימה, וההעברה לא מתבצעת. אם המשתמש בוחר לדחות את הבקשה, לא מתבצעת כל פעולה כספית. לאחר אישור או דחייה, הבקשה מוסרת מרשימת הבקשות הממתינות לטיפול והמשתמש לא יראה אותה יותר.



מסך צפייה בהעברות קודמות:

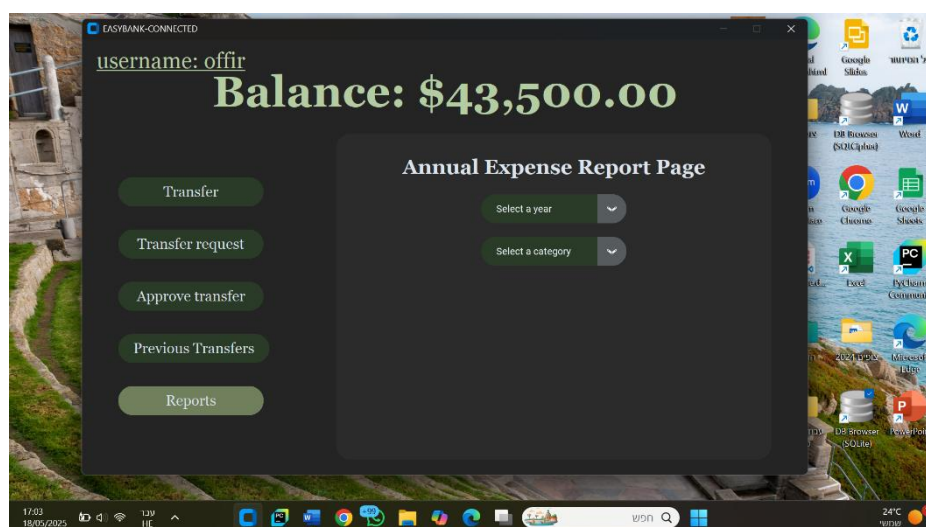
במסך צפייה בהעברות קודמות מוצגת למשתמש היסטוריית ההעברות שביצע וקיבל לאורך זמן. כל העברה מוצגת בשורה נפרדת ובצבע שונה, צבע אדום להעברה ע"י המשתמש וצבע ירוק להעברת כסף למשתמש. כל העברה כוללת את פרטי הצד השני (השולח או המקבל), הסכום שהועבר, סיבת ההעברה ותאריך הביצוע. המסך מאפשר למשתמש לעקוב אחרי הפעילות הכספית שביצע, לזהות דפוסים חוזרים, ולאמת ביצוע של פעולות קודמות במידת הצורך.

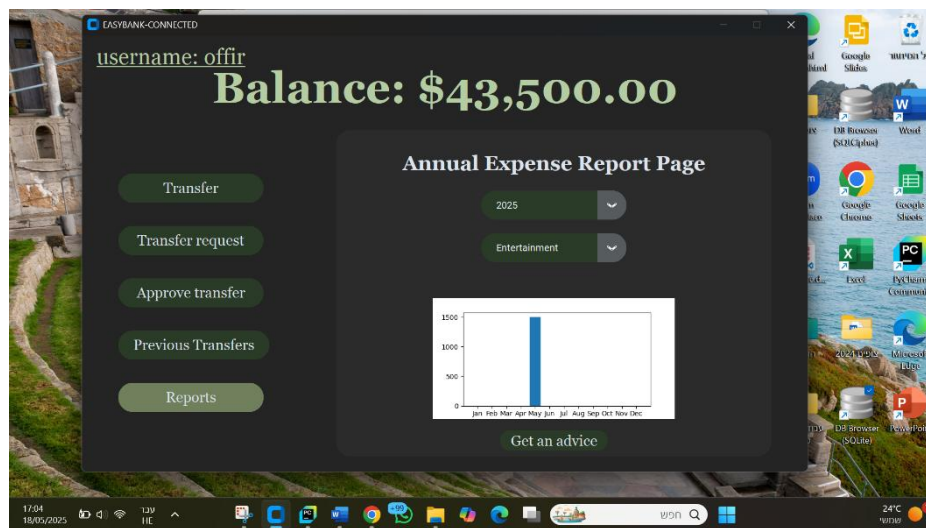


מסך צפייה בגרפי הוצאות שנתיות לפי נושאים:

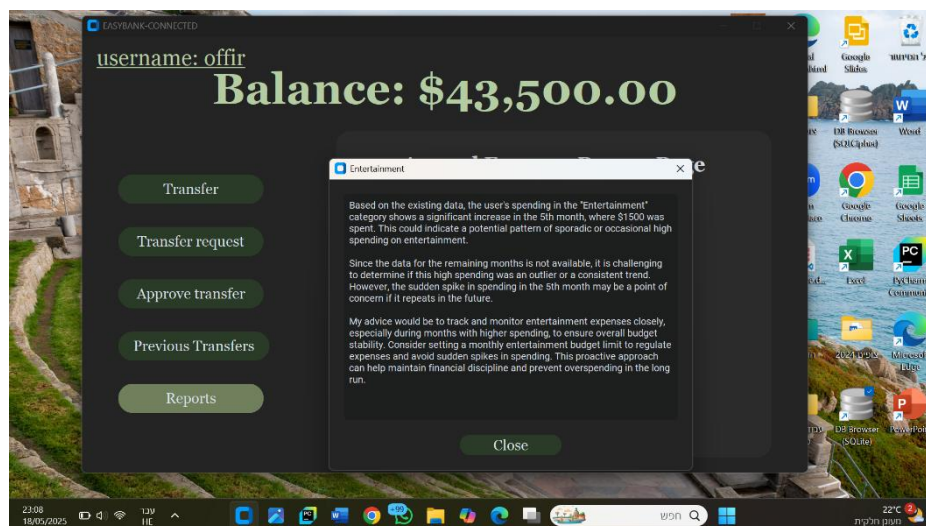
במסך צפייה בגרפים ובהוצאות שנתיות על המשתמש לבחור בשנת פעילות ובקטגוריה לגביה הוא רוצה לצפות בגרף. אם לא כל השדות מלאים, לא יוצג גרף ותוצג הודעת שגיאה בהתאם. במידה שכל השדות מלאים, יוצג גרף ההוצאות השנתי בקטגוריה זו. לאחר הצגת הגרף, מופיע כפתור שלחיצה עליו מאפשרת קבלת יעוץ פיננסי מבינה מלאכותית.

דף צפייה בגרפים לפני מילוי השדות:

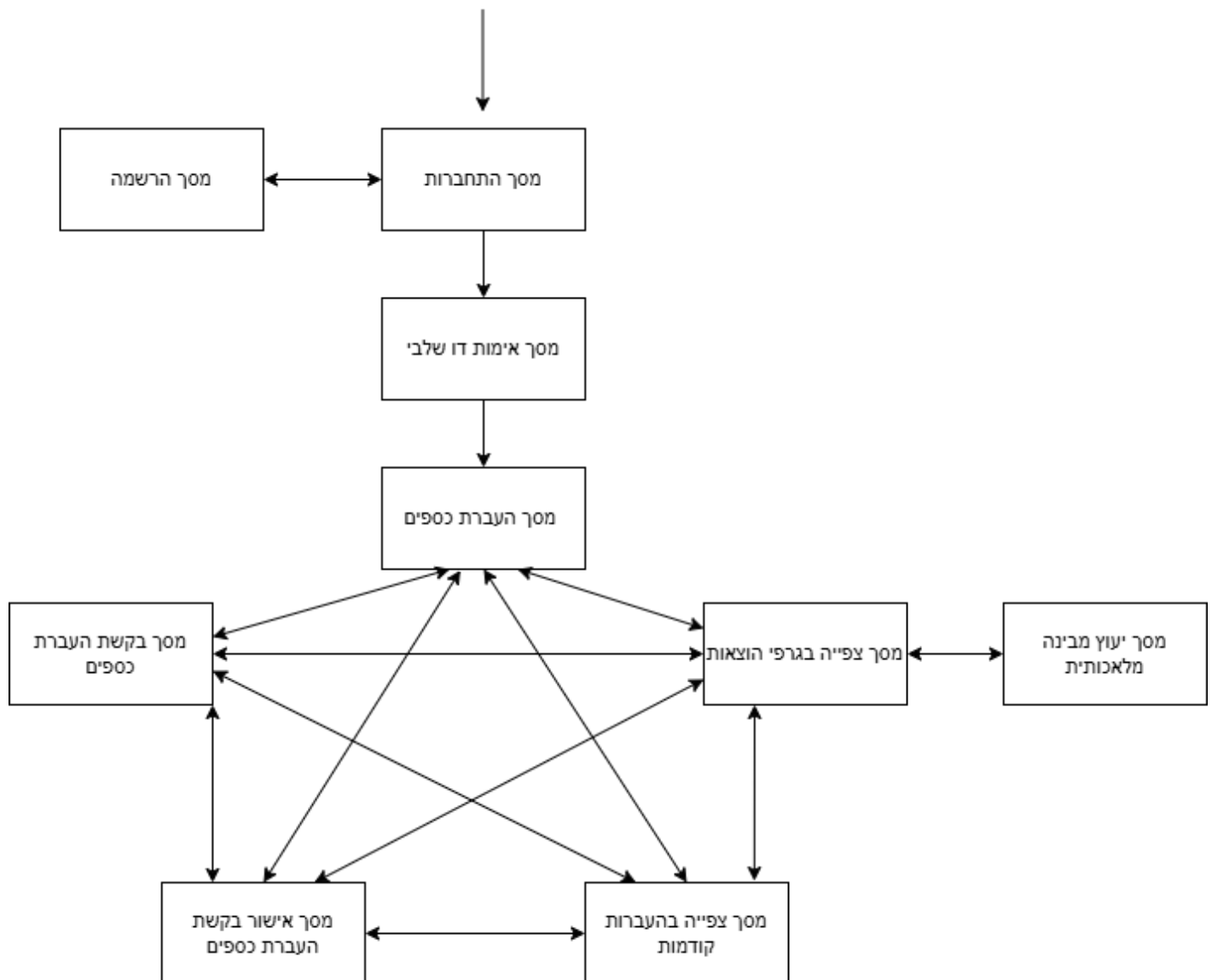


דף צפייה בגרפים לאחר מילוי השדות:**מסך קבלת יעוץ מבינה מלאכותית:**

מסך זה הוא מסך צפייה בלבד, בו המשתמש מקבל יעוץ מבינה מלאכותית לגבי ההוצאות השנתיות שלו בקטגוריה מסוימת.



תרשים זרימה של המסכים בפרויקט:



תיאור מבני הנתונים:

פרויקט זה עושה שימוש בשלושה קבצים:

בצד המשתמש:

קובץ txt המכיל בתוכו את כל המידע הרלוונטי לביצוע פעולות בצד המשתמש שיש צורך שיהיה בטוח במיוחד- מפתחות ההצפנה, קוד API לשליחת מיילים ועוד.. רק למשתמש יש גישה אליו.

בצד השרת:

קובץ txt המכיל בתוכו את כל המידע הרלוונטי לביצוע פעולות בצד השרת שיש צורך שיהיה בטוח במיוחד- מפתחות ההצפנה. רק לשרת יש גישה אליו.

קובץ db המכיל בתוכו את כל בסיס הנתונים של הפרויקט. בתוכו נמצאות 6 טבלאות (5 אני יצרתי ו- 1 נוצרה אוטומטית ומתעדכנת אוטומטית). רק לשרת יש גישה אליו.

קובץ ClientFileData.txt

נמצא בצד המשתמש. כל מידע רלוונטי נמצא בשורה נפרדת לפני החלוקה הבאה:

מספר שורה	התוכן שנמצא בשורה
1	המפתח הפרטי להצפנת RSA
2	המפתח הציבורי להצפנת RSA
3	המפתח להצפנה של הצפנת AES
4	וקטור אתחול של הצפנת AES
5	המפתח להצפנה של הצפנת DES
6	סיסמת API לשליחת מיילים
7	כתובת המייל ממנה ישלח המייל לאימות הדו שלבי
8	סיסמת API לשימוש בבינה מלאכותית

קובץ serverFileData.txt

נמצא בצד השרת. כל מידע רלוונטי נמצא בשורה נפרדת לפני החלוקה הבאה:

מספר שורה	התוכן שנמצא בשורה
1	המפתח הפרטי להצפנת RSA
2	המפתח הציבורי להצפנת RSA
3	המפתח להצפנה של הצפנת AES
4	וקטור אתחול של הצפנת AES
5	המפתח להצפנה של הצפנת DES

קובץ dataBaseTamarEyal.db

נמצא בצד השרת. נמצאות בתוכו 6 טבלאות SQLite ששונות. 1 נוצרה אוטומטית על ידי SQLite ומשתמשת למעקב אחר המספר הסידורי של שורות המידע בטבלאות.

טבלה ראשונה: users

מפתח ראשי: PRIMARY KEY("username")

שם השדה	סוג הטיפוס
Username	TEXT NOT NULL UNIQUE
Password	TEXT NOT NULL
Email	TEXT NOT NULL
Balance	INTEGER NOT NULL

דוגמא לערכים אפשריים:

username	Password	email	Balance
tamar	1234	Tamareyal07@gmail.com	990

טבלה שנייה: transfers

מפתח ראשי- PRIMARY KEY("ID" AUTOINCREMENT)

מפתחות זרים-

1. FOREIGN KEY("fromUser") REFERENCES "users"("username")

2. FOREIGN KEY("toUser") REFERENCES "users"("username")

שם השדה	סוג הטיפוס
fromUser	TEXT NOT NULL
toUser	TEXT NOT NULL
Amount	INTEGER NOT NULL
Date	TEXT NOT NULL
ID	INTEGER
reasonForTransfer	TEXT NOT NULL

דוגמא לערכים אפשריים:

fromUser	toUser	amount	date	ID	reasonForTransfer
Tamar	Einat	450	2025-05-18	21	Education

טבלה שלישית: requests

מפתח ראשי- PRIMARY KEY("ID" AUTOINCREMENT)

מפתחות זרים-

1. FOREIGN KEY("requestedFrom") REFERENCES "users"("username")
 2. FOREIGN KEY("personWhoRequested") REFERENCES "users"("username")

סוג הטיפוס	שם השדה
TEXT NOT NULL	personWhoRequested
TEXT NOT NULL	requestedFrom
INTEGER NOT NULL	amountRequested
TEXT NOT NULL	Treated
INTEGER	ID
TEXT NOT NULL	reasonForTransfer

דוגמא לערכים אפשריים:

Person Who Requested	Requested From	Amount Requested	treated	ID	Reason For Transfer
Einat	Tamar	100	True	10	Shopping

טבלה רביעית: cyber protection

מפתח ראשי- PRIMARY KEY("ID" AUTOINCREMENT)

סוג הטיפוס	שם השדה
TEXT NOT NULL	Ip
TEXT NOT NULL	Date
INTEGER NOT NULL	amountOfAttempts
TEXT NOT NULL	Username
INTEGER	ID

דוגמאות לערכים אפשריים:

Ip	Date	amountOfAttempts	username	ID
192.168.1.132	2025-05-13	1	Einat	14

טבלה חמישית: communication

מפתח ראשי- PRIMARY KEY("ID" AUTOINCREMENT)

סוג הטיפוס	שם השדה
INTEGER	ID
TEXT NOT NULL	Ip
TEXT NOT NULL	Status
TEXT NOT NULL	Date

דוגמאות לערכים אפשריים:

ID	ip	status	Date
5	192.168.1.132	OFF	2025-05-13

סקירת חולשות ואיומים:

שכבת האפליקציה:

DDOS - במערכת קיים איום שבו תוקף ינסה לחבר עשרות משתמשים לשרת ולהקריס אותו. הפתרון לכך אותו מצאתי ויישמתי בפרויקט שלי הוא מעקב אחר כמות המשתמשים המחוברים בכל רגע נתון. משתמש שמנסה להתחבר מעבר לכמות המוגדרת יקבל הודעת שגיאה וייסגר.

תהליך ההתחברות - בתהליך ההתחברות קיים איום שתוקף ינסה לפרוץ לחשבון באמצעות ניחוש סיסמאות. על מנת לפתור איום זה המערכת מבצעת מספר רב של בדיקות כדי לוודא שהמשתמש הוא המשתמש האמיתי. המערכת בודקת שכתובת ה IP לא טעתה במהלך היממה האחרונה בסיסמא של 10 משתמשים שונים לפחות פעם אחת, שהיא לא טעתה בסיסמא של 5 משתמשים שונים 3 פעמים ושהיא לא טעתה בסיסמא של המשתמש שמנסה להתחבר 3 פעמים. במידה והיא כן, המערכת חוסמת את כתובת ה IP וגם אם המשתמש מנסה באותה הפעם להתחבר בסיסמא נכונה, לא תתאפשר לו הכניסה באותה היממה. בנוסף קיים אימות דו שלבי שבמהלכו נשלח קוד חד פעמי למייל אותו יש להקליד במערכת. אם המשתמש טועה בהקלדתו 3 פעמים הוא יהיה חסום מהמערכת ליממה הקרובה.

Injection SQL - בפרויקט שלי ישנו מידע רגיש ואישי על כל משתמש. קיים איום שבו משתמש זדוני יזין קלט עם קוד SQL במטרה לשלוף, לעדכן או למחוק מידע מבסיס הנתונים. על מנת לפתור חולשה זו, כל השאילתות בפרויקט מבוצעות עם פרמטרים.

גניבת סיסמא - כדי להגן על הסיסמאות, שהן מידע רגיש מאוד, הסיסמאות מוצפנות בבסיס הנתונים בהצפנת האש.

שכבת התעבורה:

MITM - על מנת למנוע מצב שבו אדם חיצוני מצותת למידע שעובר בין המחשבים, כל הודעה שעוברת מוצפנת בהצפנה כלשהי. ההצפנה נבחרת בצורה רנדומלית. ההצפנות שממומשות הן AES, DES, RSA.

הסבר מתמטי על כל אחת מההצפנות במערכת:

DES - הוא אלגוריתם הצפנה סימטרי (כלומר אותו מפתח משמש להצפנה ולפענוח). פועל על בלוקים בגודל 64 ביטים עם מפתח של 56 ביטים. DES בנוי מ-16 סבבים של תהליך הקרוי Feistel Rounds וכל סבב כולל סדרה של פעולות מתמטיות.

מחלקים את הבלוקים לשני חלקים בגודל של 32 ביטים. בכל סבב, חצי מהמידע - כלומר הבלוק בגודל 32 ביטים עובר דרך פונקציה מסובכת שכוללת XOR שיבוץ בטבלאות

מיוחדות (S-Boxes) ופרמוטציות – והחצי השני מתערבב איתו. בסוף, כל זה הופך את המידע לבלתי קריא בלי המפתח.

RSA – היא שיטת הצפנה אסימטרית (כלומר היא משתמשת בשני מפתחות שונים – אחד להצפנה ואחד לפענוח). המפתח הציבורי פתוח וגלוי לכולם, ואיתו מצפינים את המידע. המפתח הפרטי שמור אצל המקבל בלבד, ורק בעזרתו אפשר לפענח את ההודעה. כדי ליצור את המפתחות, בוחרים שני מספרים ראשוניים גדולים, שאין להם מחלקים משותפים, ומבצעים עליהם חישובים מתמטיים שיוצרים את המפתח הציבורי, המפתח הפרטי, ומספר שנקרא מודולו-M.

ברגע שיש את המפתחות, תהליך ההצפנה נעשה לפי נוסחה זו:
(הודעה בחזקה של המפתח הציבורי) מודולו M וכך מתקבל טקסט מוצפן.
הפענוח נעשה על פי אותו רעיון, רק עם המפתח הפרטי:
(הטקסט המוצפן בחזקה של המפתח הפרטי) מודולו M והתוצאה היא ההודעה המקורית.

AES – זוהי שיטת הצפנה סימטרית. היא עובדת על בלוקים של 128 ביט, והיא תומכת במפתחות של 128, 192 או 256 ביט.

כל בלוק עובר דרך מספר סבבים – תלוי באורך המפתח. בכל אחד מהסבבים מבוצעות הפעולות הבאות: החלפת ערכים בטבלה קבועה (SubBytes), שיבוש הסדרים (ShiftRows), שילוב עמודות (MixColumns), חיבור עם המפתח (AddRoundKey).

HASH – זוהי דרך הצפנה חד כיוונית, לאחר ההצפנה אין דרך לדעת מהי ההודעה שהוצפנה. זה שימושי לאימות מידע אך לא להצפנת הודעות. קשה מאוד וכמעט בלתי אפשרי למצוא שני קלטים שיוצרים את אותו הפלט, ושני קלטים מאוד דומים יוצרים פלטים שונים לחלוטין.

פרק 4- מימוש הפרויקט:

חלק א:

סקירת כל המודולים ומחלקות המרכיבים את המערכת:

מודולים /מחלקות מיובאים:

שם המודל המיובא: socket

למה מיועד המודל: יצירת חיבור ותקשורת בין שרת ולקוח.

שם המודל המיובא: threading

למה מיועד המודל: התמודדות עם הצורך לבצע מספר פעולות בו זמנית.

שם המודל המיובא: json

למה מיועד המודל: משמש להמרה בין אובייקטים של פייתון לבין מחרוזות JSON

שם המודל המיובא: random

למה מיועד המודל: בחירה רנדומלית של ערכים

שם המודל המיובא: os

למה מיועד המודל: מודול לעבודה עם מערכת ההפעלה לדוגמא: קבצים

שם המודל המיובא: sys

למה מיועד המודל: מספק גישה לפונקציות ולנתונים של סביבת הריצה של פייתון

שם המודל המיובא: time

למה מיועד המודל: השהיות, מדידת זמן, תזמון פעולות

שם המודל המיובא: sqlite3

למה מיועד המודל: ממשק למסד נתונים SQLite

שם המודל המיובא: datetime

למה מיועד המודל: עבודה עם תאריכים, שעות ופורמט תאריך.

שם המודל המיובא: tkinter.messagebox as tkmb

למה מיועד המודל: הצגת תיבות טקסט Tkinter

שם המודל המיובא: tkinter as tk

למה מיועד המודל: ספריית GUI בסיסית בפייתון

שם המודל המיובא: customtkinter as ctk

למה מיועד המודל: הרחבה מודרנית של Tkinter ל GUI מותאם אישית

שם המודל המיובא: matplotlib.figure

למה מיועד המודל: יצירת גרפים ותרשימים

שם המודל המיובא: matplotlib.backends.backend_tkagg

למה מיועד המודל: שילוב גרפים של matplotlib בתוך ממשק Tkinter

שם המודל המיובא: openai

למה מיועד המודל: גישה ל API של OPENAI

שם המודל המיובא: hashlib

למה מיועד המודל: מאפשר לעשות האש למחרוזות בפיתון.

שם המודל המיובא: smtplib

למה מיועד המודל: שליחת מיילים דרך שרת SMTP

שם המודל המיובא: email.mime.text

למה מיועד המודל: יצירת תוכן מייל בפורמט טקסט

שם המודל המיובא: email.mime.multipart

למה מיועד המודל: יצירת מיילים עם חלקים מרובים

שם המודל המיובא: base64

למה מיועד המודל: קידוד/פענוח של נתונים בפורמט Base64

שם המודל המיובא: cryptography.hazmat.primitives.ciphers

למה מיועד המודל: הגדרת הצפנות סימטריות כמו AES

שם המודל המיובא: cryptography.hazmat.backends

למה מיועד המודל: בחירת backend להצפנה

שם המודל המיובא: cryptography.hazmat.primitives

למה מיועד המודל: אלגוריתמים קריפטוגרפיים בסיסיים

שם המודל המיובא: binascii

למה מיועד המודל: המרה בינארית והקסדצימלית

שם המודל המיובא: Cryptodome.Cipher

למה מיועד המודל: הצפנה/פענוח עם אלגוריתמים כמו DES

שם המודל המיובא: Cryptodome.Util.Padding

למה מיועד המודל: הוספה/הסרה של padding לבלוקים בהצפנה

שם המודל המיובא: Crypto.Cipher

למה מיועד המודל: הצפנה/פענוח עם אלגוריתמים כמו DES

שם המודל המיובא: Crypto.Util.Padding

למה מיועד המודל: הוספה/הסרה של padding לבלוקים בהצפנה

מחלקות שאני מפתחת:

מחלקה - Client

תפקיד המחלקה - לטפל בתקשורת של צד הלקוח עם השרת - כולל יצירת חיבור, שליחת בקשות, קבלת תגובות, והצפנת/פענוח מידע באמצעות שלוש שיטות הצפנה שונות.

תכונות המחלקה:

שם התכונה:	תפקיד ושימוש:
server_ip	כתובת ה IP של השרת
Port	מספר הפורט של השרת
Client	אובייקט socket החיבור עצמו לשרת
Encrypt	רשימה של שלוש מחלקות הצפנה שונות
local_ip	כתובת IP של הלקוח
API_key_email	מפתח API לשליחת מיילים
from_email	כתובת המייל שממנה ישלחו הודעות
API_key_AI	מפתח API של OpenAI

פעולות המחלקה:

שם הפעולה:	טענת כניסה:	טענת יציאה:
__init__	כתובת IP ופורט של השרת, אך יש ברירת מחדל	מאתחל חיבור לשרת, קורא מפתחות הצפנה מקובץ, יוצר שלוש מחלקות הצפנה
Connect	הפונקציה לא מקבלת משתנים	מבצע התחברות לשרת, מקבל הודעת פתיחה, אם אין תגובה או שיש בעיית עומס - סוגר את החיבור ומציג הודעת שגיאה
send_request	מחרוזת בקשה שנשלחת למשתמש	מצפין את הבקשה באחת משלוש שיטות (ההצפנה נבחרת באופן רנדומלי), שולח לשרת, מקבל תשובה מוצפנת ומפענח אותה. מחזיר את התגובה בצורת טקסט.
Close	הפונקציה לא מקבלת משתנים	סוגר את חיבור ה socket לשרת
getRandom	הפונקציה לא מקבלת משתנים	מחזיר מספר אקראי בין 0-2
getLocal_ip	הפונקציה לא מקבלת משתנים	מחזיר את כתובת ה IP של המשתמש
getAPIkey	הפונקציה לא מקבלת משתנים	מחזיר את מפתח ה API לשימוש בשליחת מיילים

getFromEmail	הפונקציה לא מקבלת משתנים	מחזיר את כתובת המייל שממנה נשלח קוד האימות החד פעמי
checkClientDataFile	הפונקציה לא מקבלת משתנים	בודק האם כל קבצי המערכת הדרושים קיימים אם לא מציג הודעה וסוגר את התוכנית
getAPI_key_AI	הפונקציה לא מקבלת משתנים	מחזיר את מפתח ה API של OpenAI

מחלקה - ClientGUIFirstPage

תפקיד המחלקה הוא- ממשק גרפי ראשוני של לקוח במערכת הבנקאות. מתאפשרים התחברות, רישום, אימות באמצעות מייל וניהול תצוגת GUI.

תכונות המחלקה:

שם התכונה:	תפקיד ושימוש:
Client	מופע של האובייקט client, משמש לשליחת בקשות לשרת וקבלת מידע רלוונטי
App	חלון ה GUI הראשי
signin_mode	מציין אם המשתמש נמצא במצב הרשמה
Font	שם הפונט המוגדר לעיצוב כל הטקסטים
hover_color	צבע הכפתורים בעת מעבר עכבר
button_color	צבע ברירת מחדל של הכפתורים
usernameR	פרטי שם המשתמש- לשלב השני של הרישום
passwordHashesR	פרטי הסיסמא- לשלב השני של הרישום
emailR	פרטי כתובת האימייל- לשלב השני של הרישום
passwordRegester	הסיסמא שנשלחה למייל באימות האישום
passwordSecondLogin	הסיסמא שנשלחה למייל באימות הכניסה
Username	שם המשתמש שנכנס למערכת
Counter	מונה נסיונות כושלים בשלב השני של ההתחברות

פעולות המחלקה:

שם הפעולה:	טענת כניסה:	טענת יציאה:
__init__	הפונקציה לא מקבלת משתנים	מתחיל את חלון GUI ומפעיל את הלולאה הראשית. קורא ל creat
Creat	הפונקציה לא מקבלת משתנים	יוצר ומעצב את חלון התצוגה הראשי ומפעיל את פונקציית create_widgets

GUI מוסיף את רכיבי ה-GUI למסך	הפונקציה לא מקבלת משתנים	create_widgets
סוגר את התוכנית, כולל ביטול פעולות מתוזמנות	הפונקציה לא מקבלת משתנים	on_close
מבצע ניסיון התחברות, כולל בדיקת ניסיונות כושלים, שליחה לשרת. מעבר לשלב אימות מייל אם הצליח	הפונקציה לא מקבלת משתנים	Login
משנה את GUI למצב הרשמה	הפונקציה לא מקבלת משתנים	switch_to_signin
מחזיר את GUI למצב התחברות	הפונקציה לא מקבלת משתנים	switch_to_login
שולח בקשה לשרת לבדוק אם המשתמש קיים. אם לא, עובר לשלב אימות מייל	הפונקציה לא מקבלת משתנים	Register
מחזיר True אם שם המשתמש ארוך מ-8 תווים	הפונקציה מקבלת את שם משתמש	checkDigit
מחזיר True אם שם המשתמש מכיל, או ;	הפונקציה מקבלת את שם משתמש	checkChars
מציג שדות לאימות, סיסמה שנשלחה למייל, שולח סיסמה דרך Gmail	הפונקציה לא מקבלת משתנים	second_Register
בודק האם הסיסמה מהמייל נכונה, אם כן משלים את הרישום	הפונקציה לא מקבלת משתנים	second_Register_entery
שולח סיסמה למייל של המשתמש ומכין את ממשק האימות	הפונקציה מקבלת את שם משתמש	emailPassword
בודק האם הסיסמה מהמייל נכונה. אם לא סופר ניסיונות כושלים	הפונקציה לא מקבלת משתנים	secondLogin

מחלקה - secondPageGUI

תפקיד המחלקה הוא להציג את הממשק הגרפי של דף הבית לאחר התחברות המשתמש- הממשק מאפשר ביצוע פעולות בנקאיות כמו העברות, בקשות, אישורים, הצגת העברות קודמות, הפקת גרפים ודוחות שנתיים עם ניתוח מבוסס בינה מלאכותית.

תכונות המחלקה:

שם התכונה:	תפקיד ושימוש:
Client	מופע של המחלקה Client לביצוע שליחת בקשות לשרת
App	חלון ה-GUI הראשי של המשתמש המחובר
Username	שם המשתמש המחובר

שם הפונט המוגדר לעיצוב כל הטקסטים	Font
צבע הכפתורים בעת מעבר עכבר	hover_color
צבע ברירת מחדל של הכפתורים	button_color
צבע ברירת המחדל של הטקסט	text_color
פריימים נפרדים לכל מסך בממשק- העברה, בקשה, אישור בקשה, דוחות והעברות קודמות	()_frame
קנבס להצגת גרפים סטטיסטיים	statistics_canvas
מספר הבקשות של המשתמש	amountOfRequested
מספר ההעברות של המשתמש	amoutOfTransfers
רשימת ההוצאות החודשיות עבור גרף סטטיסטי	yearSpending

פעולות של המחלקה:

שם הפעולה:	טענת כניסה:	טענת יציאה:
__init__	אובייקט client של הלקוח ושם המשתמש שלו	מאתחלת את ה-GUI הראשי, שולפת נתונים ראשוניים, יוצרת את המסכים השונים ומרעננת אוטומטית
creat	הפונקציה לא מקבלת משתנים	מציבה את חלון האפליקציה במרכז המסך, מציגה את שם המשתמש והיתרה
createButtons	הפונקציה לא מקבלת משתנים	בונה את כל המסכים השונים, כולל רכיבי קלט, כפתורים ותפריטים נפתחים
transerWasPicked, requestWasPicked, approveWasPicked, previousWasPicked, statisticsWasPicked	הפונקציה לא מקבלת משתנים	מסתירות את כל הפריימים האחרים, מראות את הפריימס הרלוונטי, משנות את צבע כפתור הפעולה שנבחר
clear_fields_Transfers, clear_fields_requests, clear_fields_statistics	הפונקציה לא מקבלת משתנים	מאפסות שדות קלט ותפריטים לפי הצורך
send_transfer	אופציונליים- ניתן להעביר פרטים (בזמן אישור העברה), או לאחר מכן לקבל את הפרטים מהקלט של המשתמש	מבצעת שליחה של העברה בין משתמשים לאחר בדיקות תקינות
sendRequest	הפונקציה לא מקבלת משתנים	שולחת בקשת העברה למשתמש אחר
showAmount	הפונקציה לא מקבלת משתנים	מציגה את היתרה המעודכנת של המשתמש
showPrevious	הפונקציה לא מקבלת משתנים	מציגה את ההעברות שבוצעו נכסות ויוצאות
refresh	הפונקציה לא מקבלת משתנים	מרעננת יתרה, העברות ובקשות

refreshRequests	הפונקציה לא מקבלת משתנים	רענון רק של בקשות
refreshAutomatic	הפונקציה לא מקבלת משתנים	בודקת אם יש שינוי בכמות ההעברות בקשות, ומרעננת בהתאם
start_auto_refresh	הפונקציה לא מקבלת משתנים	מריצה את פעולת הרענון כל 5 שניות
show_requests	הפונקציה לא מקבלת משתנים	מציגה את הבקשות שקיבל המשתמש, עם אפשרות לאשר או לדחות כל אחת
decline	הפונקציה מקבל מספר זיהוי של הבקשה (ID)	מסמנת את הבקשה כדחוייה בשרת
approve	הפונקציה מקבלת את מספר זיהוי של הבקשה, פרטי המשתמש המבקש, סכום, וסיבה	מאשרת את הבקשה ומבצעת את ההעברה בפועל
show_graph	הפונקציה לא מקבלת משתנים	אם שני השדות נבחרו- נקראת הפונקציה show_graph, אחרת מוצגת הודעת שגיאה
show_graph	הפונקציה מקבלת קטגוריה ושנת פעילות	מציגה גרף עם נתוני ההוצאות לפי חודשים, מציגה כפתור לקבלת עצה פיננסית
get_advice	הפונקציה לא מקבלת משתנים	שולחת את ההוצאות OpenAI ומחזירה עזת חיסכון מותאמת אישית
show_advice	הטקסט של העצה	מציגה חלונית עם ההעצה שהתקבלה

מחלקה- GmailEmailSender

תפקיד המחלקה הוא לשלוח דואר אלקטרוני עם סיסמא חד-פעמית דרך שרת Gmail, לצורך האימות של המשתמש.

תכונות המחלקה:

שם התכונה:	תפקיד ושימוש:
from_user	כתובת המייל שממנה נשלחת ההודעה
app_key	סיסמת API של אפליקצית Gmail
to_user	כתובת המייל של המשתמש המקבל

פעולות המחלקה:

שם הפעולה:	טענת כניסה:	טענת יציאה:
__init__	הפונקציה מקבלת את מפתח API של האפליקציה, כתובת המייל של השולח וכתובת	מאתחלת את השדות הדרושים לשליחת מייל

	המייל שאליו שולחים את ההודעה	
מחזירה מספר רנדומלי בן 6 ספרות בין 100000 ל-999999	הפונקציה לא מקבלת משתנים	get_random
שולחת מייל דרך Gmail עם קוד חד פעמי ומחזירה את הקוד	הפונקציה לא מקבלת משתנים	send_email

מחלקה - Encryptions

להוות ממשק בסיס (interface) למחלקות הצפנה אחרות, עם מתודות encrypt ו decrypt ריקות שנדרשות למימוש במחלקות היורשות.

אין למחלקה זו תכונות.

פעולות המחלקה:

שם הפעולה:	טענת כניסה:	טענת יציאה:
Encrypt	הפונקציה לא מקבלת משתנים	מוגדרת כ- placeholder בלבד
Decrypt	הפונקציה לא מקבלת משתנים	מוגדרת כ- placeholder בלבד

מחלקה - Encryption1

תפקידה ליישם הצפנה ופענוח באמצעות אלגוריתם RSA.

תכונות המחלקה:

שם התכונה:	תפקיד ושימוש:
public_key_data	מעריך החזקה של המפתח הציבורי
mod_data	מודולו- בשביל האלגוריתם המתמטי

פעולות המחלקה:

שם הפעולה:	טענת כניסה:	טענת יציאה:
__init__	הפונקציה מקבלת מפתח ציבורי ומודולו	מאתחלת את מערכת ההצפנה
Encrypt	מחרוזת טקסט עם הודעה	מחזירה מחרוזת מוצפנת המורכבת ממספרים מופרדים בפסיקים
Decrypt	מחרוזת טקסט מוצפנת	מפענחת ומחזירה את הטקסט המקורי

מחלקה- Encryption2

תפקידה ליישם הצפנה ופענוח באמצעות אלגוריתם AES.

שם התכונה:	תפקיד ושימוש:
Key	מפתח AES
lv	וקטור אתחול באורך 16 בתים

פעולות המחלקה:

שם הפעולה:	טענת כניסה:	טענת יציאה:
__init__	הפונקציה מקבלת מפתח הצפנה ווקטור אתחול	מאתחלת את מערכת ההצפנה
Encrypt	מחרוזת טקסט עם הודעה	מחזירה טקסט מוצפן לאחר הצפנת AES
Decrypt	מחרוזת טקסט מוצפנת	מפענחת ומחזירה את הטקסט המקורי

מחלקה- Encryption3

תפקידה ליישם הצפנה סימטרית בעזרת אלגוריתם DES.

תכונות המחלקה:

שם התכונה:	תפקיד ושימוש:
Key	מפתח ההצפנה

פעולות המחלקה:

שם הפעולה:	טענת כניסה:	טענת יציאה:
__init__	הפונקציה מקבלת את מפתח ההצפנה של DES	אתחול מחלקת ההצפנה עם המפתח
Encrypt	מחרוזת טקסט עם הודעה	מחזירה טקסט מוצפן לאחר הצפנת DES
Decrypt	מחרוזת טקסט מוצפנת	מפענחת ומחזירה את הטקסט המקורי

מחלקה- Server

תפקידה לנהל שרת שמטפל בבקשות של לקוחות, כולל רישום, התחברות, העברות כספים, בקשות תשלום, ניהול ניסיונות כושלים ועוד.. תוך שימוש בהצפנה, מסד נתונים, וריבוי תהליכים.

תכונות המחלקה:

שם התכונה:	תפקיד ושימוש:
------------	---------------

אובייקט socket ליצירת שרת TCP	Server
מופע של מחלקת Database לניהול הנתונים	Database
רשימה של חיבורים פעילים	Clients
מנגנון נעילה לעבודה בטוחה עם ממסד נתונים בריבוי תהליכים	Lock
רשימה של שלוש מחלקות הצפנה AES DES RSA לצורך תקשורת עם לקוחות	Encrypt

פעולות המחלקה:

שם הפעולה:	טענת כניסה:	טענת יציאה:
__init__	הפונקציה מקבלת כתובת IP ופורט	יוצרת את הsocket של השרת, טוענת את ממסד הנתונים וטוענת את מפתחות ההצפנה מהקובץ
Start	הפונקציה לא מקבלת משתנים	מאזינה לחיבורים נכנסים, בודקת אם יש פחות מ-4 מחוברים ואם כן מוסיפה את החיבור ומטפלת בהם.
actionSelection	מקבלת אובייקט client וכתובת IP	מקבלת הודעות מהלקוח ומפענחת אותן, מפעילה את הפונקציה המתאימה. כשהלקוח מתנתק סוגרת את החיבור ומעדכנת בממסד הנתונים
sendClient	הפונקציה מקבלת אובייקט של משתמש והודעה להעברה	מצפינה את הודעה ללקוח לפי שיטה אקראית מתוך שלוש השיטות, ושולחת אותה.
getRandom	הפונקציה לא מקבלת משתנים	מחזירה מספר אקראי בין 0-2
checkClientDataFile	הפונקציה לא מקבלת משתנים	בודקת אם כל הקבצים הדרושים לשרת קיימים. אם לא יוצאת מהתוכנית
checkHowManyConected	הפונקציה לא מקבלת משתנים	בודקת כמה לקוחות מחוברים כרגע לפי המסד, ומחזירה את הכמות
changStatusCommunication	הפונקציה מקבלת את כתובת הIP של המשתמש	מעדכנת סטטוס ניתוק של IP
insertCommunication	הפונקציה מקבלת את כתובת הIP של המשתמש	רושמת במסד הנתונים ש IP התחבר לשרת
Login	הפונקציה מקבלת אובייקט מסוג client, שם משתמש וסיסמא	בודקת סיסמא במסד הנתונים
Register	הפונקציה מקבלת אובייקט מסוג client, שם משתמש, סיסמא וכתובת אימייל	מוסיפה משתמש חדש למסד הנתונים

מבצעת העברה בין חשבונות	הפונקציה מקבלת אובייקט מסוג client, משתמש מעביר, משתמש שמעבירים לו, כמות וסיבה	Transfer
מחזירה למשתמש את יתרת המשתמש	הפונקציה מקבלת אובייקט מסוג client, ושם משמש	returnBalnce
מחזירה למשתמש רשימת העברות קודמות	הפונקציה מקבלת אובייקט מסוג client, ושם משתמש	returnPrevious
רושמת בקשת תשלום	הפונקציה מקבלת אובייקט מסוג client, מי מבקש, ממי מבקש, כמות וסיבה	Requests
מחזירה למשתמש את הבקשות שנשלחו אליו	הפונקציה מקבלת אובייקט מסוג client, ושם משתמש	get_requests
משנה סטטוס של בקשה	הפונקציה מקבלת אובייקט מסוג client, ומספר זיהוי של בקשה	change_status
מחזירה למשתמש כמה בקשות התקבלו	הפונקציה מקבלת אובייקט מסוג client, ושם משתמש	getAmountRequest
מחזירה למשתמש כמה העברות בוצעו	הפונקציה מקבלת אובייקט מסוג client, ושם משתמש	getAmountOfTransfers
מחזירה למשתמש שנים שבהן בוצעו העברות	הפונקציה מקבלת אובייקט מסוג client, ושם משתמש	getYears
מחזירה למשתמש הוצאות חודשיות לפי שנה וקטגוריה	הפונקציה מקבלת אובייקט מסוג client, שם משתמש, שנה וקטגוריה	getYearSpending
מחזירה למשתמש כמה ניסיונות כושלים היו למשתמש ב IP הזה	הפונקציה מקבלת אובייקט מסוג client, שם משתמש וכתובת IP	check_Attempts
מעדכנת ניסיון כושל	הפונקציה מקבלת אובייקט מסוג client, שם משתמש וכתובת IP	update_attempts
מחזירה למשתמש לכמה משתמשים ה IP טעה 3 פעמים בסיסמא	הפונקציה מקבלת אובייקט מסוג client, וכתובת IP	checkAmountFailAttempts
מחזירה למשתמש לכמה משתמשים ה IP טעה פעם אחת בסיסמא	הפונקציה מקבלת אובייקט מסוג client, וכתובת IP	checkAllAmountFailAttempts

getEmail	הפונקציה מקבלת אובייקט מסוג client, ושם משתמש	מחזירה את כתובת המייל של המשתמש
failedSecondLogin	הפונקציה מקבלת אובייקט מסוג client, שם משתמש וכתובת IP	רושמת כשולן בהזדהות דו-שלבית
checkRegistration	הפונקציה מקבלת אובייקט מסוג client, ושם משתמש	בודקת אם שם המשתמש פנוי

מחלקה - Database

תפקיד המחלקה היא לנהל את כל הגישה למסד הנתונים SQLite עבור אפליקציית הבנק-
כולל רישום משתמשים, אימות, ניהול העברות ובקשות, שמירת תקשורת, ומעקב אבטחה
(ניסיונות כושלים).

תכונות המחלקה:

שם התכונה:	תפקיד ושימוש:
db_name	שם קובץ מסד הנתונים

פעולות המחלקה:

שם הפעולה:	טענת כניסה:	טענת יציאה:
__init__	הפונקציה מקבלת את שם הקובץ של מסד הנתונים- אופציונלי	שומר את שם המסד ויוצר את הטבלאות הדרושות
create_tables	הפונקציה לא מקבלת משתנים	יוצרת את כל טבלאות בסיס הנתונים, אם הן לא קיימות
insert_user	הפונקציה מקבלת שם משתמש, סיסמא ואימייל	מוסיף משתמש חדש עם יתרה התחלתית של 0 מחזיר True אם הצליח, False, אם שם המשתמש כבר תפוס
checkRegistration	הפונקציה מקבלת שם משתמש	מחזיר "ALL_RIGHT" אם שם המשתמש פנוי, או "NOT_ALL_RIGHT" אם תפוס
is_correct_password	הפונקציה מקבלת שם משתמש וסיסמא	מחזיר "LOGIN_SUCCESS" אם הסיסמא נכונה, אחרת "LOGIN_FAILED"
getBalance	הפונקציה מקבלת את שם משתמש	מחזיר את היתרה הנוכחית של המשתמש
isExists	הפונקציה מקבלת את שם המשתמש	מחזיר True אם המשתמש קיים במסד, אחרת False

changeBalance	הפונקציה מקבלת משתמש להעביר ממנו, משתמש להעביר אליו, יתרת המשתמש שמעביר, יתרת המשתמש שמעבירים אליו	מעדכן את היתרה של שני משתמשים
insert_transfer	הפונקציה מקבלת שם משתמש של המעביר, של מי שמעבירים אליו, כמות וסיבה	שומר העברה בטבלת transfers כולל את הסיבה, הסכום והתאריך
intert_requests	הפונקציה מקבלת שם משתמש שמבקש העברה, של מי שמבקשים ממנו, כמות וסיבה	מוסיף בקשת העברה בטבלת requests עם status treated=False
getTransfer	הפונקציה מקבלת שם משתמש	מחזיר את כל ההעברות של המשתמש וגם שהעבירו אליו
get_requests	הפונקציה מקבלת שם משתמש	מחזיר בקשות חדשות שנשלחו למשתמש
change_status	הפונקציה מקבלת מספר זיהוי של בקשה	משנה את הסטטוס של treated
getAmountOfRequests	הפונקציה מקבלת שם משתמש	מחזיר כמה בקשות קיבל המשתמש
getAmountTransfers	הפונקציה מקבלת שם משתמש	מחזיר כמה העברות המשתמש קיבל
getYears	הפונקציה מקבלת שם משתמש	מחזיר את רשימת השנים שבהן המשתמש ביצע העברות
getYearSpending	הפונקציה מקבלת שם משתמש, שנה וקטגוריה	מחזיר רשימת הוצאות חודשיות לפי קטגוריה ושנה
checkAttempts	הפונקציה מקבלת שם משתמש IP	בודק כמה ניסיונות כושלים היו היום למשתמש מ IP מסוים
updateDate	הפונקציה מקבלת שם משתמש IP	אם קיים רשומה מה IP למשתמש- מעלה את מונה הניסיונות או מאפס לתאריך חדש אם לא קיימת- מוסיף רשומה חדשה
checkAmountFailAttempts	הפונקציה מקבלת כתובת IP	מחזיר כמה משתמשים נכשלו 3 פעמים באותו IP באותו יום
checkAllAmountFailAttempts	הפונקציה מקבלת כתובת IP	מחזיר סך כל הניסיונות הכושלים מה IP הזה באותו יום

getEmail	הפונקציה מקבלת שם משתמש	מחזיר את כתובת האימייל של המשתמש
failedSecondLogin	הפונקציה מקבלת שם משתמש ו IP	אם יש רשומה- מגדיר אותה כ- 3 ניסיונות אחרת - מוסיף רשומה חדשה עם 3 ניסיונות
insertCommunication	הפונקציה מקבלת כתובת IP	רושם את החיבור של ה IP עם סטטוס ON, ועם התאריך של היום
changStatusCommunication	הפונקציה מקבלת כתובת IP	משנה סטטוס של IP ל OFF
checkHowManyConected	הפונקציה לא מקבלת משתנים	סופר כמה חיבורים פעילים

חלק ב:

קטעי קוד רלוונטיים:

אלגוריתם התחברות למערכת:

ההתחברות הראשונית:

```
def login(self):
    ip=self.client.getLocal_ip()
    username = self.user_entry.get()
    password = self.user_pass.get()
    passwordHashes = hashlib.sha256(password.encode()).hexdigest()
    if username and password:
        response = self.client.send_request(f"ALL_FAIL_ATTEMPTS;{ip}")
        if(response=="10"):
            tkmb.showinfo("Security error", "You have entered your password
incorrectly at least once for 10 different users. Please try again
tomorrow..")
        else:
            response =
self.client.send_request(f"AMOUNT_OF_FAIL_ATTEMPTS;{ip}")
            if(response=="5"):
                tkmb.showinfo("Security error", "You entered your password
incorrectly three times for five different users. Please try again
tomorrow.")
            else:
                response =
self.client.send_request(f"CHECK_ATTEMPTS;{username};{ip}")
                if(response=="3"):
                    tkmb.showinfo("Security error", "You have entered your
password incorrectly three times. Please try again tomorrow.")
                else:
                    response =
self.client.send_request(f"LOGIN;{username};{passwordHashes}")
                    if response == "LOGIN_SUCCESS":
                        tkmb.showinfo("First Login", "First login was
successful!")
                    self.emailPassword(username)
                    else:
                        tkmb.showerror("Login", "Invalid credentials")
```

```
self.client.send_request(f"UPDATE_ATTEMPTS;{username};{ip}")
else:
    tkmb.showinfo("Login Error", "Please enter both username and password")
```

זוהי הפונקציה המרכזית של פעולת ההתחברות למערכת. היא נמצאת במחלקה של המסך הראשוני בצד הלקוח. היא שולחת מספר הודעות לשרת על מנת לבדוק את התקינות של ההתחברות בהתאם לבסיס הנתונים. היא מציגה הודעת שגיאה/ הצלחת ההתחברות ראשונית בהתאם לתקינות הקלט. בצד של השרת יש שימוש במספר רב של פונקציות על מנת לוודא כי שם המשתמש והסיסמא נכונים וכי כתובת ה IP של המשתמש לא חסומה- לאחר מספר ניסיונות ההתחברות כושלים. במידה והפרטים נכונים המשתמש מועבר למסך ההתחברות השני.

הפונקציות שצד השרת משתמש בהן (חוץ מפונקציית שליחת הודעה) ומה הן עושות:

- checkAllAmountFailAttempts - בודקת את מספר הפעמים הכללי שטעתה כתובת ה IP של המשתמש בסיסמא של משתמש כלשהו.
- checkAmountFailAttempts - בודקת את מספר המשתמשים החסומים (טעו 3 פעמים בסיסמתם ביממה האחרונה) של כתובת ה IP של המשתמש.
- check_Attempts - בודקת את מספר הפעמים שכתובת ה IP טעתה בסיסמא של המשתמש שמנסה להיכנס ביממה האחרונה.
- update_attempts - מעדכנת את כמות הטעויות בהזנת הסיסמא של כתובת ה IP למשתמש מסוים.
- Login - בודקת כי הנתונים שהוזנו על ידי המשתמש הם נתונים נכונים.

ההתחברות השנייה:

בתהליך ההתחברות השנייה יש שימוש בשתי פונקציות: פונקציה שתשלח את האימייל עם קוד האימות ופונקציה שלאחר לחיצה על כפתור הכניסה תאמת את הנתונים.

```
def emailPassword(self,username):
    self.counter=0
    self.user_entry.pack_forget()
    self.user_pass.pack_forget()
    self.loginButton.pack_forget()
    self.signInButton.pack_forget()
    response = self.client.send_request(f"GET_EMAIL;{username}")
    self.username=username

emailObj=EmailSendingFile.GmailEmailSender(self.client.getAPIkey(),self.client.getFromEmail(),response)
self.passwordSecondLogin=emailObj.send_email()
self.labelEmail.pack(pady=12, padx=10)
self.getPassword.pack(pady=12, padx=10)
loginButton = ctk.CTkButton(self.app, text="LOGIN",
command=self.secondLogIn, font=(self.font, 20),corner_radius=40,
fg_color=self.button_color, hover_color=self.hover_color)
loginButton.pack(pady=12, padx=10)
```

פונקציה זו נמצאת במחלקה של העמוד הראשון- כלומר, היא נמצאת בצד הלקוח. במהלך פונקציה זו נשלח קוד חד פעמי לכתובת המייל של המשתמש. כתובת המייל היא כתובת המייל אותה הוא הזין בהרשמה למערכת. בפונקציה זו מקבלים מהשרת את כתובת המייל של המשתמש, שולחים לו מייל, יוצרים את הממשק עצמו ובמידה ונלחץ כפתור הכניסה

נקראת הפונקציה `secondLogIn`. על מנת לשלוח מייל למשתמש קיימת יצירה של אובייקט `GmailEmailSender`.

הפונקציות שצד השרת משתמש בהן (חוץ מפונקציית שליחת הודעה) ומה הן עושות:

- `getEmail` - מחזיר את כתובת האימייל של המשתמש

```
def secondLogIn(self):
    passwordAttempted=self.getPassword.get()
    if self.counter<3:
        if str(self.passwordSecondLogin)==passwordAttempted:
            tkmb.showinfo("Login", "the login was successful!")
            self.on_close()
            ClientGUSecondPage.secondPageGUI(self.client, self.username)
        else:
            tkmb.showinfo("failed Login", "the password is not correct")
            self.counter=self.counter+1
    else:
        if self.counter==3:
            tkmb.showinfo("Try tomorrow",
                          "you have entered the wring password for three
times. please try again tomorrow")
            ip=self.client.getLocal_ip()
            self.client.send_request(f"FAILED_SECOND_LOGIN;{self.username};{ip}")
            self.counter=self.counter+1
        else:
            tkmb.showinfo("Try tomorrow",
                          "you have entered the wrng password for three
times. please try again tomorrow")
```

פונקציה זו נמצאת במחלקה של העמוד הראשון- כלומר, היא נמצאת בצד הלקוח. בפונקציה זו קיימת בדיקה של הקלט מהמשתמש. בנוסף, נבדק האם הוא טעה בסיסמא החד פעמית שלו 3 פעמים ברצף במהלך היום. אם כן הוא חסום. במידה והוא מזין קוד חד פעמי נכון הוא יעבור למסך הראשי של המערכת. אם הוא טועה תוצג לו הודעת שגיאה.

הפונקציות שצד השרת משתמש בהן (חוץ מפונקציית שליחת הודעה) ומה הן עושות:

- `failedSecondLogIn` - מעדכנת את בסיס הנתונים שמשתמש זה חסום מהמערכת ליום הקרוב.

אלגוריתם הרשמה למערכת:

הרשמה ראשונית:

```
def register(self):
    username = self.user_entry.get()
    password = self.user_pass.get()
    email = self.email_entry.get()
    passwordHashes = hashlib.sha256(password.encode()).hexdigest()
    self.usernameR=username
    self.passwordHashesR=passwordHashes
    self.emailR=email
    if self.checkDigit(username) or self.checkChars(username) or
self.checkChars(password) or self.checkChars(email):
        tkmb.showinfo("Length error", "The maximum number of characters for
a username is 8. Also you can not have , or ; in your username, password or
email.")
    return
    if username and password and email:
        response =
```

```

self.client.send_request(f"CHECK_REGISTRATION;{username}")
if response == "ALL_RIGHT":
    tkmb.showinfo("First Register", "The first step in the
registration process was successful!")
    self.second_Register()
elif response == "NOT_ALL_RIGHT":
    tkmb.showerror("Register", "Username already exists")
else:
    tkmb.showerror("Register", "Server error")
else:
    tkmb.showinfo("Register Error", "Please fill all fields")

```

פונקציה זו נמצאת במחלקה של העמוד הראשון- כלומר, היא נמצאת בצד הלקוח. במהלך הפונקציה המשתמש מנסה ליצור חשבון חדש במערכת. קיימות באלגוריתם זה מספר בדיקות- בדיקה ששם המשתמש לא עולה על מספר תווים מסוים, ובדיקה שתווים מסוימים לא קיימים בשם המשתמש, באימייל או בסיסמא. בנוסף, קיימת בדיקה ששם המשתמש פנוי- בדיקה זו נעשית בצד השרת על פי בסיס הנתונים. במידה וההרשמה הראשונית הייתה מוצלחת ונכונה המשתמש מועבר לשלב השני בהרשמה.

הפונקציות שצד הלקוח משתמש בהן (חוץ מפונקציית שליחת הודעה) ומה הן עושות:

- checkDigit- בדיקה ששם המשתמש לא עולה על 8 תווים
- checkChars- בדיקה ששם המשתמש, הסיסמא והאימייל לא מכילים ";", "ו", "

הפונקציות שצד השרת משתמש בהן (חוץ מפונקציית שליחת הודעה) ומה הן עושות:

- checkRegistration- בדיקה שלא קיים שם משתמש בשם זה בבסיס הנתונים.

ההרשמה השנייה:

בתהליך ההרשמה השנייה יש שימוש בשתי פונקציות: פונקציה שתשלח את האימייל עם קוד האימות ופונקציה שלאחר לחיצה על כפתור הכניסה תאמת את הנתונים.

```

def second_Register(self):
    self.user_entry.pack_forget()
    self.user_pass.pack_forget()
    self.loginButton.pack_forget()
    self.sighInButton.pack_forget()
    self.email_entry.pack_forget()
    self.register_button.pack_forget()
    self.back_button.pack_forget()
    self.labelEmail.pack(pady=12, padx=10)
    self.getPassword.pack(pady=12, padx=10)
    self.SecondRegisterButton.pack(pady=12, padx=10)
    self.back_button.pack(pady=12, padx=10)
    emailObj = EmailSendingFile.GmailEmailSender(self.client.getAPIkey(),
self.client.getFromEmail(), self.emailR)
    self.passwordRegester = emailObj.send_email()

```

פונקציה זו נמצאת במחלקה של העמוד הראשון- כלומר, היא נמצאת בצד הלקוח. במהלך פונקציה זו נשלחת הודעת מייל עם קוד אימות חד פעמי לכתובת המייל שהמשתמש הזין בשלב הקודם של ההרשמה. על מנת לשלוח מייל למשתמש קיימת יצירה של אובייקט מסוג

GmailEmailSender. במידה והמשתמש לוחץ על כפתור ההרשמה הפונקציה קוראת לפונקציה second_Register_entery.

```
def second_Register_entery(self):
    passwordAtempted=self.getPassword.get()
    if str(self.passwordRegester) == passwordAtempted:

self.client.send_request(f"REGISTER;{self.usernameR};{self.passwordHashesR};{self.emailR}")
        tkmb.showinfo("Second Register", "The second step in the registration process was successful!")
        self.on_close()
        ClientGUSecondPage.secondPageGUI(self.client, self.usernameR)
    else:
        tkmb.showerror("second register", "The password is not correct. Try again!")
```

פונקציה זו נמצאת במחלקה של העמוד הראשון- כלומר, היא נמצאת בצד הלקוח. פונקציה זו מוודאת שהסיסמא שהוכנסה על ידי המשתמש היא באמת הסיסמא שנשלחה לו במייל. במידה והקוד שהוזן הוא הקוד הנכון, המשתמש מועבר למסך המרכזי של המערכת. בנוסף, נשלחת הודעה לשרת שמכניסה את פרטי המשתמש לבסיס הנתונים.

הפונקציות שצד השרת משתמש בהן (חוץ מפונקציית שליחת הודעה) ומה הן עושות:

- register- מוסיף לבסיס הנתונים את פרטי המשתמש החדש.

אלגוריתם למניעת הצפת משתמשים:

```
def start(self):
    while True:
        try:
            client, address = self.server.accept()
            if self.checkHowManyConected()<4:
                self.clients.append(client)
                self.sendClient(client, "CONNECTION_SUCCESS")
                time.sleep(0.1)
                threading.Thread(target=self.actionSelection,
args=(client,address)).start()
            else:
                self.sendClient(client, "TOO_MANY_USERS")
                time.sleep(0.1)
                client.close()
        except Exception as e:
            print(f"Error accepting connection: {e}")
```

פונקציה זו נמצאת בצד השרת. היא בודקת ומוודאת- לפי בסיס הנתונים, שכמות המשתמשים במערכת לא גדולה מ-4 (ניתן לשנות את המספר). בכל מצב היא שולחת עדכון מוצפן למשתמש. במידה ויש כבר 4 משתמשים פעילים, המשתמש יקבל הודעת שגיאה ויסגור את הממשק.

כאן מתקיימת הבדיקה בצד המשתמש- בתוך פונקציית connect שנקראת עם יצירת האובייקט:

```
if respondDec == "TOO_MANY_USERS":
    tkmb.showerror("Error", "Too many users connected. Please try again later.")
    if self.client:
        self.client.close()
    sys.exit()
```

הפונקציות שצד השרת משתמש בהן (חוץ מפונקציית שליחת הודעה) ומה הן עושות:

- checkHowManyConected - הפונקציה בודקת כמה משתמשים מחוברים לממשק.

אלגוריתם ההצפנה:

בכל אחד מהצדדים שמקבלים את ההודעה: מחלקת client ומחלקת server, יצירתי מערך עם 3 הצפנות שונות. הצפנת ASE, DES, וRSAI. בכל שליחת הודעה נבחרת הצפנה באופן רנדומלי וההודעה מוצפנת בהתאם. בתחילת ההודעה משורשר מספר ההצפנה וכך הצד השני יודע איך לפענח את ההודעה. כל מפתחות ההצפנה נמצאים בקבצים שונים. אחד נמצא בצד השרת ואחד בצד הלקוח.

דוגמא למערך ההצפנות (זהה בשני הצדדים):

```
encrypt1=EncryptionFile.Encryption1(int(lines[0]),int(lines[1]))
encrypt2 = EncryptionFile.Encryption2(lines[2].encode(),
lines[3].encode())
encrypt3 = EncryptionFile.Encryption3(lines[4].encode())
self.encrypt = [encrypt1, encrypt2,encrypt3]
```

לכל אחת מהמחלקות מחלקה מותאמת אישית עם האלגוריתם להצפנה ופענוח שלה, אך כל הצפנה יורשת מהמחלקה Encryptions.

כיצד הודעות מוצפנות ומפוענחות בכל אחד מהצדדים (אותו אלגוריתם בשני הצדדים אך במיקומים שונים ובשמות שונים למשתמשים- כאן הדוגמא היא אצל המשתמש):

מוצפנות:

```
ranNum=self.getRandom()
requestEncrypt=self.encrypt[ranNum].encrypt(request)
requestEncryptToSend=f"{ranNum}"+requestEncrypt
```

יש שימוש בפונקציית getRandom שנמצאת במחלקה (של כל אחד מהצדדים), היא מחזירה את המספר שלפיו ההודעה תוצפן. היא מוסיפה את מספר ההצפנה משורשר לתחילת ההודעה המוצפנת.

מפוענחות:

```
radomNum=int(respond[0])
respondCut= respond[1:]
respondDec=self.encrypt[radomNum].decrypt(respondCut)
```


ראשית מפצלים את מספר ההצפנה מההודעה המוצפנת. לאחר מכן לפי המספר מפענחים את ההודעה.

חלק ג:

מסמך בדיקות מלא:

שם בדיקה	מטרת בדיקה	מה בוצע בפועל	תוצאות הבדיקה
1	תקינות ההתחברות.	ווידוא ששם המשתמש קיים במערכת והסיסמא של המשתמש היא נכונה.	קיימת בדיקה בבסיס הנתונים האם שם המשתמש קיים והאם הסיסמא היא הסיסמא הנכונה.
2	תקינות האימות הדו שלבי- קוד למייל.	אימות דו שלבי שמוודא שהמשתמש שמנסה להתחבר הוא האדם שאליו שייך החשבון.	מתבצעת בדיקה שהקוד שהמשתמש מזין הוא הקוד החד פעמי שנשלח לו למייל
3	בדיקה שמשתמש (מאותו IP) לא הזין 3 פעמים סיסמא אישית לא נכונה.	לודא שאם מאותו IP משתמש הזין 3 פעמים סיסמא לא נכונה, הוא לא יוכל להיכנס לחשבון זה במשך יממה.	כאשר המשתמש מזין את הסיסמא שלו, קיימת בדיקה מהמידע בבסיס הנתונים האם הוא טעה כבר 3 פעמים (עם אותו ה IP) ביממה האחרונה
4	בדיקה שלא 5 משתמשים (מאותו IP) הזינו 3 פעמים סיסמא אישית לא נכונה ביממה האחרונה.	בדיקה האם חמישה משתמשים כבר נחסמו במהלך היום מאותה כתובת IP.	כאשר משתמש מזין שם משתמש וסיסמא הסרסר בודק- לפי המידע מבסיס הנתונים, האם מאותה כתובת IP כבר נחסמו באותו היום 5 משתמשים
5	בדיקה שלא 10 אנשים (מאותו IP) הזינו לפחות פעם אחת סיסמא שגויה ביממה האחרונה.	בדיקה שלא 10 משתמשים מאותה כתובת IP טעו לפחות פעם אחת ביממה האחרונה.	כאשר משתמש מזין שם משתמש וסיסמא הסרבר בודק- לפי בסיס הנתונים, האם מאותה כתובת IP לא טעו בסיסמא של 10 משתמשים

			שונים לפחות פעם אחת במהלך היממה האחרונה.	
6	רישום משתמש חדש: האם השם פנוי.	בדיקה האם כבר קיים שם משתמש זה במאגר.	כאשר משתמש מנסה להירשם למערכת, השרת בודק- לפי בסיס הנתונים שאין שם משתמש כזה כבר במערכת	אם השם תפוס מוצגת הודעה בהתאם. אם השם לא תפוס המשתמש ימשיך לשלב הבא של ההרשמה
7	רישום משתמש חדש: האם האורך של שם המשתמש הוא פחות מ-9 תווים. ושאינו בו ";;", ";;", ";;".	בדיקה ששם המשתמש שהוזן בהרשמה הוא פחות מ-9 תווים, ושאינו את התווים הבאים: ";;", ";;", ";;", ";;".	כאשר המשתמש מנסה להירשם, שתי פונקציות שונות במחלקה של העמוד הראשון בודקות את תקינות הקלט.	אם הקלט לא תקין המשתמש מקבל הודעה בהתאם, אם תקין הוא יכול להמשיך בתהליך ההרשמה.
8	בדיקת תקינות הקלט בהעברה: בדיקה ששם המשתמש קיים.	לבדוק שהמשתמש שמועבר אליו כסף קיים.	כאשר המשתמש מנסה להעביר כסף השרת בודק- לפי בסיס הנתונים, שהמשתמש שאליו רוצים להעביר כסף קיים.	אם המשתמש לא קיים מוצגת הודעה בהתאם ולא מתקיימת העברה. אם הוא קיים מתבצעת העברה.
9	בדיקת תקינות הקלט בהעברה: בדיקה ששם המשתמש שמועבר אליו כסף הוא לא המשתמש עצמו.	לבדוק שהמשתמש לא מנסה להעביר כסף לעצמו.	כאשר המשתמש מנסה להעביר כסף, ישנה בדיקה ששם המשתמש שמועבר אליו הכסף הוא לא שם המשתמש של המשתמש עצמו.	אם המשתמש מנסה להעביר לעצמו מוצגת הודעה בהתאם ולא מתקיימת העברה. אם הוא לא מנסה להעביר לעצמו מתבצעת העברה.
10	בדיקת תקינות הקלט בהעברה: בדיקה שהסכום המועבר הוא תקין.	כאשר משתמש מנסה להעביר כסף, קיימת בדיקה על הקלט של הסכום. בודקים שהסכום חיובי ושהוא מספר ולא תו אחר.	כשהמשתמש מנסה להעביר כסף, בודקים שהקלט הוא מספר חיובי.	אם הקלט של כמות הכסף להעברה הוא לא מספר חיובי, מוצגת הודעה בהתאם ולא מתקיימת העברה.
11	בדיקת תקינות הקלט בהעברה: בדיקה שנשאר כסף ביתרה.	לבדוק שלאחר ההעברה תישאר יתרה חיובית.	כאשר משתמש מנסה להעביר כסף, מתבצעת בדיקה- לפי המידע שהשרת מעביר- לפי בסיס הנתונים, שהסכום שהמשתמש מנסה להעביר קטן	במידה והסכום שהמשתמש מנסה להעביר גבוה מהסכום שיש לו ביתרה, תוצג הודעת שגיאה וההעברה לא תתבצע.

		מכמות הכסף שיש לו ביתרה		
12	בדיקת תקינות הקלט בבקשת העברה: בדיקה ששם המשתמש קיים.	לבדוק שהמשתמש שמבוקש ממנו כסף קיים.	כאשר המשתמש מנסה לבקש כסף השרת בודק- לפי בסיס הנתונים, שהמשתמש שממנו רוצים לבקש כסף קיים.	אם המשתמש לא קיים מוצגת הודעה בהתאם ולא מתקיימת בקשת העברה. אם הוא קיים מתבצעת בקשת העברה.
13	בדיקת תקינות הקלט בבקשת העברה: בדיקה ששם המשתמש שמבוקש ממנו כסף הוא לא המשתמש עצמו.	לבדוק שהמשתמש לא מנסה לבקש כסף מעצמו.	כאשר המשתמש מנסה לבקש כסף, ישנה בדיקה ששם המשתמש שמבוקש ממנו הכסף הוא לא שם המשתמש של המשתמש עצמו.	אם המשתמש מנסה לבקש מעצמו כסף מוצגת הודעה בהתאם ולא מתקיימת בקשת העברה. אם הוא לא מנסה לבקש מעצמו מתבצעת בקשת העברה.
14	בדיקת תקינות הקלט בבקשת העברה: בדיקה שהסכום המועבר הוא תקין.	כאשר משתמש מנסה לבקש, קיימת בדיקה על הקלט של הסכום. בודקים שהסכום חיובי ושהוא מספר ולא תו אחר.	כשהמשתמש מנסה לבקש כסף ממשתמש אחר, בודקים שהקלט הוא מספר חיובי.	אם הקלט של כמות הכסף בבקשת העברה הוא לא מספר חיובי, מוצגת הודעה בהתאם ולא מתקיימת בקשת העברה.
15	בדיקת תקינות הקלט באישור העברה: בדיקה שנשאר כסף ביתרה.	בדיקה שלאחר ההעברה תישאר יתרה חיובית.	כאשר המשתמש מנסה לאשר העברה למשתמש אחר, השרת- לפי בסיס הנתונים, בודק האם לאחר העברה היתרה שתישאר למשתמש היא חיובית.	אם לאחר אישור ההעברה היתרה של המשתמש לא תהיה חיובית, תוצג הודעת שגיאה ואישור ההעברה לא יתקיים.
16	חסימת משתמשים רבים בו זמנית.	לזוודא שלא יותר ממספר מסוים של משתמשים מחוברים בו זמנית כדי למנוע עומס משתמשים.	בכל יצירת תקשורת השרת בודק את מספר המשתמשים המחוברים אליו.	במידה וכמות המשתמשים המקסימלית מחוברים, ממשק המשתמש לא יפתח ותישלח הודעת שגיאה

בדיקות נוספות שביצעתי למערכת:

שם בדיקה	מטרת בדיקה	מה בוצע בפועל	תוצאות הבדיקה
1	בדיקה שכל הקבצים הדרושים להפעלת צד המשתמש קיימים	לוודא שיש אפשרות להריץ את הממשק- בלי כל הקבצים אין אפשרות	במידה ולא יפתח ממשק המשתמש ותתקבל הודעת שגיאה.
2	בדיקה שכל הקבצים הדרושים להפעלת צד השרת קיימים	לוודא שיש אפשרות להריץ את הממשק- בלי כל הקבצים אין אפשרות	במידה ולא תתקבל הודעת שגיאה והתוכנית תיפסק.
3	בדיקה חיבור הלקוח לשרת	לבדוק האם הלקוח מצליח להתחבר לשרת, ולוודא שלא הייתה התנתקות לאחר ההתחברות	אם לא קיים חיבור או התרחשה שגיאה בתקשורת, מוצגת הודעת שגיאה, והתוכנית תיסגר.

פרק 5- מדריך למשתמש:

פירוט של כל קבצי המערכת:

ClientGUIFirstPage.py - הקוד למסך הראשוני של המשתמש (התחברות והרשמה) - יש להריץ רק אותו בצד הלקוח (יש צורך בקובץ זה רק בצד הלקוח).

ClientGUSecondPage.py - הקוד למסך העיקרי של המשתמש - כל הפעולות מתבצעות פה (יש צורך בקובץ זה רק בצד הלקוח).

ClientFile.py - הקוד שאחראי על התקשורת עם השרת (יש צורך בקובץ זה רק בצד הלקוח).

EmailSendingFile.py - מכיל את המחלקה שמאפשרת שליחת מיילים למשתמשים (יש צורך בקובץ זה רק בצד הלקוח).

ClientFileData.txt - מכיל את כל המידע הרגיש של התוכנית בצד הלקוח (יש צורך בקובץ זה רק בצד הלקוח).

EncryptionFile.py - מכיל את המחלקות שמצפינות את המידע שעובר בין הצדדים השונים (יש צורך בקובץ זה גם בצד השרת וגם בצד הלקוח).

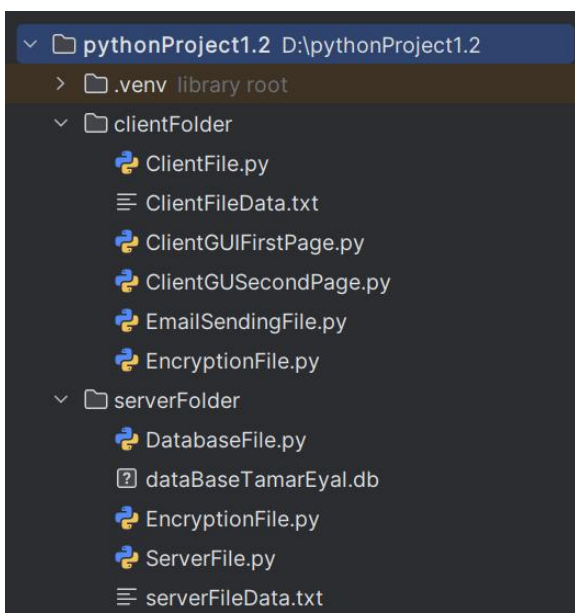
serverFileData.txt - מכיל את כל המידע הרגיש של התוכנית בצד השרת (יש צורך בקובץ זה רק בצד השרת).

DatabaseFile.py - מכיל את כל הפעולות שנעשות על בסיס הנתונים (יש צורך בקובץ זה רק בצד השרת).

dataBaseTamarEyal.db - זהו בסיס הנתונים המכיל את כל המידע של משתמשי הממשק (יש צורך בקובץ זה רק בצד השרת).

ServerFile.py - קוד השרת - יש להריץ אותו רק בצד השרת (יש צורך בקובץ זה רק בצד השרת).

עץ הקבצים:



על מנת להריץ את התוכנית בצד הלקוח יש להריץ את הקובץ: **ClientGUIFirstPage.py**, ובצד השרת יש צורך להריץ את הקובץ: **ServerFile.py**.

התקנת המערכת:

סביבת העבודה:

על מנת להריץ את התוכנית נדרש להתקין Python בגרסה 3.11.4 ומעלה (זו הגרסה בה הוא נבדק). בנוסף, יש צורך להוריד את כל המודולים הנלווים.

ניתן להריץ את קבצי השרת והלקוח ישירות על ידי לחיצה כפולה על הקבצים:

- ClientGUIFirstPage.py - בצד הלקוח
- ServerFile.py - בצד השרת

פירוט הכלים הנדרשים:

על מנת לעבוד על הקוד או להריץ אותו, מומלץ להשתמש ב-PYCHARM. על מנת לקרוא את המידע בבסיס הנתונים, מומלץ להשתמש ב-SQLite.

רשת:

עבודה על מחשב בודד (לבדיקה ולפיתוח)

ניתן להריץ את השרת והלקוחות על אותו מחשב באמצעות מספר חלונות טרמינל שונים. אך גם במצב זה יש צורך בחיבור לאינטרנט- מאחר ומתבצעת שליחת הודעה למייל.

עבודה על מספר מחשבים (לשימוש מלא)

כל המחשבים (השרת והלקוחות) חייבים להיות מחוברים לאותה רשת מקומית. המחשבים צריכים להיות מחוברים לאותו נתב- זה מפשט את ההגדרה ומבטל צורך בפתיחת פורטים חיצוניים.

בקובץ ClientFile.py יש לעדכן את כתובת ה-IP של השרת.

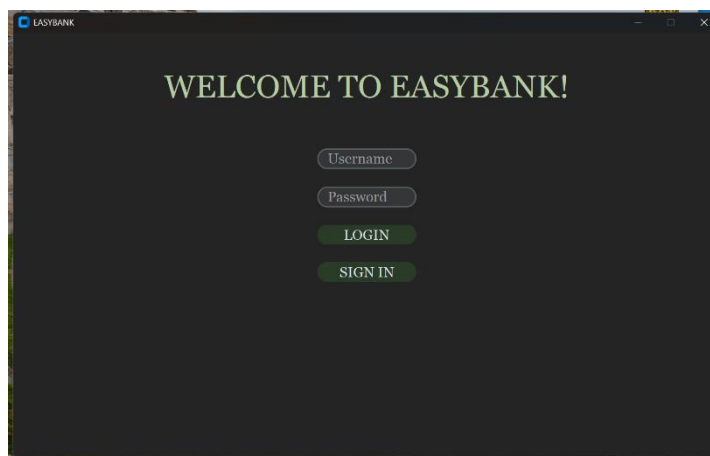
אם משתמשים ב-Windows ויש בעיה בחיבור:
יש לאפשר ל-python.exe לעבור דרך ה-Windows Defender Firewall.
לחלופין, יש לפתוח את פורט 1729 ידנית.

משתמשי המערכת:

המערכת שלי יש סוג אחד של משתמשים- משתמשי קצה.

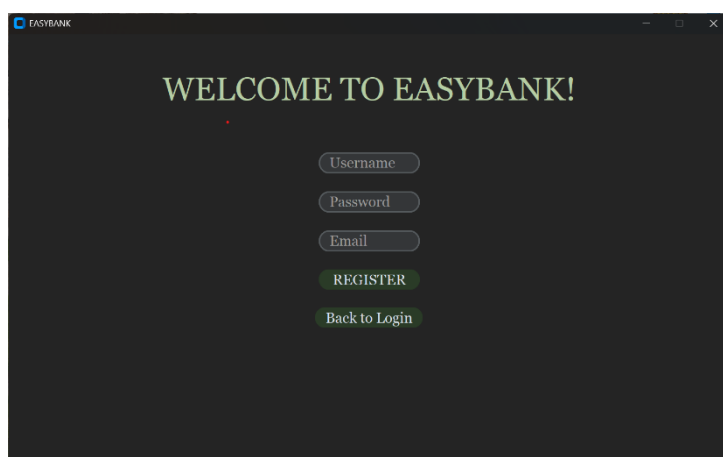
אופן הפעלת המערכת:

בהפעלת הממשק, המסך הראשוני שמוצג הוא מסך התחברות המשתמש:



על המשתמש להזין את שם המשתמש והסיסמא שלו. במידה ואין משתמש קיים, ניתן להירשם על ידי לחיצה על כפתור "SIGN IN". לאחר לחיצה על כפתור "LOGIN" קיימת בדיקה שהמשתמש הזין פרטים נכונים- במידה וכן הוא מועבר למסך האימות הדו שלבי.

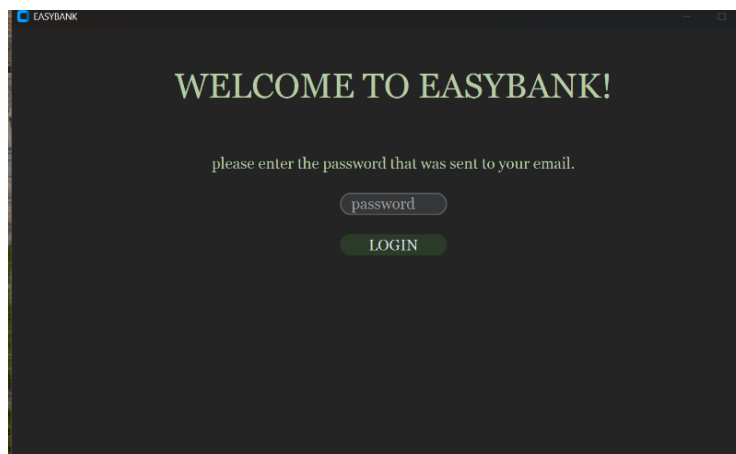
לאחר לחיצה על הכפתור "SIGN IN" יפתח מסך הרשמה:



במסך זה על המשתמש להזין את הפרטים של המשתמש החדש. קיימת בדיקה ששם המשתמש לא תפוס כבר ושהוא לא ארוך מ-8 תווים. בנוסף, קיימת בדיקה שלא קיימים התווים ";", "ו", " בכל אחד מהשדות.

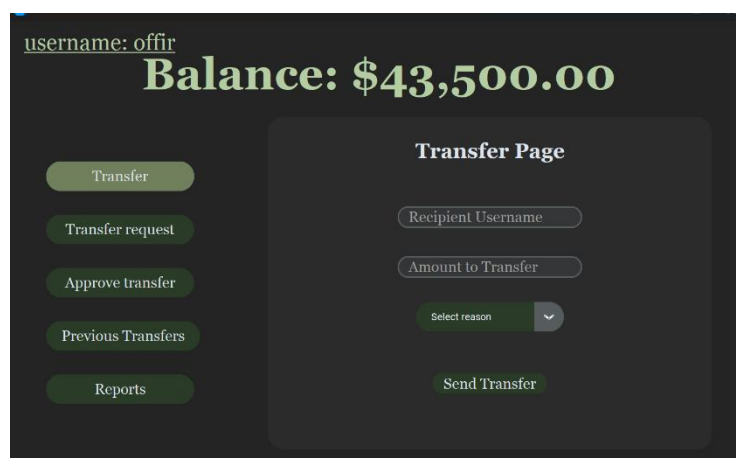
בלחיצה על כפתור "Back to login" המשתמש יחזור למסך ההתחברות.

במידה וכל הנתונים שהוזנו תקינים (גם במסך ההתחברות וגם במסך ההרשמה) המשתמש יעבור למסך אימות דו שלבי:



במסך זה על המשתמש להזין את הקוד החד פעמי שנשלח לו למייל. אם המסך נפתח לאחר מסך ההרשמה במידה והמשתמש לא יזין קוד נכון לא תתבצע הרשמה. במידה והמסך מגיע לאחר מסך התחברות קיימת בקרה על כמות הניסיונות הכושלים של ההתחברות שלו.

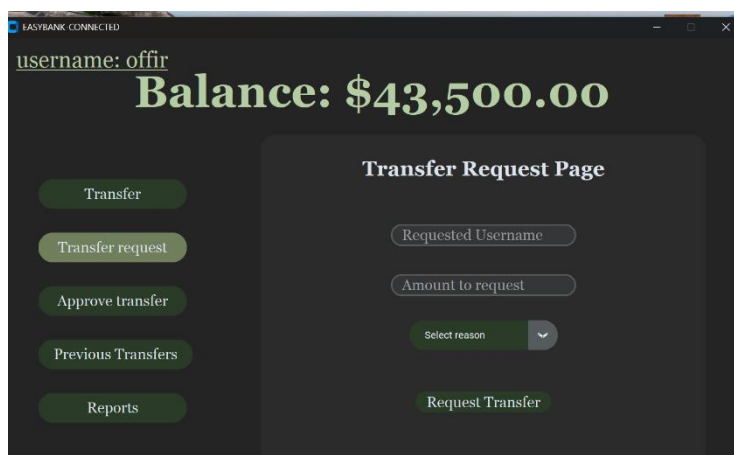
במידה והמשתמש מזין קוד נכון הוא מועבר למסך המרכזי של הפרויקט (למסך ההעברות):



מגיעים למסך זה גם אוטומטית לאחר הכניסה לחשבון וגם על ידי לחיצה על כפתור "Transfer". במסך ההעברה המשתמש מזין את שם המשתמש אלו הוא רוצה להעביר את הכסף, את הכמות ואת סיבת העברה. קיימות מספר בדיקות לוודא שהקלט תקין ובמידה וכן מתבצעת העברה.

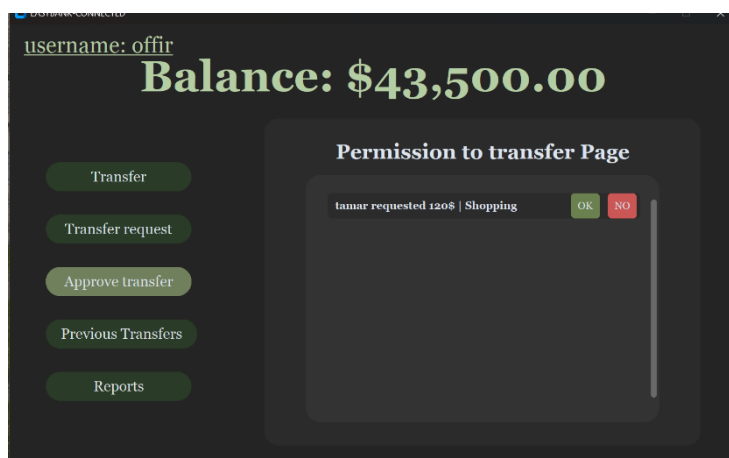
ממסך זה ניתן להגיע למסכי הפעולות של כל הפעולות המתאפשרות למשתמש- על ידי לחיצה על הכפתורים בצד שמאל.

במידה ונלחץ הכפתור "Transfer request" המשתמש מועבר למסך בקשת העברה:



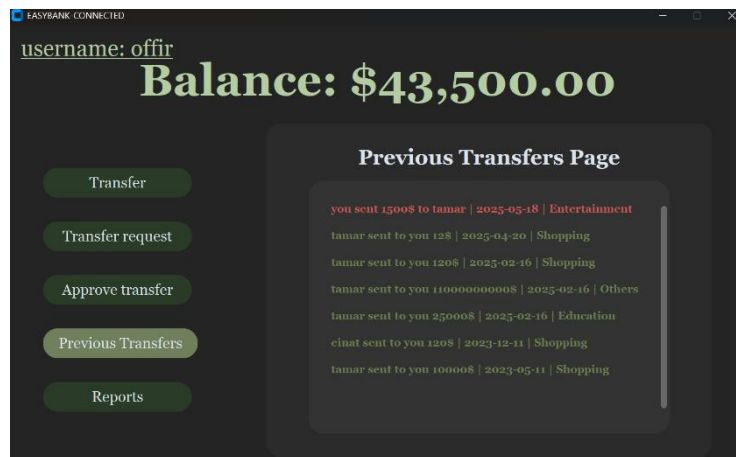
במסך זה המשתמש מזין את שם המשתמש של המשתמש אלו הוא רוצה להעביר את הכסף, כמות הכסף וסיבת העברה. קיימות מספר בדיקות לוודא שהקלט תקין ובמידה וכן מתבצעת בקשת ההעברה.

במידה ונלחץ הכפתור "Approve transfer" המשתמש מועבר למסך אישור ההעברות:



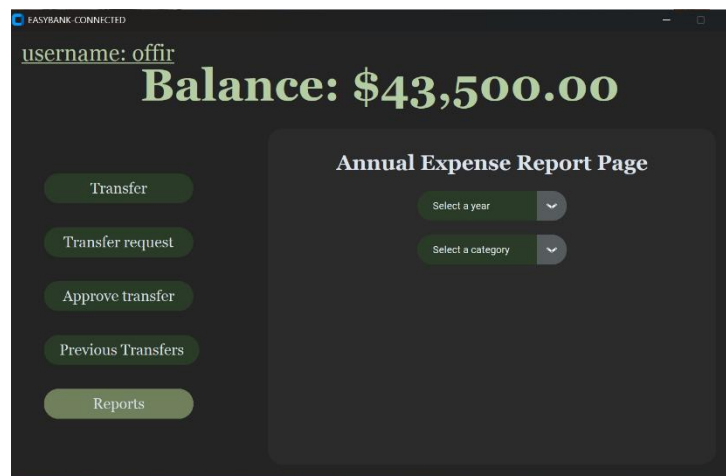
במסך זה המשתמש מאשר או דוחה את בקשות ההעברה של משתמשים אחרים ממנו. כאשר הוא לוחץ על כפתור "OK" קיימת בדיקה שיש למשתמש מספיק כסף ביתרה לביצוע ההעברה.

כאשר נלחץ הכפתור "Previous Transfers" המשתמש מועבר למסך שמציג לו את כל ההעברות הקודמות שהוא ביצע ושהעבירו אליו:



במסך זה מוצג למשתמש כל ההעברות הקודמות שנעשו אליו או שהוא ביצע. מפורט תאריך ההעברה, סכום ההעברה, שם המעביר וסיבת ההעברה. העברות שמוצגות בצבע ירוק הן העברות שבוצעו אל המשתמש, העברות שמוצגות בצבע אדום הן העברות שהמשתמש ביצע.

כאשר נלחץ הכפתור "Reports" המשתמש מועבר למסך דוחות שנתיים:

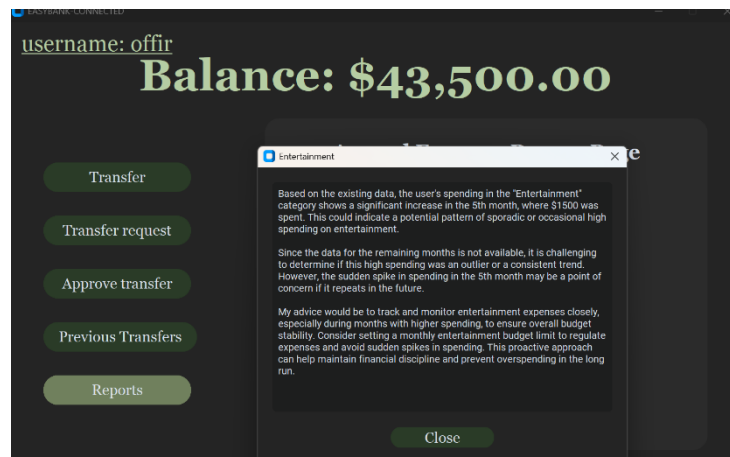


במסך זה המשתמש מזין את שנת הפעילות (מוצגות רק שנים שבמהלכן התבצעו העברות) ואת הקטגוריה שהוא רוצה לראות את דוחות הפעילות שלו. קיימת בדיקה שכל השדות מוזנים.

דוח לדוגמא:



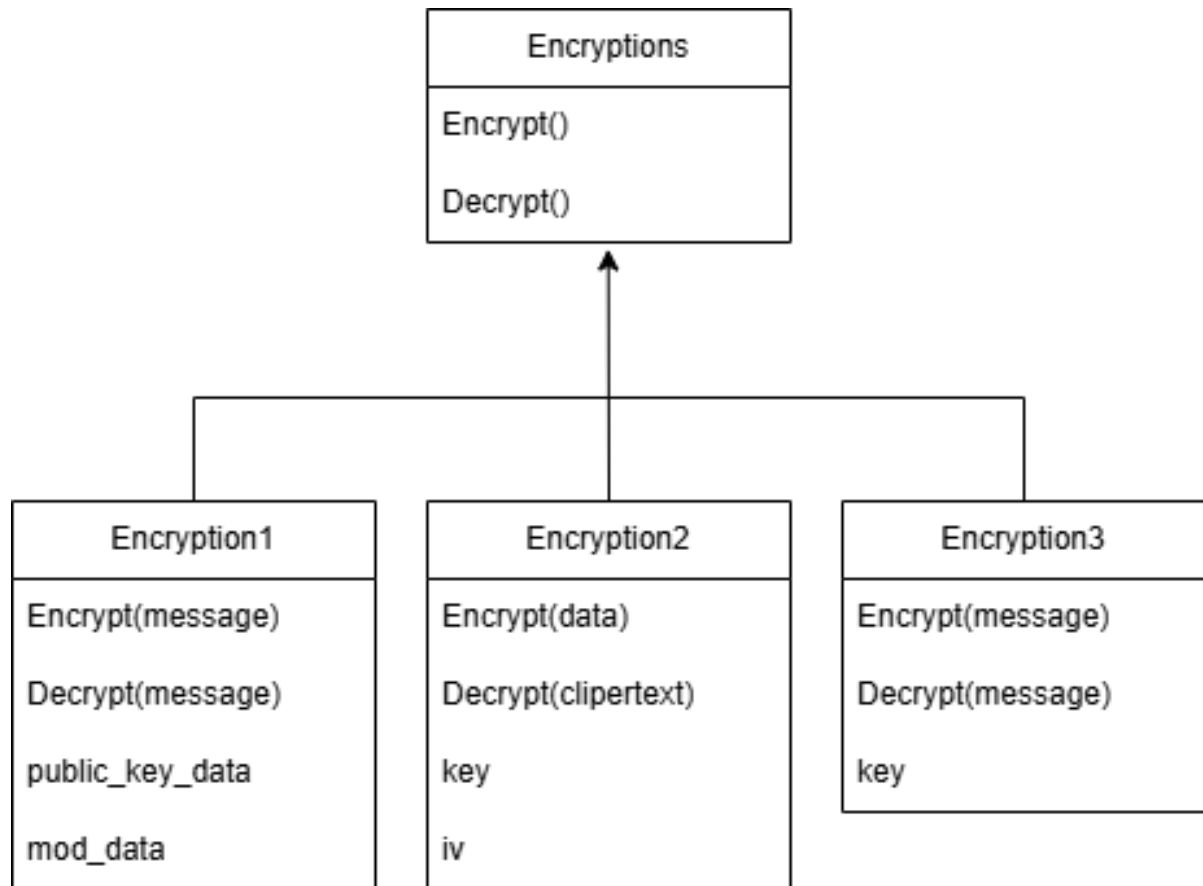
לאחר הצגת הגרף, ניתנת אפשרות לקבלת יעוץ מבינה מלאכותית (בלחיצה על כפתור "Get an advice"):



בחלונית זו מוצג ייעוץ מבינה מלאכותית לגבי ההוצאות השנתיות של המשתמש בקטגוריה מסוימת.

הרחבות נוספות:

תרשים מחלקות המתאר קשרי ירושה:



האלגוריתמיקה והמתמטיקה של ההצפנות:

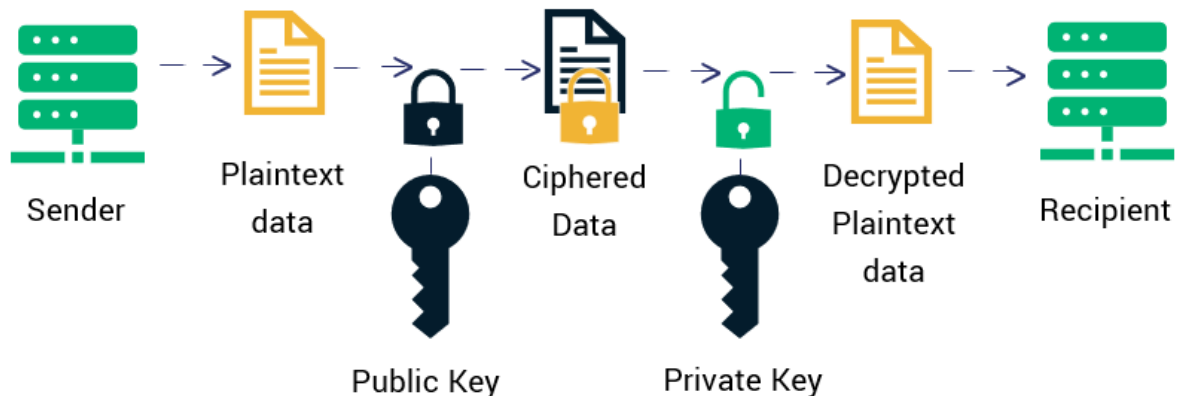
:RSA

RSA הוא אלגוריתם להצפנה אסימטרית. הוא מבוסס על עקרונות מתמטיים של מכפלות של מספרים ראשוניים וחשבון מודולרי.

1. צריך לבחור שני מספרים ראשוניים שונים. מספרים אלה ישמשו לבניית המפתחות. ככל שהמספרים גדולים יותר, כך ההצפנה בטוחה יותר.
2. מחשבים את המודולו. מכילים את שני המספרים הראשוניים ומסמנים את התוצאה כ- N . מספר זה משמש כבסיס גם להצפנה וגם לפענוח.
3. חישוב פונקציית אוילר של N - מחשבים את פונקציית אוילר של $\phi(N)$, שהיא מספר המספרים הקטנים מ- N שזרים לו (כלומר אין להם מחלקים משותפים איתו). N הוא מכלפת שני המספרים הראשוניים. הפונקציה היא: $\phi(N) = (p - 1) \times (q - 1)$.
4. נבחר מפתח ציבורי. נבחר מספר טבעי e המקיים את התנאים: גדול מ-1 וקטן מ- $\phi(N)$, אין לו מחלקים משותפים עם $\phi(N)$, מספר זה ישמש כמפתח ההצפנה הציבורי.
5. יש למצוא מספר d כך שמתקיים $e \times d \equiv 1 \pmod{\phi(N)}$. מספר זה יהווה את המפתח הפרטי, המשמש לפענוח.

6. כדי להצפין הודעה, ממירים את ההודעה (למשל אות או מספר) למספר מתאים, ומבצעים:
 $C = M^e \bmod N$. כאשר M הוא ההודעה המקורית (כמספר) C , היא ההודעה המוצפנת.
7. על מנת להצפין את ההודעה, משתמשים במפתח הפרטי: $M = C^d \bmod N$. התוצאה היא ההודעה המקורית.

How RSA Encryption Works



- AES

(אתר האינטרנט eitca, "מהן הפעולות העיקריות המבוצעות בכל סבב של אלגוריתם AES וכיצד הן תורמות לאבטחה הכוללת של תהליך ההצפנה?", 2023)

AES הוא אלגוריתם הצפנה סימטרי. האלגוריתם פועל על גודל בלוק קבוע של 128 סיביות ומשתמש בגודל מפתח של 128, 192 או 256 סיביות. מספר הסיבובים נקבע בהתאם לגודל המפתח.

בכל אחד מהסבבים (חוץ מהאחרון) נקראות 4 פעולות:

SubBytes- בפעולה זו, כל בייט של בלוק הקלט מוחלף בבייט מתאים מטבלת החיפוש של AES S-Box. זוהי טבלת החלפה המספקת מיפוי לא ליניארי, ומציגה בלבול בתהליך ההצפנה. שלב ההחלפה הזה עוזר למנוע קשרים אלגבריים פשוטים בין טקסט פשוט לטקסט צופן, מה שמקשה על התוקף להסיק את הנתונים המקוריים.

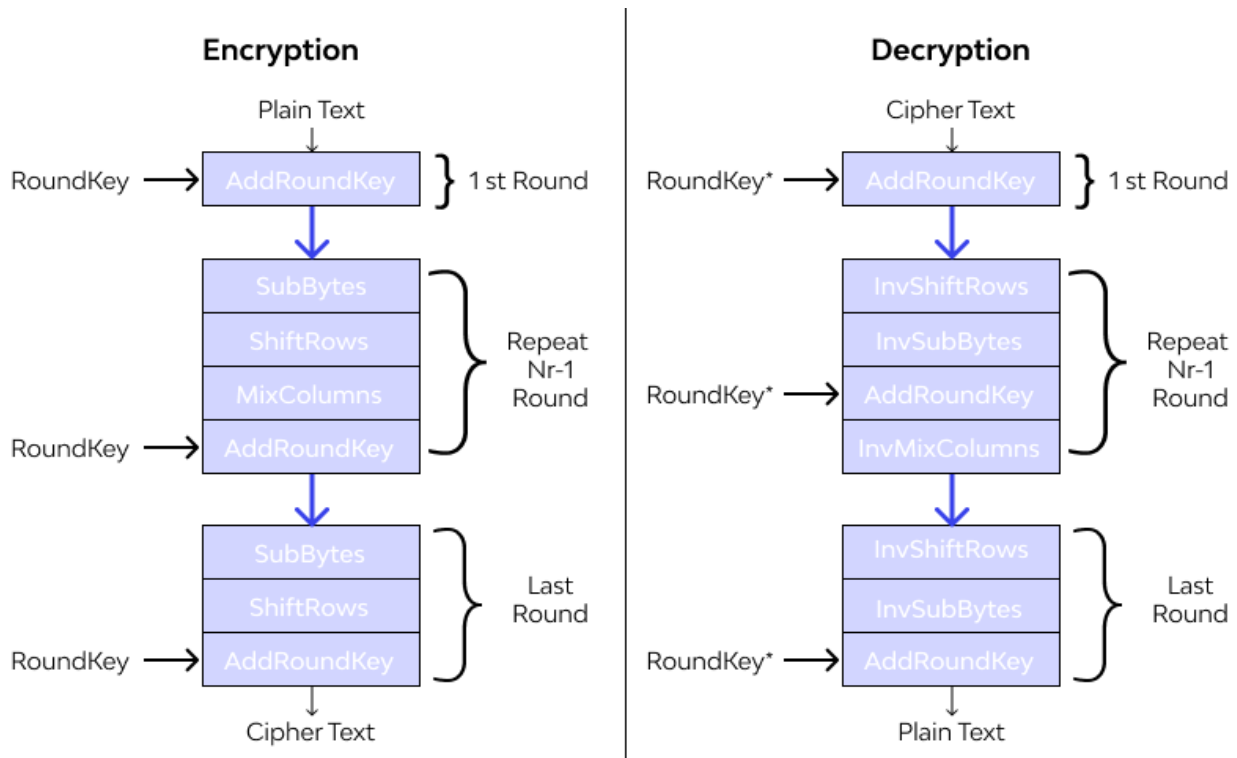
ShiftRows- בפעולה זו, הבתים של כל שורה במערך המצבים מוזזים שמאלה באופן מחזורי. השורה הראשונה נשארת ללא שינוי, השורה השנייה מוזזת בעמדה אחת שמאלה, השורה השלישית בשתי עמדות, והשורה הרביעית בשלוש עמדות.

MixColumns- בפעולה זו, כל עמודה של מערך המצב עוברת טרנספורמציה באמצעות כפל מטריצה. זה מגביר את המורכבות של תהליך ההצפנה ומחזק את ההתנגדות נגד התקפות ליניאריות.

AddRoundKey- בפעולה זו, מפתח עגול משולב עם מערך המצב על ידי ביצוע פעולת XOR בצורה סיבית. המפתח העגול נגזר ממפתח ההצפנה המקורי באמצעות אלגוריתם לוח זמנים של מפתחות. שלב זה מכניס את המפתח לתהליך ההצפנה, מה שהופך כל סבב לתלוי בחלק אחר של המפתח. זה מבטיח שאפילו שינוי קטן במפתח יביא לטקסט צופן שונה לחלוטין.

פעולות אלו חוזרות על עצמן עבור מספר מסוים של סיבובים, כאשר מספר הסיבובים תלוי בגודל המפתח. בסבב האחרון מתבצעות רק 3 פעולות- MixColumns לא מתבצעת.

כדי לפענח טקסט מוצפן באלגוריתם AES מבצעים את אותן הפעולות כמו בהצפנה, אך בסדר הפוך ובאמצעות גרסאות הפוכות של הפונקציות. מכיוון שזה אלגוריתם סימטרי, נדרש אותו מפתח ששימש להצפנה לפענוח.



-DES

DES הוא אלגוריתם ההצפנה סימטרי. הוא עובד על בלוקים בגודל 64 ביט. נדרש מפתח באורך 64 ביט אך רק 56 ביט משתמשים להצפנה. ההצפנה מתבצעת ב16 סבבים לפי מבנה Feistel.

- Initial Permutation - מבוצעת על הביטים באמצעות טבלת IP קבועה בגודל 8X8. זה לא משפיע על הביטים עצמם אלא רק על המיקום שלהם.
- חלוקה לחצאים- הטקסט מחולק לשני חצאים של 32 ביט. ימני ושמאלי.
- תהליך יצירת מפתחות סבב- המפתח עובר דרך (PC-1) Permuted Choice – מוסר כל 28 ביט. נוצר מפתח של 56 ביט והוא מחולק לשניים- כל אחד 28 ביט.
- כל סבב כולל הזזה סיבובית שמאלה של חלקי המפתח- סבבים 1,2,9,16 הזזה בביט 1 בשאר הסבבים ב2 ביטים. לאחר ההזזה Ci Di, מחוברים ועוברים Permuted Choice 2 (PC-2). מופק מפתח סבב באורך 48 ביט.

מבנה סבב Feistel (16 פעמים):

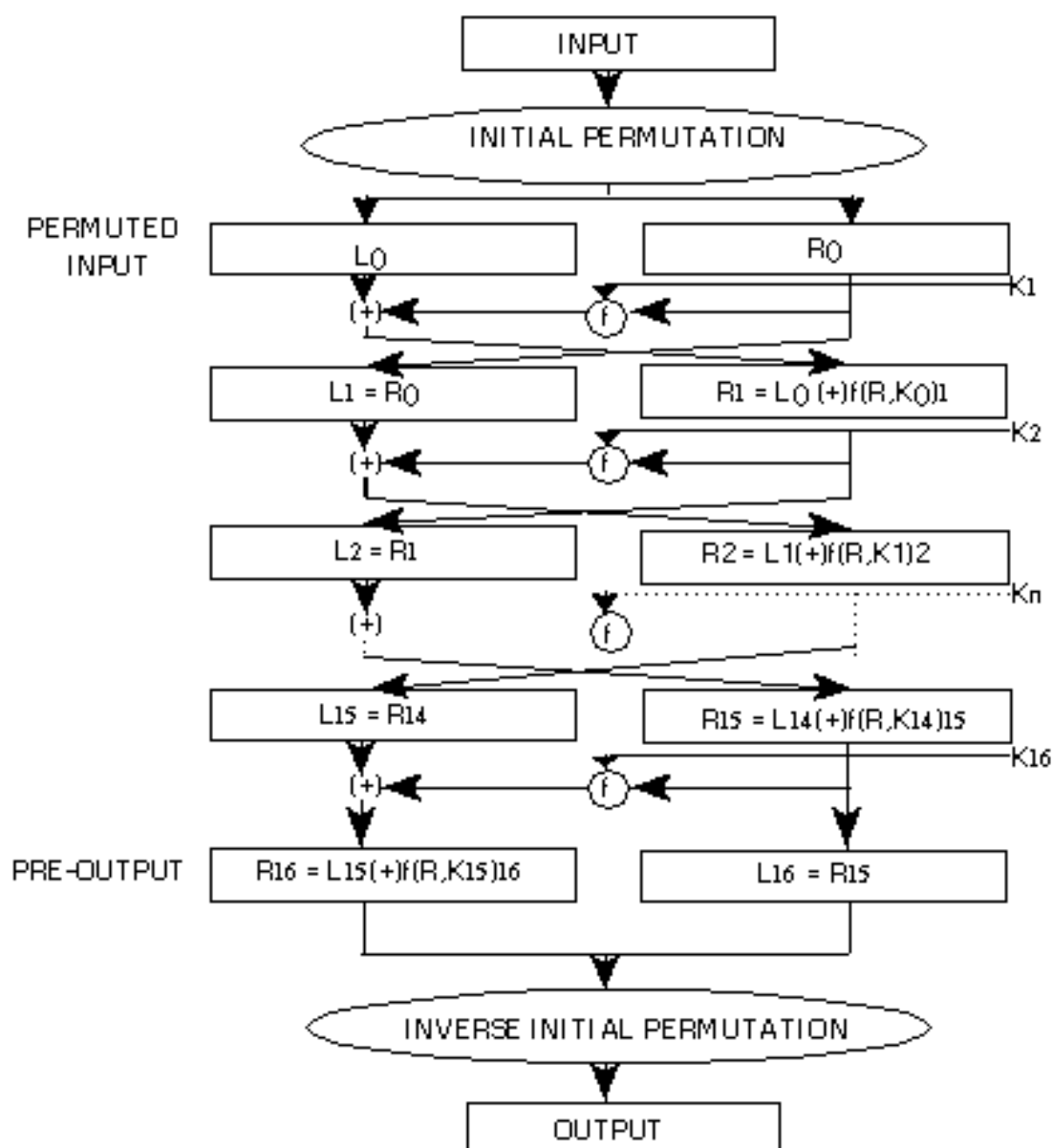
1. הרחבת RPT ל 48 ביט.
2. XOR בין RPT המורחב לבין מפתח הסבב.
3. S-box Substitution - כל 6 ביט נכנסים ל- S-box ומומר ל 4 ביט.
4. P-box Permutation - ממקם מחדש את 32 הביטים המתקבלים.
5. XOR עם LPT - תוצאה היא RPT הבא הRPT הנוכחי הופך ל LPT הבא.

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

לאחר 16 סבבים מחברים את RPT האחרון עם LPT האחרון, מבצעים Inverse Initial Permutation (IP⁻¹) ומתקבל התוכן מוצפן.

על מנת לפענח משתמשים באותו אלגוריתם בדיוק אך מפתחות הסבב משמשים בסדר הפוך.



פרק 6- רפלקציה:

העבודה על הפרויקט ליוותה אותי לאורך רוב כיתה יב. זה היה תהליך קשה, מאתגר, מהנה, מעיף, מלמד, משמעותי ובעיקר מעניין. בתחילת הפרויקט לא היה לי רעיון לנושא לפרויקט, אך ככל שעבר הזמן התגבשו במוחי רעיונות שבסופו של דבר הפכו להיות הפרויקט הסופי שלי. דבר שעזר לי מאוד בבחירת הנושא לפרויקט היה צפייה בפרויקטים משנים קודמות ולקייחת השראה מהם. כל אחד מהם עזר לי להבין מהם החלקים שאני רוצה להתמקד ולהתעמק בהם.

במהלך העבודה התמקדתי כל פעם בחלק מסוים. רק כשסיימתי אותו עברתי לחלק הבא. דבר זה גרם לכך שכשעברתי לחלק חדש בפרויקט שאר הפרויקט עבד.

דבר ספציפי שנהניתי ממנו מאוד היה עיצוב המסכים. זה היה כמו פאזל- לאט לאט הכל מתחבר ופתאום ניתן היה לראות תוצר על כל העבודה הקשה שלי מאחורי הקלעים.

במהלך הפרויקט התמקדתי במיוחד על למידת מספר הצפנות. מאחר ובמהלך התקשורת בין הלקוח והשרת מועבר מידע רגיש על משתמשים רציתי ללמוד עוד סוגי הצפנה על מנת להגן על המידע בצורה מיטבית. למדתי בעצמי את הצפנת AES והצפנת DES. בנוסף, למדתי איך להשתמש בבינה מלאכותית בתוך הקוד שלי- דבר שהלהיב אותי מאוד.

במהלך הפרויקט למדתי המון דברים ולקחתי המון כלים להמשך החיים שלי. הדבר העיקרי שאני אקח להמשך הוא- לא לפחד ממטלות גדולות. גם אם משהו נראה מאיים בהתחלה תמיד אפשר לפרק אותו למשימות קטנות יותר ולהפוך אותו לאפשרי.

בראייה לאחר הייתי מתכננת מראש יותר טוב את הפרויקט. הייתי משקיעה יותר זמן בתכנון ומפרטת לעצמי אילו פעולות אני רוצה לאפשר וכיצד לעשות זאת. זה היה גורם לקוד שלי להיות יותר יעיל ומסודר ולא הייתי צריכה לשנות דברים לאחר שסיימתי אותם.

אם הייתי יכולה להתחיל את הפרויקט עכשיו אני בטוחה שהייתי עובדת בצורה הרבה יותר יעילה, מהירה ומסודרת.

השאלה העיקרית ששאלתי את עצמי במהלך התוכנית היא:

- איך אפשר להפוך את הממשק להכי בטוח שאפשר

אני רוצה להודות למורי המגמה שלי, שליוו אותי לאורך כל הדרך- גם ברגעים הקלים ובעיקר ברגעים הקשים. הם תמיד היו שם לצידי ועשו כל מה שהם יכולים כדי לעזור- גם מורים שלא מלמדים את כיתתי. בנוסף, אני רוצה להודות לחבריי מהמגמה, שגרמו לשיעורים להיות הכי כיפים שאפשר. ובעיקר אני רוצה להודות להוריי על כך שתמיד היו שם בשבילי, תמיד ניסו לעזור בכל דרך שהם רק יכולים ותמיד האמינו בי.

תודה רבה לכולם!

פרק 7- נספחים:

ClientGUIFirstPage.py:

```
import customtkinter as ctk
import tkinter.messagebox as tkmb
import ClientGUSecondPage
import hashlib
import EmailSendingFile
from clientFolder import ClientFile

class ClientGUIFirstPage:
    def __init__(self):
        self.client = ClientFile.Client()
        self.app = ctk.CTk()
        self.app.title("EASYBANK")
        self.signin_mode = False
        self.app.protocol("WM_DELETE_WINDOW", self.on_close)
        self.creat()

        self.app.mainloop()

    def creat(self):
        window_width, window_height = 1000, 600
        screen_width, screen_height = self.app.winfo_screenwidth(),
self.app.winfo_screenheight()
        x_position, y_position = (screen_width - window_width) // 2,
(screen_height - window_height) // 2 - 50

self.app.geometry(f"{window_width}x{window_height}+{x_position}+{y_po
sition}")
        self.app.resizable(False, False)

        self.font = 'Georgia'
        self.hover_color = "#70805D"
        self.button_color = "#2A3B27"

        ctk.set_appearance_mode("dark")

        self.create_widgets()

    def create_widgets(self):
        self.label = ctk.CTkLabel(self.app, text="WELCOME TO
EASYBANK!", font=(self.font, 45), text_color="#B5CDA3")
        self.label.pack(pady=50)

        self.user_entry = ctk.CTkEntry(self.app,
placeholder_text="Username", font=(self.font, 20), corner_radius=40)
        self.user_entry.pack(pady=12, padx=10)
        self.user_pass = ctk.CTkEntry(self.app,
placeholder_text="Password", show="*", font=(self.font,
20), corner_radius=40)
        self.user_pass.pack(pady=12, padx=10)
        self.loginButton = ctk.CTkButton(self.app, text="LOGIN",
command=self.login, font=(self.font, 20), corner_radius=40,
fg_color=self.button_color, hover_color=self.hover_color)
        self.loginButton.pack(pady=12, padx=10)
        self.sighInButton = ctk.CTkButton(self.app, text="SIGN IN",
command=self.switch_to_signin, font=(self.font, 20),
```

```

corner_radius=40,fg_color=self.button_color,hover_color=self.hover_color)

        self.sighInButton.pack(pady=12, padx=10)
        self.register_button = ctk.CTkButton(self.app,
text="REGISTER", command=self.register, font=(self.font, 20),
corner_radius=40,fg_color=self.button_color,hover_color=self.hover_color)

        self.back_button = ctk.CTkButton(self.app, text="Back to
Login", command=self.switch_to_login, font=(self.font, 20),
corner_radius=40,fg_color=self.button_color,hover_color=self.hover_color)

        self.getPassword = ctk.CTkEntry(self.app,
placeholder_text="password", font=(self.font, 20), corner_radius=40)
        self.labelEmail = ctk.CTkLabel(self.app, text="please enter
the password that was sent to your email.",
font=(self.font, 20),
text_color="#B5CDA3")
        self.SecondRegisterButton = ctk.CTkButton(self.app,
text="Register", command=self.second_Register_entry,
font=(self.font, 20),corner_radius=40, fg_color=self.button_color,
hover_color=self.hover_color)

    def on_close(self):
        try:
            for after_id in self.app.tk.call("after", "info"):
                self.app.after_cancel(after_id)
        except Exception as e:
            print(f"Error canceling after events: {e}")

        if self.app.winfo_exists():
            self.app.quit()
            self.app.update_idletasks()
            self.app.destroy()

    def login(self):
        ip=self.client.getLocal_ip()
        username = self.user_entry.get()
        password = self.user_pass.get()
        passwordHashes =
hashlib.sha256(password.encode()).hexdigest()
        if username and password:
            response =
self.client.send_request(f"ALL_FAIL_ATTEMPTS;{ip}")
            if(response=="10"):
                tkmb.showinfo("Security error", "You have entered
your password incorrectly at least once for 10 different users.
Please try again tomorrow..")
            else:
                response =
self.client.send_request(f"AMOUNT_OF_FAIL_ATTEMPTS;{ip}")
                if(response=="5"):
                    tkmb.showinfo("Security error", "You entered your
password incorrectly three times for five different users. Please try
again tomorrow.")
                else:
                    response =
self.client.send_request(f"CHECK_ATTEMPTS;{username};{ip}")
                    if(response=="3"):
                        tkmb.showinfo("Security error", "You have
entered your password incorrectly three times. Please try again
tomorrow.")

```

```

        else:
            response =
self.client.send_request(f"LOGIN;{username};{passwordHashes}")
            if response == "LOGIN_SUCCESS":
                tkmb.showinfo("First Login", "First login
was successful!")
                self.emailPassword(username)
            else:
                tkmb.showerror("Login", "Invalid
credentials")

self.client.send_request(f"UPDATE_ATTEMPTS;{username};{ip}")
        else:
            tkmb.showinfo("Login Error", "Please enter both username
and password")

def switch_to_signin(self):
    if not self.signin_mode:
        self.email_entry = ctk.CTkEntry(self.app,
placeholder_text="Email", font=(self.font, 20), corner_radius=40)
        self.email_entry.pack(pady=12, padx=10)
        self.loginButton.pack_forget()
        self.sighInButton.pack_forget()
        self.register_button.pack(pady=12, padx=10)
        self.back_button.pack(pady=12, padx=10)

        self.signin_mode = True

def switch_to_login(self):
    if self.signin_mode:
        self.email_entry.pack_forget()

        self.register_button.pack_forget()
        self.labelEmail.pack_forget()
        self.back_button.pack_forget()
        self.SecondRegisterButton.pack_forget()
        self.getPassword.pack_forget()

        self.user_entry.pack(pady=12, padx=10)
        self.user_pass.pack(pady=12, padx=10)
        self.loginButton.pack(pady=12, padx=10)
        self.sighInButton.pack(pady=12, padx=10)

        self.signin_mode = False

def register(self):
    username = self.user_entry.get()
    password = self.user_pass.get()
    email = self.email_entry.get()
    passwordHashes =
hashlib.sha256(password.encode()).hexdigest()
    self.usernameR=username
    self.passwordHashesR=passwordHashes
    self.emailR=email
    if self.checkDigit(username) or self.checkChars(username) or
self.checkChars(password) or self.checkChars(email):
        tkmb.showinfo("Length error", "The maximum number of
characters for a username is 8. Also you can not have , or ; in your

```

```

username, password or email.")
    return
    if username and password and email:
        response =
self.client.send_request(f"CHECK_REGISTRATION;{username}")
        if response == "ALL_RIGHT":
            tkmb.showinfo("First Register", "The first step in
the registration process was successful!")
            self.second_Register()
        elif response == "NOT_ALL_RIGHT":
            tkmb.showerror("Register", "Username already exists")
        else:
            tkmb.showerror("Register", "Server error")
    else:
        tkmb.showinfo("Register Error", "Please fill all fields")

def checkDigit(self,username):
    if len(username)>8:
        return True
    return False
def checkChars(self,username):
    if(username.find(',')!=-1 or username.find(';')!=-1):
        return True
    return False

def second_Register(self):
    self.user_entry.pack_forget()
    self.user_pass.pack_forget()
    self.loginButton.pack_forget()
    self.sighInButton.pack_forget()
    self.email_entry.pack_forget()
    self.register_button.pack_forget()
    self.back_button.pack_forget()
    self.labelEmail.pack(pady=12, padx=10)
    self.getPassword.pack(pady=12, padx=10)
    self.SecondRegisterButton.pack(pady=12, padx=10)
    self.back_button.pack(pady=12, padx=10)
    emailObj =
EmailSendingFile.GmailEmailSender(self.client.getAPIkey(),
self.client.getFromEmail(), self.emailR)
    self.passwordRegester = emailObj.send_email()

def second_Register_entery(self):
    passwordAtempted=self.getPassword.get()
    if str(self.passwordRegester) == passwordAtempted:

self.client.send_request(f"REGISTER;{self.usernameR};{self.passwordHa
shesR};{self.emailR}")
        tkmb.showinfo("Second Register", "The second step in the
registration process was successful!")
        self.on_close()
        ClientGUSecondPage.secondPageGUI(self.client,
self.usernameR)
    else:
        tkmb.showerror("second register", "The password is not
correct. Try again!")

```

```

def emailPassword(self, username):
    self.counter=0
    self.user_entry.pack_forget()
    self.user_pass.pack_forget()
    self.loginButton.pack_forget()
    self.signInButton.pack_forget()
    response = self.client.send_request(f"GET_EMAIL;{username}")
    self.username=username
    emailObj=
EmailSendingFile.GmailEmailSender(self.client.getAPIkey(),
self.client.getFromEmail(), response)
    self.passwordSecondLogin=emailObj.send_email()
    self.labelEmail.pack(pady=12, padx=10)
    self.getPassword.pack(pady=12, padx=10)
    loginButton = ctk.CTkButton(self.app, text="LOGIN",
command=self.secondLogIn, font=(self.font, 20), corner_radius=40,
fg_color=self.button_color, hover_color=self.hover_color)
    loginButton.pack(pady=12, padx=10)

def secondLogIn(self):
    passwordAttempted=self.getPassword.get()
    if self.counter<3:
        if str(self.passwordSecondLogin)==passwordAttempted:
            tkmb.showinfo("Login", "the login was successful!")
            self.on_close()
            ClientGUSecondPage.secondPageGUI(self.client,
self.username)
        else:
            tkmb.showinfo("failed Login", "the password is not
correct")
            self.counter=self.counter+1
        else:
            if self.counter==3:
                tkmb.showinfo("Try tomorrow",
                    "you have entered the wring password
for three times. please try again tomorrow")
                ip=self.client.getLocal_ip()
                self.client.send_request(f"FAILED_SECOND_LOGIN;{self.username};{ip}")
                self.counter=self.counter+1
            else:
                tkmb.showinfo("Try tomorrow",
                    "you have entered the wrng password for
three times. please try again tomorrow")

if __name__ == "__main__":
    ClientGUIFirstPage()

```

ClientGUSecondPage.py:

```

import customtkinter as ctk
import tkinter.messagebox as tkmb
import json
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from openai import OpenAI

class secondPageGUI():
    def __init__(self, client, username):
        self.client = client
        self.app = ctk.CTk()
        self.app.title("EASYBANK-CONNECTED")
        self.username = username
        self.font = "Georgia"
        self.button_color = "#2A3B27"
        self.hover_color = "#70805D"
        self.text_color = "#B5CDA3"
        self.frame_width = 600
        self.frame_height = 450
        self.statistics_canvas = None

        response =
self.client.send_request(f"AMOUNT_REQUESTED;{self.username}")
        self.amountOfRequested = int(response)

response=self.client.send_request(f"AMOUNT_TRANSFERS;{self.username}")
)
        self.amoutOfTransfers=int(response)

        self.creat()
        self.createButtoms()
        self.start_auto_refresh()
        self.app.mainloop()

    def creat(self):
        window_width, window_height = 1000, 600
        screen_width, screen_height = self.app.winfo_screenwidth(),
self.app.winfo_screenheight()
        x_position = (screen_width - window_width) // 2
        y_position = (screen_height - window_height) // 2 - 50

self.app.geometry(f"{window_width}x{window_height}+{x_position}+{y_po
sition}")
        ctk.set_appearance_mode("dark")

        self.app.resizable(False, False)

        balance =
self.client.send_request(f"GET_BALANCE;{self.username}")
        self.balance_label = ctk.CTkLabel(self.app, text=f"Balance:
${int(balance):,.2f}", font=(self.font, 60, "bold"),
text_color=self.text_color)
        self.balance_label.place(relx=0.5, rely=0.12,
anchor="center")

```

```

        self.username_label = ctk.CTkLabel(self.app, text=f"username:
{self.username}", font=(self.font, 30,"underline"),
text_color=self.text_color)
        self.username_label.place(relx=0.02, rely=0.02)

    def createButtons(self):
        self.frame_transfer = ctk.CTkFrame(self.app,
width=self.frame_width, height=self.frame_height, corner_radius=20)
        ctk.CTkLabel(self.frame_transfer, text="Transfer Page",
font=(self.font, 28,"bold")).place(relx=0.5,
rely=0.1,anchor="center")

        self.transfer_to = ctk.CTkEntry(self.frame_transfer,
width=250,corner_radius=40,font=(self.font, 20),
placeholder_text="Recipient Username")
        self.transfer_to.place(relx=0.5, rely=0.3, anchor="center")
        self.amount_of_money = ctk.CTkEntry(self.frame_transfer,
placeholder_text="Amount to
Transfer",corner_radius=40,font=(self.font, 20), width=250)
        self.amount_of_money.place(relx=0.5, rely=0.45,
anchor="center")
        self.button_send = ctk.CTkButton(self.frame_transfer,
text="Send Transfer",
command=self.send_transfer,corner_radius=40,fg_color=self.button_color,
hover_color=self.hover_color, font=(self.font, 20))
        self.button_send.place(relx=0.5, rely=0.80, anchor="center")
        self.transferB = ctk.CTkButton(self.app, font=(self.font,
20), text="Transfer", width=200, corner_radius=40,height=40,
fg_color=self.button_color,
hover_color=self.hover_color, command=self.transferWasPicked)
        self.transferB.place(relx=0.05, rely=0.35, anchor="w")

        self.comboboxTransfer = ctk.CTkComboBox(self.frame_transfer,
values=["Food",
"Entertainment", "Education", "Shopping", "Others"],
width=200, height=40,
corner_radius=20, fg_color=self.button_color,
button_hover_color=self.hover_color, border_color=self.button_color,
border_width=2,
text_color="#F0F8FF")

        self.comboboxTransfer.set("Select reason")
        self.comboboxTransfer.place(relx=0.5, rely=0.60,
anchor="center")

        self.frame_request = ctk.CTkFrame(self.app,
width=self.frame_width, height=self.frame_height, corner_radius=20)
        ctk.CTkLabel(self.frame_request, text="Transfer Request
Page", font=(self.font, 28,"bold")).place(relx=0.5,
rely=0.1,anchor="center")
        self.requestB = ctk.CTkButton(self.app, text="Transfer
request", font=(self.font, 20), width=200,
height=40,corner_radius=40,
fg_color=self.button_color,
hover_color=self.hover_color,command=self.requestWasPicked)
        self.requestB.place(relx=0.05, rely=0.47, anchor="w")

        self.requestFrom = ctk.CTkEntry(self.frame_request,
placeholder_text="Requested Username", width=250,

```

```

corner_radius=40,
font=(self.font, 20))
self.requestFrom.place(relx=0.5, rely=0.3, anchor="center")
self.amount_money_requested =
ctk.CTkEntry(self.frame_request, placeholder_text="Amount to
request", width=250,
corner_radius=40,
font=(self.font, 20))
self.amount_money_requested.place(relx=0.5, rely=0.45,
anchor="center")
self.button_Request = ctk.CTkButton(self.frame_request,
text="Request Transfer", command=self.sendRequest,
corner_radius=40,
fg_color=self.button_color, hover_color=self.hover_color,
font=(self.font, 20))
self.button_Request.place(relx=0.5, rely=0.80,
anchor="center")

self.comboboxRequest = ctk.CTkComboBox(self.frame_request,
values=["Food",
"Entertainment", "Education", "Shopping", "Others"],
width=200, height=40,
corner_radius=20, fg_color=self.button_color,
button_hover_color=self.hover_color, border_color=self.button_color,
border_width=2,
text_color="#F0F8FF")
self.comboboxRequest.set("Select reason")
self.comboboxRequest.place(relx=0.5, rely=0.60,
anchor="center")

self.frame_permissions = ctk.CTkFrame(self.app,
width=self.frame_width, height=self.frame_height, corner_radius=20)
ctk.CTkLabel(self.frame_permissions, text="Permission to
transfer Page", font=(self.font, 28, "bold")).place(relx=0.5,
rely=0.1, anchor="center")
self.approveB = ctk.CTkButton(self.app, text="Approve
transfer", font=(self.font, 20), width=200, height=40,
corner_radius=40,
fg_color=self.button_color, hover_color=self.hover_color,
command=self.approveWasPicked)
self.approveB.place(relx=0.05, rely=0.59, anchor="w")
self.scroll_frame_aprove =
ctk.CTkScrollableFrame(master=self.frame_permissions, width=450,
height=300, corner_radius=20)
self.scroll_frame_aprove.place(relx=0.5, rely=0.55,
anchor="center")

self.frame_previous = ctk.CTkFrame(self.app,
height=self.frame_height, width=self.frame_width, corner_radius=20)
ctk.CTkLabel(self.frame_previous, font=(self.font,
28, "bold"), text="Previous Transfers Page").place(relx=0.5, rely=0.1,
anchor="center")
self.scroll_frame =
ctk.CTkScrollableFrame(master=self.frame_previous, height=300,
width=450, corner_radius=20)
self.scroll_frame.place(relx=0.5, rely=0.55, anchor="center")
self.previousB = ctk.CTkButton(self.app, text="Previous

```



```

Transfers", font=(self.font, 20), width=200, height=40,
corner_radius=40,
                                fg_color=self.button_color,
hover_color=self.hover_color, command=self.previousWasPicked)
    self.previousB.place(relx=0.05, rely=0.71, anchor="w")

    self.statisticsB = ctk.CTkButton(self.app, text="Reports",
font=(self.font, 20), width=200, height=40,
                                corner_radius=40,
                                fg_color=self.button_color,
hover_color=self.hover_color,
command=self.statisticsWasPicked)
    self.statisticsB.place(relx=0.05, rely=0.83, anchor="w")

    self.frame_statistics = ctk.CTkFrame(self.app,
height=self.frame_height, width=self.frame_width,
                                corner_radius=20)
    ctk.CTkLabel(self.frame_statistics, font=(self.font, 28,
"bold"), text="Annual Expense Report Page").place(relx=0.5,
rely=0.1,
anchor="center")
    self.comboboxStatisticsCategory =
ctk.CTkComboBox(self.frame_statistics,
values=["Food", "Entertainment", "Education", "Shopping", "Others"],
width=200,
height=40, corner_radius=20, fg_color=self.button_color,
button_hover_color=self.hover_color, border_color=self.button_color,
border_width=2,
text_color="#F0F8FF", command=lambda _:
self.show_graph(self.comboboxStatisticsYears.get()))
    self.comboboxStatisticsCategory.set("Select a category")
    self.comboboxStatisticsCategory.place(relx=0.5, rely=0.36,
anchor="center")

response=self.client.send_request(f"GET_YEARS;{self.username}")
years = json.loads(response)

    self.comboboxStatisticsYears=
ctk.CTkComboBox(self.frame_statistics,
values=years,
width=200,
height=40, corner_radius=20,
fg_color=self.button_color,
button_hover_color=self.hover_color,
border_color=self.button_color,
border_width=2,
text_color="#F0F8FF", command=self.show_graph_year)

```

```

        self.comboboxStatisticsYears.set("Select a year")
        self.comboboxStatisticsYears.place(relx=0.5, rely=0.23,
anchor="center")

        self.adviceB = ctk.CTkButton(self.frame_statistics, text="Get
an advice", command=self.get_advice,
corner_radius=40,
fg_color=self.button_color, hover_color=self.hover_color,
font=(self.font, 20))

        self.transerWasPicked()
        self.refresh()

    def hideFrames(self):
        self.frame_transfer.place_forget()
        self.frame_request.place_forget()
        self.frame_premissions.place_forget()
        self.frame_previous.place_forget()
        self.frame_statistics.place_forget()
        self.clear_fields_statistics()
        self.adviceB.place_forget()

    def changeColors(self):
        self.transferB.configure(fg_color=self.button_color)
        self.requestB.configure(fg_color=self.button_color)
        self.approveB.configure(fg_color=self.button_color)
        self.previousB.configure(fg_color=self.button_color)
        self.statisticsB.configure(fg_color=self.button_color)

    def transerWasPicked(self):
        self.clear_fields_Transfers()
        self.hideFrames()
        self.changeColors()
        self.transferB.configure(fg_color=self.hover_color)
        self.frame_transfer.place(x=350, y=130)

    def requestWasPicked(self):
        self.clear_fields_requests()
        self.hideFrames()
        self.changeColors()
        self.requestB.configure(fg_color=self.hover_color)
        self.frame_request.place(x=350, y=130)

    def approveWasPicked(self):
        self.hideFrames()
        self.changeColors()
        self.approveB.configure(fg_color=self.hover_color)
        self.frame_premissions.place(x=350, y=130)

    def previousWasPicked(self):
        self.hideFrames()
        self.changeColors()
        self.previousB.configure(fg_color=self.hover_color)
        self.frame_previous.place(x=350, y=130)

    def statisticsWasPicked(self):
        self.hideFrames()
        self.changeColors()
        self.statisticsB.configure(fg_color=self.hover_color)
        self.frame_statistics.place(x=350, y=130)
        response =

```

```

self.client.send_request(f"GET_YEARS;{self.username}")
years = json.loads(response)
self.comboboxStatisticsYears.configure(values=years)
self.comboboxStatisticsYears.set("Select a year")

def clear_fields_Transfers(self):
    self.transfer_to.delete(0, 'end')
    self.amount_of_money.delete(0, 'end')
    self.comboboxTransfer.set("Select reason")

def clear_fields_requests(self):
    self.requestFrom.delete(0, 'end')
    self.amount_money_requested.delete(0, 'end')
    self.comboboxRequest.set("Select reason")

def clear_fields_statistics(self):
    self.comboboxStatisticsYears.set("Select a year")
    self.comboboxStatisticsCategory.set("Select a category")
    if self.statistics_canvas is not None:
        self.statistics_canvas.get_tk_widget().destroy()
        self.statistics_canvas = None

def send_transfer(self, userTo=None, amount=None, reason=None):
    if userTo and amount and reason:
        userToTransfer = userTo
        amountToTransfer = amount
        reasonToTransfer = reason
    else:
        userToTransfer = self.transfer_to.get()
        amountToTransfer = self.amount_of_money.get()
        reasonToTransfer = self.comboboxTransfer.get()
        if userToTransfer and userToTransfer != self.username and
amountToTransfer and reasonToTransfer != "Select reason":
            if not amountToTransfer.isdigit():
                tkmb.showerror("Invalid number", "Please enter a
valid number")

                self.clear_fields_Transfers()
                return
            if (int(amountToTransfer) < 0):
                tkmb.showerror("Negative amount", "you need to
enter a positive amount of money.")
                self.clear_fields_Transfers()
                return

        if userToTransfer and userToTransfer!=self.username and
amountToTransfer and reasonToTransfer!="Select reason":
            response =
self.client.send_request(f"TRANSFER;{self.username};{userToTransfer};
{amountToTransfer};{reasonToTransfer}")
            if response == "TRANSFER_SUCCESS":
                tkmb.showinfo("TRANSFER", f"{amountToTransfer}$
transferred successfully!")
                self.clear_fields_Transfers()
                self.refresh()
            elif response == "NO_ACCOUNT":
                tkmb.showerror("no account", "There is no account
under this name")
            elif response=="NO MONEY":
                tkmb.showerror("no enough money", "There is no enough
money in your account")

```

```

        elif response == "INVALID_AMOUNT":
            tkmb.showerror("Invalid input", "Please enter a valid
number for amount.")
        else:
            tkmb.showerror("error", "There has been an error")
    else:
        self.clear_fields_Transfers()
        tkmb.showinfo("Login Error", "Please enter both username,
amount and reason to the transfer. The username can't be yours")

    def showAmount(self):
        new_balance =
self.client.send_request(f"GET_BALANCE;{self.username}")
        self.balance_label.configure(text=f"Balance:
${float(new_balance):,.2f}")

    def showPrevious(self):
        for widget in self.scroll_frame.winfo_children():
            widget.destroy()
        response =
self.client.send_request(f"PREVIOUS;{self.username}")
        if response!="NO_TRANSFERS":
            transfers = json.loads(response)
            reversedListResponse = transfers[::-1]
            for item in reversedListResponse:
                if(item[0]==f"{self.username}") :
                    label = ctk.CTkLabel(self.scroll_frame,
text=f"you sent {item[2]}$ to {item[1]} | {item[3]} | {item[5]}",
font=(self.font, 15,"bold"),text_color='#CA5655')
                    label.pack(anchor="w", padx=10, pady=4)
                else:
                    label = ctk.CTkLabel(self.scroll_frame,
text=f"{item[0]} sent to you {item[2]}$ | {item[3]} | {item[5]}",
font=(self.font, 15,"bold"),
                                text_color='#6A804F')
                    label.pack(anchor="w", padx=10, pady=4)

    def start_auto_refresh(self):
        self.refreshAutomatic()
        self.app.after(5000, self.start_auto_refresh)

    def refreshAutomatic(self):
        try:
            responseR =
self.client.send_request(f"AMOUNT_REQUESTED;{self.username}")
            responseT=self.client.send_request(f"AMOUNT_TRANSFERS;{self.username}
")
            if self.amountOfRequested != int(responseR) and
self.amoutOfTransfers!= int(responseT):
                self.amountOfRequested = int(responseR)
                self.amoutOfTransfers=int(responseT)
                self.refresh()
            elif self.amountOfRequested != int(responseR):
                self.amountOfRequested = int(responseR)
                self.refresh()
            elif self.amoutOfTransfers!= int(responseT):
                self.amoutOfTransfers = int(responseT)
                self.refresh()
        except Exception as e:

```

```

        print("Error in refreshAutomatic:", e)

    def refresh(self):
        self.showAmount()
        self.showPrevious()
        self.show_requests()

    def refreshRequests(self):
        self.show_requests()

    def sendRequest(self):
        requestedUser = self.requestFrom.get()
        amount_money_requested = self.amount_money_requested.get()
        reasonToTransfer = self.comboboxRequest.get()

        if requestedUser and requestedUser != self.username and
amount_money_requested and reasonToTransfer != "Select reason":
            if not amount_money_requested.isdigit():
                tkmb.showerror("Invalid number", "Please enter a
valid number")
                self.clear_fields_Transfers()
                return
            if(int(amount_money_requested)<0):
                tkmb.showerror("negetive amount", "you need to enter
a positive amount of money.")
                self.clear_fields_requests()
                return

            if requestedUser and requestedUser != self.username and
amount_money_requested and reasonToTransfer != "Select reason":
                response =
self.client.send_request(f"REQUEST;{self.username};{requestedUser};{a
mount_money_requested};{reasonToTransfer}")
                self.clear_fields_requests()
                if response=="REQUEST_SUCCESS":
                    tkmb.showinfo("REQUEST", f"{amount_money_requested}$
Requested successfully!")
                elif response=="INVALID_AMOUNT":
                    tkmb.showerror("Invalid input", "Please enter a valid
number for amount.")
                elif response == "NO_ACCOUNT":
                    tkmb.showerror("no account", "There is no account
under this name")
                else:
                    tkmb.showerror("error", "There has been an error")
            else:
                tkmb.showinfo("Login Error", "Please enter both username,
amount and reason to the transfer. The username can't be yours")

    def show_requests(self):
        for widget in self.scroll_frame_aprove.winfo_children():
            widget.destroy()
        response =
self.client.send_request(f"GET_REQUESTS;{self.username}")
        if response != "NO_REQUESTS":
            requests = json.loads(response)
            reversedListResponse = requests[::-1]
            for item in reversedListResponse:
                userWhoRequested = item[0]
                amount = item[2]
                ID_num = item[4]

```

```

        reason = item[5]
        frame = ctk.CTkFrame(self.scroll_frame_aprove)
        frame.pack(fill="x", pady=5, padx=10)

        request_message =
        ctk.CTkLabel(frame, text=f"{userWhoRequested} requested {amount}$ | {reason}", font=(self.font, 15, 'bold'))
        request_message.pack(side="left", padx=10)

        decline_button =
        ctk.CTkButton(frame, text="NO", width=40, height=35, font=(self.font, 14), fg_color="#CA5655", hover_color="#8B0000", command=lambda r=ID_num: self.decline(r))
        decline_button.pack(side="right", padx=5)

        approve_button =
        ctk.CTkButton(frame, text="OK", width=40, height=35, font=(self.font, 14), fg_color="#6A804F", hover_color="#357A38", command=lambda r=ID_num, u=userWhoRequested, a=amount, rea=reason: self.approve(r, u, a, rea))
        approve_button.pack(side="right", padx=5)

    def decline(self, ID_num):
        self.client.send_request(f"CHANGE_STATUS;{ID_num}")
        self.refreshRequests()

    def approve(self, ID_num, userWhoRequested, amount, reason):
        self.client.send_request(f"CHANGE_STATUS;{ID_num}")
        self.send_transfer(userWhoRequested, amount, reason)
        self.refresh()

    def show_graph(self, year):
        if self.comboboxStatisticsCategory.get() != "Select a category" and self.comboboxStatisticsYears.get() != "Select a year":
            if self.statistics_canvas is not None:
                self.statistics_canvas.get_tk_widget().destroy()
                self.statistics_canvas = None

        response=self.client.send_request(f"GET_YEAR_SPENDING;{self.username};{year};{self.comboboxStatisticsCategory.get()}")
        self.yearSpending = json.loads(response)
        months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

        fig = Figure(figsize=(5, 2.5), dpi=100)
        ax = fig.add_subplot(111)
        ax.bar(months, self.yearSpending, align='center')
        ax.set_xticks(range(len(months)))
        ax.set_xticklabels(months, ha='center')

        self.statistics_canvas = FigureCanvasTkAgg(fig,
        master=self.frame_statistics)
        self.statistics_canvas.draw()
        self.statistics_canvas.get_tk_widget().place(relx=0.5,
        rely=0.70, anchor="center")

        self.adviceB.place(relx=0.5, rely=0.95, anchor="center")
    else:
        tkmb.showerror("choose a category and a year", "Please select the year you would like to see the spending of.")

```

```

def show_graph_year(self, year):
    if self.comboboxStatisticsCategory.get() != "Select a category":
        self.show_graph(year)

def get_advice(self):
    clientAI = OpenAI(api_key=self.client.getAPI_key_AI())
    expenses=self.yearSpending
    category=self.comboboxStatisticsCategory.get()
    if(self.comboboxStatisticsYears.get()=="2025"):
        prompt = f"""
            I am building a banking management app. The user
has monthly expenses in the "{category}" category for a full year.
            The list contains 12 elements, one per month:
            {expenses}.

            Please analyze only the months that already
occurred – that is, ignore months with a value of 0 as they represent
future months.

            Summarize the trend (increase, decrease,
fluctuations), whether the spending is stable or concerning, and
provide personalized advice based on the existing expenses.
            Make it concise and useful.
            """
    else:
        prompt = f"""
            I am building a banking management app. The user has
monthly expenses in the "{category}" category for 12 months:
            {expenses}.

            Summarize the trend (increase, decrease, fluctuations),
whether the spending is stable or concerning, and provide
personalized advice based on the existing expenses.
            Make it concise and useful.
            """

    response = clientAI.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": "You are a financial
advisor in a budgeting app."},
            {"role": "user", "content": prompt}
        ]
    )
    self.show_advice(response.choices[0].message.content)

def show_advice(self, message):
    popup = ctk.CTkToplevel(self.app)
    popup.title(self.comboboxStatisticsCategory.get())
    popup_width = 500
    popup_height = 400

    app_x = self.app.wininfo_rootx()
    app_y = self.app.wininfo_rooty()
    app_width = self.app.wininfo_width()
    app_height = self.app.wininfo_height()

    x = app_x + (app_width - popup_width) // 2
    y = app_y + (app_height - popup_height) // 2

    popup.geometry(f"{popup_width}x{popup_height}+{x}+{y}")

    popup.transient(self.app)

```

```

        popup.grab_set()
        popup.focus()
        textbox = ctk.CTkTextbox(popup, wrap="word")
        textbox.insert("1.0", message)
        textbox.configure(state="disabled")
        textbox.pack(padx=20, pady=20, fill="both", expand=True)
        close_button = ctk.CTkButton(popup, text="Close",
        command=popup.destroy, corner_radius=40,
        fg_color=self.button_color,
        hover_color=self.hover_color, font=(self.font, 20))
        close_button.pack(pady=(0, 20))

```

ClientFile.py

```

import socket
import tkinter.messagebox as tkmb
import EncryptionFile
import random
import os
import sys
import tkinter as tk

class Client:
    def __init__(self, server_ip='192.168.1.124', port=1729):
        self.server_ip = server_ip
        self.port = port
        self.client = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.client.settimeout(5)

        clientDataPath=self.checkClientDataFile()
        with open(str(clientDataPath), 'r') as data:
            lines = [line.strip() for line in data]

        encrypt1=EncryptionFile.Encryption1(int(lines[0]),int(lines[1]))
        encrypt2 = EncryptionFile.Encryption2(lines[2].encode(),
lines[3].encode())
        encrypt3 = EncryptionFile.Encryption3(lines[4].encode())
        self.encrypt = [encrypt1, encrypt2,encrypt3]
        hostname = socket.gethostname()
        self.local_ip = socket.gethostbyname(hostname)
        self.API_key_email=lines[5]
        self.from_email=lines[6]
        self.API_key_AI=lines[7]
        self.connect()

    def connect(self):
        try:
            self.client.connect((self.server_ip, self.port))
            respond= self.client.recv(16384).decode()
            if not respond:
                tkmb.showerror("Connection Error", "Server closed the
connection unexpectedly.")
                if self.client:
                    self.client.close()
            sys.exit()

```



```

        radomNum = int(respond[0])
        respondCut = respond[1:]
        respondDec = self.encrypt[radomNum].decrypt(respondCut)
        if respondDec == "TOO_MANY_USERS":
            tkmb.showerror("Error", "Too many users connected.
Please try again later.")
            if self.client:
                self.client.close()
            sys.exit()
    except socket.error as e:
        tkmb.showerror("Connection Error", f"Could not connect to
server: {e}")
        if self.client:
            self.client.close()
        sys.exit()

def send_request(self, request):
    if self.client is None:
        tkmb.showerror("Error", "Not connected to server")
        if self.client:
            self.client.close()
        sys.exit()
    try:
        ranNum=self.getRandom()
        requestEncrypt=self.encrypt[ranNum].encrypt(request)
        requestEncryptToSend=f"{ranNum}"+requestEncrypt
        self.client.sendall(requestEncryptToSend.encode())
        respond= self.client.recv(16384).decode()
        if not respond:
            tkmb.showerror("Error", "The server closed the
connection.")
            if self.client:
                self.client.close()
            sys.exit()
        radomNum=int(respond[0])
        respondCut= respond[1:]
        respondDec=self.encrypt[radomNum].decrypt(respondCut)
        return respondDec
    except socket.error as e:
        tkmb.showerror("Error", f"Network error: {e}")
        if self.client:
            self.client.close()
        sys.exit()

def close(self):
    if self.client:
        self.client.close()

def getRandom(self):
    values = [0,1,2]
    return random.choice(values)

def getLocal_ip(self):
    return self.local_ip

def getAPIkey(self):
    return self.API_key_email

def getFromEmail(self):
    return self.from_email

```

```

def checkClientDataFile(self):
    clientDataFileName = "clientFileData.txt"
    directory = "."
    filePath = os.path.join(directory, clientDataFileName)
    everythingThere=True

files=["EncryptionFile.py", "ClientGUIFirstPage.py", "ClientGUSecondPage.py", "EmailSendingFile.py"]
for item in files:
    pyfilePath = os.path.join(directory, item)
    if not os.path.isfile(pyfilePath):
        everythingThere=False
    if os.path.isfile(filePath) and everythingThere:
        return filePath
    else:
        root = tk.Tk()
        root.withdraw()
        tkmb.showerror("Error", "YOU DONT HAVE ALL THE FILES. YOU CANT CONNECT WITHOUT ALL OF THEM !!!")
        if self.client:
            self.client.close()
        sys.exit()

def getAPI_key_AI(self):
    return self.API_key_AI

```

EmailSendingFile.py:

```

import smtplib
import random
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

class GmailEmailSender:
    def __init__(self, key, from_user, to_user):
        self.from_user = from_user
        self.app_key = key
        self.to_user = to_user

    def get_random(self):
        return random.randint(100000, 999999)

    def send_email(self):
        password = self.get_random()

        msg = MIMEMultipart()
        msg['From'] = self.from_user
        msg['To'] = self.to_user
        msg['Subject'] = 'Your One-Time Password for EASY-BANK'

        body = f"""\
Hello,

```

```

Your one-time password for EASY-BANK is: {password}
Please do not share it with anyone.

Best regards,
EASY-BANK
"""
    msg.attach(MIMEText(body, 'plain'))

    try:
        with smtplib.SMTP_SSL('smtp.gmail.com', 465) as server:
            server.login(self.from_user, self.app_key)
            server.send_message(msg)
    except Exception as e:
        print(f"Failed to send email: {e}")

    return password

```

EncryptionFile:

```

import base64
from cryptography.hazmat.primitives.ciphers import Cipher,
algorithms, modes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import padding

from Cryptodome.Cipher import DES # first pip uninstall crypto, pip
uninstall pycrypto, pip install pycryptodome
from Cryptodome.Util.Padding import pad, unpad
import binascii

"""
החמשה בטוב תלוי
from Crypto.Cipher import DES # first pip uninstall crypto, pip
uninstall pycrypto, pip install pycryptodome
from Crypto.Util.Padding import pad, unpad
import binascii

"""

class Encryptions():
    def decrypt(self):
        pass
    def encrypt(self):
        pass

class Encryption1(Encryptions):
    def __init__(self, key, mod):
        self.public_key_data = key
        self.mod_data = mod

    def decrypt(self, message):
        try:
            encrypted_message = list(map(int, message.split(',')))
        except ValueError:
            print(f"Invalid encrypted message part: {message}")
            return ""

        dec = [chr(pow(ch, self.public_key_data, self.mod_data)) for
ch in encrypted_message]
        output_data = ''.join(dec)

```

```

        return output_data

    def encrypt(self, message):
        enc = [pow(ord(ch), self.public_key_data, self.mod_data) for
ch in message]
        messageToReturn=', '.join(map(str, enc))
        return messageToReturn

class Encryption2(Encryptions):
    def __init__(self, key, iv):
        self.key = key
        self.iv = iv

    def encrypt(self, data):
        padder = padding.PKCS7(128).padder()
        padded_data = padder.update(data.encode('utf-8')) +
padder.finalize()
        cipher = Cipher(algorithms.AES(self.key), modes.CBC(self.iv),
backend=default_backend())
        encryptor = cipher.encryptor()
        ciphertext = encryptor.update(padded_data) +
encryptor.finalize()
        return base64.b64encode(ciphertext).decode('utf-8')

    def decrypt(self, ciphertext):
        ciphertext = base64.b64decode(ciphertext.encode('utf-8'))
        decryptor = Cipher(algorithms.AES(self.key),
modes.CBC(self.iv), backend=default_backend()).decryptor()
        decrypted_data = decryptor.update(ciphertext) +
decryptor.finalize()
        unpadder = padding.PKCS7(128).unpadder()
        unpadded_data = unpadder.update(decrypted_data) +
unpadder.finalize()
        return unpadded_data.decode('utf-8')

class Encryption3(Encryptions):
    def __init__(self, key):
        self.key = key

    def encrypt(self, message):
        cipher = DES.new(self.key, DES.MODE_CBC)
        padded_data = pad(message.encode(), DES.block_size)
        encrypted_data = cipher.encrypt(padded_data)
        return binascii.hexlify(cipher.iv +
encrypted_data).decode('utf-8')

    def decrypt(self, message):
        message = binascii.unhexlify(message.encode())
        iv = message[:8]
        encrypted_message = message[8:]
        cipher = DES.new(self.key, DES.MODE_CBC, iv)
        decrypted_data = unpad(cipher.decrypt(encrypted_message),
DES.block_size)
        return decrypted_data.decode('utf-8')

```

DatabaseFile:

```

import sqlite3
from datetime import date

class Database:
    def __init__(self, db_name='dataBaseTamarEyal.db'):
        self.db_name = db_name
        self.create_tables()

    def create_tables(self):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS users (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    username TEXT UNIQUE NOT NULL,
                    password TEXT NOT NULL,
                    email TEXT NOT NULL,
                    balance REAL DEFAULT 0.0
                )
            ''')
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS cyber_protection (
                    ip TEXT NOT NULL,
                    date TEXT NOT NULL,
                    amountOfAttempts INTEGER NOT NULL,
                    username TEXT NOT NULL,
                    ID INTEGER PRIMARY KEY AUTOINCREMENT
                )
            ''')
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS requests (
                    personWhoRequested TEXT NOT NULL,
                    requestedFrom TEXT NOT NULL,
                    amountRequested INTEGER NOT NULL,
                    treated TEXT NOT NULL,
                    ID INTEGER PRIMARY KEY AUTOINCREMENT,
                    reasonForTransfer TEXT NOT NULL,
                    FOREIGN KEY(personWhoRequested) REFERENCES
users(username),
                    FOREIGN KEY(requestedFrom) REFERENCES
users(username)
                )
            ''')
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS transfers (
                    fromUser TEXT NOT NULL,
                    toUser TEXT NOT NULL,
                    amount INTEGER NOT NULL,
                    date TEXT NOT NULL,
                    ID INTEGER PRIMARY KEY AUTOINCREMENT,
                    reasonForTransfer TEXT NOT NULL,
                    FOREIGN KEY(toUser) REFERENCES users(username)
                )
            ''')
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS communication (
                    ID INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

        ip TEXT NOT NULL,
        status TEXT NOT NULL,
        date TEXT NOT NULL
    )
    '''
    conn.commit()

    def insert_user(self, username, password, email):
        try:
            with sqlite3.connect(self.db_name) as conn:
                cursor = conn.cursor()
                cursor.execute('INSERT INTO users (username,
password, email, balance) VALUES (?, ?, ?, ?)', (username, password,
email, 0))
                conn.commit()
                return True
        except sqlite3.IntegrityError:
            return False

    def checkRegistration(self, username):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT COUNT(*) FROM users WHERE username
= ?', (username,))
            amount = cursor.fetchall()
            if amount[0][0]==0:
                return "ALL_RIGHT"
            else:
                return "NOT_ALL_RIGHT"

    def is_correct_password(self, username, password):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT password FROM users WHERE username
= ?', (username,))
            stored_password = cursor.fetchone()
            if stored_password and password == stored_password[0]:
                return 'LOGIN_SUCCESS'
            return 'LOGIN_FAILED'

    def getBalance(self, username):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT balance FROM users WHERE username
= ?', (username,))
            balance = cursor.fetchone()
            return balance[0] if balance is not None else 0.0

    def isExists(self, username):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT username FROM users WHERE username
= ?', (username,))
            isThereUsername = cursor.fetchone()
            if isThereUsername:
                return True
            return False

    def changeBalance(self, amountUser, amountToSend, User, userToSend):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute("UPDATE users SET balance = ? WHERE

```

```

username = ?", (amountUser, User))
        cursor.execute("UPDATE users SET balance = ? WHERE
username = ?", (amountToSend, userToSend))
        conn.commit()

    def insert_transfer(self, userWhoSent, userToSend, amount, reason):
        try:
            with sqlite3.connect(self.db_name) as conn:
                today = date.today()
                cursor = conn.cursor()
                cursor.execute('INSERT INTO transfers (fromUser,
toUser, amount, date, reasonForTransfer) VALUES (?, ?, ?, ?, ?)',
                                (userWhoSent, userToSend,
amount, today, reason))
                conn.commit()
            return True
        except sqlite3.IntegrityError:
            return False

    def
insert_requests(self, userWhoRequested, userRequestedFrom, amount, reason):
        try:
            with sqlite3.connect(self.db_name) as conn:
                cursor = conn.cursor()
                cursor.execute('INSERT INTO requests
(personWhoRequested, requestedFrom, amountRequested,
treated, reasonForTransfer) VALUES (?, ?, ?, "False", ?)',
                                (userWhoRequested, userRequestedFrom,
amount, reason))
                conn.commit()
            return True
        except sqlite3.IntegrityError:
            return False

    def getTransfer(self, username):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT * FROM transfers WHERE fromUser =
? OR toUser = ?', (username, username))
            transfers = cursor.fetchall()
            return transfers

    def get_requests(self, username):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT
personWhoRequested, requestedFrom, amountRequested, treated,
ID, reasonForTransfer FROM requests WHERE requestedFrom = ? AND
treated=?', (username, "False" ))
            requests = cursor.fetchall()
            return requests

    def change_status(self, ID_num):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute("UPDATE requests SET treated = ? WHERE ID
= ?", ("True", ID_num))

```

```

        conn.commit()

    def getAmountOfRequests(self, username):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT COUNT(*) FROM requests WHERE requestedFrom = ?', (username,))
            amount = cursor.fetchall()
            return amount[0][0] if amount else 0

    def getAmountTransfers(self, username):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT COUNT(*) FROM transfers WHERE toUser = ?', (username,))
            amount = cursor.fetchall()
            return amount[0][0] if amount else 0

    def getYears(self, username):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT date FROM transfers WHERE fromUser = ?', (username,))
            expenses = cursor.fetchall()
            years = []
            for item in expenses:
                year = item[0].split("-")[0]
                if year not in years:
                    years.append(year)
            years.sort()
            return years

    def getYearSpending(self, username, year, category):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT amount, date, reasonForTransfer FROM transfers WHERE fromUser = ?', (username,))
            expenses = cursor.fetchall()
            month = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
            for item in expenses:
                if category == item[2]:
                    yearOfTransfer = item[1].split("-")[0]
                    if yearOfTransfer == year:
                        numOfMonth = item[1].split("-")[1]
                        numOfMonthInt = int(numOfMonth)
                        month[numOfMonthInt - 1] = month[numOfMonthInt - 1] + item[0]

            return month

    def checkAttempts(self, username, ip):
        with sqlite3.connect(self.db_name) as conn:
            today = date.today()
            cursor = conn.cursor()
            cursor.execute('SELECT amountOfAttempts FROM cyber_protection WHERE ip = ? AND username = ? AND date=?', (ip, username, today))
            amount = cursor.fetchall()
            if amount:
                return str(amount[0][0])
            else:

```



```

        return 0

    def updateDate(self, username, ip):
        with sqlite3.connect(self.db_name) as conn:
            today = date.today()
            cursor = conn.cursor()
            cursor.execute('SELECT * FROM cyber_protection WHERE ip = ? AND username = ?', (ip, username))
            allData = cursor.fetchall()
            if allData:
                dateA = allData[0][1]
                amountOfAttempts = allData[0][2]
                if (today == dateA):
                    totalAttempts = amountOfAttempts + 1
                    cursor.execute("UPDATE cyber_protection SET amountOfAttempts = ? WHERE ip = ? AND username = ?", (totalAttempts, ip, username))
                    conn.commit()
                else:
                    cursor.execute("UPDATE cyber_protection SET amountOfAttempts = ?, date = ? WHERE ip = ? AND username = ?", (1, today, ip, username))
                    conn.commit()
            else:
                cursor.execute('INSERT INTO cyber_protection (ip, date, amountOfAttempts, username) VALUES (?, ?, ?, ?)', (ip, today, "1", username))
                conn.commit()
            return "UPDATED"

    def checkAmountFailAttempts(self, ip):
        with sqlite3.connect(self.db_name) as conn:
            today = date.today()
            cursor = conn.cursor()
            cursor.execute('SELECT COUNT(*) FROM cyber_protection WHERE ip = ? AND date = ? AND amountOfAttempts = ?', (ip, today, "3"))
            amount = cursor.fetchall()
            return amount[0][0] if amount else 0

    def checkAllAmountFailAttempts(self, ip):
        with sqlite3.connect(self.db_name) as conn:
            today = date.today()
            cursor = conn.cursor()
            cursor.execute('SELECT COUNT(*) FROM cyber_protection WHERE ip = ? AND date = ?', (ip, today))
            amount = cursor.fetchall()
            return amount[0][0] if amount else 0

    def getEmail(self, username):
        with sqlite3.connect(self.db_name) as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT email FROM users WHERE username = ?', (username,))
            email = cursor.fetchall()
            return email[0][0] if email else None

```

```

def failedSecondLogIn(self,username,ip):
    with sqlite3.connect(self.db_name) as conn:
        today = date.today()
        cursor = conn.cursor()
        cursor.execute('SELECT * FROM cyber_protection WHERE
username = ? AND ip = ?',
                        (username,ip))
        accountUser = cursor.fetchall()
        if accountUser:
            cursor.execute("UPDATE cyber_protection SET
amountOfAttempts = ? WHERE ip = ? AND username = ?",
                            (3, ip, username))
            conn.commit()
        else:
            cursor.execute(
                'INSERT INTO cyber_protection (ip, date,
amountOfAttempts, username) VALUES (?, ?, ?,?)',
                (ip, today, 3, username))
            conn.commit()
            return "UPDATED"

def insertCommunication(self,ip):
    with sqlite3.connect(self.db_name) as conn:
        today=date.today()
        cursor = conn.cursor()
        cursor.execute('INSERT INTO communication (ip, status,
date) VALUES (?, ?, ?)',
                        (ip, "ON",today))
        conn.commit()

def changStatusCommunication(self,ip):
    with sqlite3.connect(self.db_name) as conn:
        cursor = conn.cursor()
        cursor.execute("UPDATE communication SET status = ? WHERE
ip = ? ",
                        ("OFF", ip))
        conn.commit()

def checkHowManyConected(self):
    with sqlite3.connect(self.db_name) as conn:
        cursor = conn.cursor()
        cursor.execute('SELECT COUNT(*) FROM communication WHERE
status = ?',
                        ("ON",))
        amount = cursor.fetchone()
        return amount[0] if amount else 0

```

ServerFile:

```

import socket
import threading
import DatabaseFile
import EncryptionFile
import json
import random

```

```

import os
import sys
import time

"""
2. אש חומת (Firewall)
אפשרה שלא לתוכנה נכנסים חיבורים לחסום היא המחדל ברירת, Windows-ב
פתרון:
"Windows Defender Firewall" > Allow an app through firewall
לכך
Public ו Private גם (python.exe) לפייתון גישה אפשרי
Inbound Rules: ידנית 1729 הפורט את תפתחי, לחלופין או
Advanced Settings → Inbound Rules → New Rule.
Allow the connection. מכן ולאחר, TCP 1729 ואז, Port בחר
"""

class Server:
    def __init__(self, host='192.168.1.124', port=1729):
        self.server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.server.bind((host, port))
        self.server.listen()
        self.database = DatabaseFile.Database()
        self.clients = []
        self.lock = threading.Lock()

        serverDataPath=self.checkClientDataFile()
        with open(str(serverDataPath), 'r') as data:
            lines = [line.strip() for line in data]
            encrypt1= EncryptionFile.Encryption1(int(lines[0]),
int(lines[1]))
            encrypt2= EncryptionFile.Encryption2(lines[2].encode(),
lines[3].encode())
            encrypt3= EncryptionFile.Encryption3(lines[4].encode())
            self.encrypt=[encrypt1,encrypt2,encrypt3]

    def start(self):
        while True:
            try:
                client, address = self.server.accept()
                if self.checkHowManyConected()<4:
                    self.clients.append(client)
                    self.sendClient(client, "CONNECTION_SUCCESS")
                    time.sleep(0.1)
                    threading.Thread(target=self.actionSelection,
args=(client,address)).start()
            else:
                self.sendClient(client, "TOO_MANY_USERS")
                time.sleep(0.1)
                client.close()
        except Exception as e:
            print(f"Error accepting connection: {e}")

```

```

def actionSelection(self, client, address):
    ipClient=-1
    try:
        ipClient = address[0]
        self.insertCommunication(ipClient)
        while True:
            request = client.recv(16384).decode().strip()
            if not request:
                break
            radomNum = int(request[0])
            respondCut = request[1:]
            respondDec =
self.encrypt[radomNum].decrypt(respondCut)
            messageSent = respondDec.split(";")
            action=messageSent[0]
            if action == "LOGIN":
                self.login(client, messageSent[1],
messageSent[2])
            elif action == "REGISTER":
                self.register(client, messageSent[1],
messageSent[2], messageSent[3])
            elif action == "TRANSFER":
self.transfer(client, messageSent[1], messageSent[2], messageSent[3],
messageSent[4])
            elif action=="GET_BALANCE":
                self.returnBalnce(client, messageSent[1])
            elif action=="PREVIOUS":
                self.returnPrevious(client, messageSent[1])
            elif action=="REQUEST":
self.requests(client, messageSent[1], messageSent[2], messageSent[
3], messageSent[4])
            elif action=="GET_REQUESTS":
                self.get_requests(client, messageSent[1])
            elif action=="CHANGE_STATUS":
                self.change_status(client, messageSent[1])
            elif action=="AMOUNT_REQUESTED":
                self.getAmountRequest(client, messageSent[1])
            elif action=="AMOUNT_TRANSFERS":
                self.getAmountOfTransfers(client, messageSent[1])
            elif action=="GET_YEARS":
                self.getYears(client, messageSent[1])
            elif action=="GET_YEAR_SPENDING":
self.getYearSpending(client, messageSent[1], messageSent[2], messageS
ent[3])
            elif action == "CHECK_ATTEMPTS":
self.check_Attempts(client, messageSent[1], messageSent[2])
            elif action == "UPDATE_ATTEMPTS":
self.update_attempts(client, messageSent[1], messageSent[2])
            elif action=="AMOUNT_OF_FAIL_ATTEMPTS":
self.checkAmountFailAttempts(client, messageSent[1])
            elif action=="ALL_FAIL_ATTEMPTS":
self.checkAllAmountFailAttempts(client, messageSent[1])
            elif action=="GET_EMAIL":
                self.getEmail(client, messageSent[1])

```

```

        elif action=="FAILED_SECOND_LOGIN":
self.failedSecondLogIn(client,messageSent[1],messageSent[2])
        elif action=="CHECK_REGISTRATION":
            self.checkRegistration(client,messageSent[1])
        else:
            message="UNKNOWN_ACTION"
            self.sendClient(client,message)
    except (ConnectionResetError, BrokenPipeError):
        self.changStatusCommunication(ipClient)
    finally:
        with self.lock:
            if client in self.clients:
                self.clients.remove(client)
            client.close()
            self.changStatusCommunication(ipClient)

def sendClient(self,client, message):
    ranNum = self.getRandom()
    requestEncript = self.encrypt[ranNum].encrypt(message)
    requestEncriptToSend = f"{ranNum}" + requestEncript
    client.send(requestEncriptToSend.encode())

def getRandom(self):
    values = [0,1,2]
    return random.choice(values)

def checkRegistration(self,client,username):
    with self.lock:
        message=self.database.checkRegistration(username)
        self.sendClient(client,message)

def login(self, client, username, password):
    with self.lock:
        message = self.database.is_correct_password(username,
password)
        self.sendClient(client,message)

def register(self, client, username, password, email):
    with self.lock:
        success = self.database.insert_user(username, password,
email)
        message="REGISTER_SUCCESS"
        self.sendClient(client,message)

def
transfer(self,client,username,username_to_transfer,amount,reason):
    try:
        amount = int(amount)
    except ValueError:
        message="INVALID_AMOUNT"
        self.sendClient(client,message)
        return

    with self.lock:
        if self.database.getBalance(username) < amount:
            message = "NO MONEY"

```

```

        self.sendClient(client, message)
        return
    if not self.database.isExists(username_to_transfer):
        message = "NO_ACCOUNT"
        self.sendClient(client, message)
        return
    amountUser = self.database.getBalance(username) - amount
    amountOther =
self.database.getBalance(username_to_transfer) + amount
    self.database.changeBalance(amountUser, amountOther,
username, username_to_transfer)

self.database.insert_transfer(username,username_to_transfer,amount,reason)

    message = "TRANSFER_SUCCESS"
    self.sendClient(client, message)

def returnBalnce(self,client,username):
    with self.lock:
        message = str(self.database.getBalance(username))
        self.sendClient(client, message)

def returnPrevious(self,client,username):
    with self.lock:
        transfers = self.database.getTransfer(username)
        if transfers is None:
            message = "NO_TRANSFERS"
            self.sendClient(client, message)
        else:
            message =json.dumps(transfers)
            self.sendClient(client, message)

def requestions(self,client, userWhoRequested,userRequsetedFrom,
amount,reason):
    try:
        amount = float(amount)
    except ValueError:
        message = "INVALID_AMOUNT"
        self.sendClient(client, message)
        return
    with self.lock:
        if not self.database.isExists(userRequsetedFrom):
            message = "NO_ACCOUNT"
            self.sendClient(client, message)
            return
        self.database.intert_requests(userWhoRequested,
userRequsetedFrom, amount,reason)
        message = "REQUEST_SUCCESS"
        self.sendClient(client, message)

def get_requests(self,client,userRequestedFrom):
    with self.lock:
        requests = self.database.get_requests(userRequestedFrom)
        if requests is None:
            message = "NO_REQUESTS"
            self.sendClient(client, message)
        else:
            message = json.dumps(requests)
            self.sendClient(client, message)

def change_status(self,client,ID_num):

```

```
        with self.lock:
            self.database.change_status(ID_num)
            message = "STATUS_CHANGED"
            self.sendClient(client, message)

    def getAmountRequest(self, client, userRequestedFrom):
        with self.lock:
            amount=self.database.getAmountOfRequests(userRequestedFrom)
            self.sendClient(client, str(amount))

    def getAmountOfTransfers(self, client, userTrasferTo):
        with self.lock:
            amount=self.database.getAmountTransfers(userTrasferTo)
            self.sendClient(client, str(amount))

    def getYears(self, client, userwhoTransferred):
        with self.lock:
            years=self.database.getYears(userwhoTransferred)
            message = json.dumps(years)
            self.sendClient(client, message)

    def getYearSpending(self, client, username, year, catagory):
        with self.lock:
            spending=self.database.getYearSpending(username, year, catagory)
            message = json.dumps(spending)
            self.sendClient(client, message)

    def check_Attempts(self, client, username, ip):
        with self.lock:
            amount=self.database.checkAttempts(username, ip)
            self.sendClient(client, str(amount))

    def update_attempts(self, client, username, ip):
        with self.lock:
            amount=self.database.updateDate(username, ip)
            self.sendClient(client, str(amount))

    def checkAmountFailAttempts(self, client, ip):
        with self.lock:
            amount = self.database.checkAmountFailAttempts(ip)
            self.sendClient(client, str(amount))

    def checkAllAmountFailAttempts(self, client, ip):
        with self.lock:
            amount = self.database.checkAllAmountFailAttempts(ip)
            self.sendClient(client, str(amount))

    def getEmail(self, client, username):
        with self.lock:
            email = self.database.getEmail(username)
            self.sendClient(client, email)

    def failedSecondLogIn(self, client, username, ip):
        with self.lock:
            respond = self.database.failedSecondLogIn(username, ip)
            self.sendClient(client, respond)
```

```
def insertCommunication(self, ip):
    with self.lock:
        self.database.insertCommunication(ip)

def changStatusCommunication(self, ip):
    with self.lock:
        self.database.changStatusCommunication(ip)

def checkHowManyConected(self):
    with self.lock:
        amount= self.database.checkHowManyConected()
    return amount

def checkClientDataFile(self):
    directory = "."
    serverDataFileName = "serverFileData.txt"
    serverFilePath = os.path.join(directory, serverDataFileName)
    databaseFileName = "dataBaseTamarEyal.db"
    databaseFilePath = os.path.join(directory, databaseFileName)
    databasePyFileName = "DatabaseFile.py"
    databasePyFilePath = os.path.join(directory,
databasePyFileName)
    if os.path.isfile(databaseFilePath) and
os.path.isfile(serverFilePath) and
os.path.isfile(databasePyFilePath):
        return serverFilePath
    else:
        sys.exit()

if __name__ == "__main__":
    server = Server()
    server.start()
```


serverFileData.txt:

1877

7373

omrsyvfdkvryznkdsfhodshfkcxjvowr

fycsofjisijvifpe

roshnctf

ClientFileData.txt:

5213

7373

omrsyvfdkvryznkdsfhodshfkcxjvowr

fycsofjisijvifpe

roshnctf

yvcf vppo pfeq ytjx

tamareyal07@gmail.com

sk-proj-

ZzCDG9xBgo5cB81bWSd2nk8XR6Ekx3EpVtfeT4g6hmwWaFncDrVxzfgIBd

QF8hMjXrw1Iz_6JT3BIbkFJlDdDc_zlfuFINDKJz9822rssXBFuBIUDRduu8hjr0

vk2TOTrSoBCTa5fzKS-ENYVCtU28W-V8A